

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	3
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ВВЕДЕНИЕ	8
1 Анализ предметной области	13
1.1 Задача распознавания ключевых слов для устройств интернета вещей	13
1.2 Предварительная обработка аудиосигнала	14
1.3 Классификация подходов к реализации систем распознавания ключевых слов	15
1.4 Каскадные подходы к снижению энергопотребления	19
1.5 Архитектуры нейронных сетей для распознавания ключевых слов ...	22
1.6 Методы квантизации нейронных сетей	23
1.7 Фреймворки для развёртывания моделей на микроконтроллерах	25
1.8 Выбор целевой платформы	26
1.9 Выводы и формулировка требований к решению	27
2 Проектирование каскадной архитектуры системы распознавания голосовых команд	31
2.1 Принцип каскадной детекции и формализация энергобюджета	31
2.2 Архитектура каскада из трёх ступеней	33
2.3 Критерий выбора порога ступени-предусловия	34
2.4 Архитектура нейросетевой модели и обоснование гиперпараметров ..	35
2.5 Методы квантизации применяемой модели	36
2.6 Выводы по разделу	39
3 Реализация системы и методика экспериментальной апробации	41
3.1 Программная архитектура и реализация каскада	41
3.2 Реализация тракта обработки звука	44
3.3 Реализация инференса нейросетевой модели	48
3.4 Методика исследования пространства архитектур	49
3.5 Лабораторный измерительный стенд	53
3.6 Методика экспериментальной апробации	55
3.7 Выводы по разделу	58
4 Анализ результатов	60
4.1 Систематическое исследование архитектур DS-CNN	60
4.2 Сравнение методов квантизации PTQ и QAT	63

4.3 Анализ времени инференса на платформе ESP32-S3	65
4.4 Выбор финальной модели по Парето-критерию энергия × точность ..	67
4.5 Качество классификации выбранной модели	72
4.6 Профиль энергопотребления полного цикла каскадной обработки	76
4.7 Декомпозиция энергобюджета цикла	78
4.8 Сравнительная оценка автономности	80
4.9 Дополнительные наблюдения и направления развития работы	81
4.10 Выводы по разделу	84
ЗАКЛЮЧЕНИЕ	86
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	91
ПРИЛОЖЕНИЕ А (Принципиальная электрическая схема лабораторного стенда)	94
ПРИЛОЖЕНИЕ Б (Диаграмма состояний конечного автомата)	96
ПРИЛОЖЕНИЕ В (Фотографии лабораторного стенда в различных режимах работы)	98
ПРИЛОЖЕНИЕ Г (Исходный код и репозитории проекта)	100
ПРИЛОЖЕНИЕ Д (Ключевые фрагменты программного обеспечения микроконтроллера)	101
ПРИЛОЖЕНИЕ Е (Ключевые фрагменты модулей обучения)	104
ПРИЛОЖЕНИЕ Ж (Таблица разделов флеш-памяти микроконтроллера)	107
ПРИЛОЖЕНИЕ И (Диаграмма последовательности сбора измерительных данных)	108

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

- AAD – Acoustic Activity Detection, обнаружение акустической активности
- ASIC – Application-Specific Integrated Circuit, специализированная интегральная схема
- BLE – Bluetooth Low Energy, беспроводной интерфейс малого энергопотребления
- CMSIS-NN – Cortex Microcontroller Software Interface Standard for Neural Networks, библиотека нейросетевых ядер Arm
- CNN – Convolutional Neural Network, свёрточная нейронная сеть
- CRNN – Convolutional Recurrent Neural Network, свёрточно-рекуррентная нейронная сеть
- DMA – Direct Memory Access, прямой доступ к памяти
- DS-CNN – Depthwise Separable Convolutional Neural Network, свёрточная нейронная сеть с разделимыми по глубине свёртками
- DSP – Digital Signal Processor, цифровой сигнальный процессор
- ESP-IDF – Espressif IoT Development Framework, фреймворк разработки для устройств Espressif
- ESP-NN – Espressif Neural Network kernels, библиотека ядер нейросетевого инференса
- ESP-SR – Espressif Speech Recognition, фреймворк распознавания речи Espressif
- EXT0, EXT1 – External wakeup sources, аппаратные источники пробуждения микроконтроллера из режима глубокого сна
- FAR – False Acceptance Rate, доля ложных срабатываний
- FN – False Negative, ложноотрицательное срабатывание
- FP32 – 32-bit floating-point, представление числа с плавающей точкой одинарной точности
- FPGA – Field-Programmable Gate Array, программируемая логическая интегральная схема
- FRR – False Rejection Rate, доля ложных отказов

GPIO – General-Purpose Input/Output, универсальный порт ввода-вывода

I2C – Inter-Integrated Circuit, двухпроводная последовательная шина

I2S – Inter-IC Sound, последовательный цифровой аудиоинтерфейс

INT8 – 8-bit signed integer, 8-битное целое со знаком

IoT – Internet of Things, интернет вещей

KWS – Keyword Spotting, распознавание ключевых слов

LDO – Low-Dropout regulator, линейный стабилизатор напряжения с малым падением

MAC – Multiply-Accumulate, операция умножения с накоплением

MCU – Microcontroller Unit, микроконтроллер

MEMS – Microelectromechanical Systems, микроэлектромеханические системы

MFCC – Mel-Frequency Cepstral Coefficients, мел-частотные кепстральные коэффициенты

MLPerf – Machine Learning Performance benchmark, эталонный набор тестов производительности машинного обучения

PSRAM – Pseudo-Static Random Access Memory, псевдостатическая оперативная память

PTQ – Post-Training Quantization, посттренировочная квантизация

QAT – Quantization-Aware Training, квантизация с учётом при обучении

ReLU – Rectified Linear Unit, выпрямленная линейная функция активации

RNN – Recurrent Neural Network, рекуррентная нейронная сеть

RTC – Real-Time Clock, часы реального времени

SIMD – Single Instruction Multiple Data, одна инструкция над множеством данных

SRAM – Static Random Access Memory, статическая оперативная память

STE – Straight-Through Estimator, метод сквозного прохождения градиента

TC-ResNet – Temporal Convolution Residual Network, остаточная сеть с временными свёртками

TFLM – TensorFlow Lite Micro, среда исполнения нейронных сетей для мик-

роконтроллеров

UART – Universal Asynchronous Receiver-Transmitter, универсальный асинхронный приёмопередатчик

ULP – Ultra-Low Power coprocessor, сопроцессор сверхмалой мощности

USB – Universal Serial Bus, универсальная последовательная шина

VAD – Voice Activity Detector, детектор голосовой активности

Xtensa LX7 – архитектура ядра процессоров ESP32-S3

БПФ – быстрое преобразование Фурье

ДКП – дискретное косинусное преобразование

ДПФ – дискретное преобразование Фурье

п.п. – процентные пункты

ПЛИС – программируемая логическая интегральная схема

ЦПУ – центральное процессорное устройство

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Глубокий сон – режим энергопотребления микроконтроллера, при котором основные вычислительные ядра, цифровая периферия и беспроводные модули обесточены, а активными остаются домен реального времени и сопроцессор сверхмалой мощности; пробуждение инициируется аппаратным событием на выводе RTC GPIO, таймером RTC или сигналом сопроцессора ULP

Декомпозиция энергобюджета – разложение интегральной энергии цикла обработки по вкладам отдельных этапов, позволяющее идентифицировать доминирующие компоненты и определить приоритеты дальнейшей оптимизации

Детектор голосовой активности – компонент аудиосистемы, отличающий периоды наличия речевого сигнала от периодов тишины или фонового шума при минимальных вычислительных затратах; в каскадной архитектуре служит первой ступенью — фильтром-предусловием для активации последующих ступеней

Двухдоменный измерительный стенд – архитектура лабораторной установки, в которой измеряемая система и система измерений электрически изолированы и питаются от независимых источников, что обеспечивает отсутствие паразитного влияния системы измерений на регистрируемое потребление

Каскадная архитектура детекции – организация системы непрерывного мониторинга в виде последовательности ступеней с возрастающей вычислительной сложностью и энергопотреблением, при которой каждая ступень активирует следующую только при срабатывании по своему критерию

Квантизация нейронной сети – процедура снижения разрядности представления весов и активаций модели, как правило — переход от 32-битного представления с плавающей точкой к 8-битному целочисленному

Мел-частотные кепстральные коэффициенты – спектрально-временное представление аудиосигнала, получаемое последовательным применением кратковременного преобразования Фурье, проекции амплитудного спектра на психоакустически мотивированную мел-шкалу, логарифмирования и дискретного косинусного преобразования

Распознавание ключевых слов – задача обнаружения заранее определённых

слов-команд в непрерывном аудиопотоке без полной транскрипции речи; частный случай распознавания речи с ограниченным словарём

Свёрточная сеть с разделимыми по глубине свёртками – архитектура свёрточной нейронной сети, в которой стандартная двумерная свёртка по входному тензору заменена двухступенчатой декомпозицией — пространственной свёрткой по каждому каналу независимо и линейной комбинацией каналов с ядром 1×1 — что обеспечивает сокращение числа параметров и операций умножения с накоплением в 8–10 раз по сравнению со стандартной свёрткой при сопоставимой точности классификации

Тензорная арена – статически выделяемая область оперативной памяти микроконтроллера, используемая фреймворком TensorFlow Lite Micro для размещения промежуточных тензоров нейросетевого инференса без обращения к динамической аллокации

ВВЕДЕНИЕ

Актуальность. Автономные устройства с голосовым управлением — носимая электроника, датчики промышленного мониторинга, беспроводные узлы умного дома — сталкиваются с общим архитектурным ограничением: конечным запасом энергии автономного источника. Микрофон должен функционировать в режиме непрерывного мониторинга, иначе устройство пропустит голосовую команду, однако постоянная работа аудиотракта на микроконтроллере приводит к разряду источника питания в течение единиц часов вместо месяцев и лет, ожидаемых от энергоэффективных IoT-устройств. Указанное противоречие между непрерывной готовностью к приёму команды и ограниченным энергетическим бюджетом определяет актуальность исследований в области энергоэффективных систем распознавания ключевых слов на периферии сети.

Альтернативой локальному распознаванию служат облачные голосовые ассистенты (Amazon Alexa, Google Assistant, Apple Siri), передающие аудиосигнал на удалённые серверы. Для батарейных устройств такой подход неприемлем по совокупности факторов: непрерывная передача аудиоданных по беспроводному каналу потребляет существенно больше энергии, чем локальная обработка [1]; устройство становится зависимым от наличия и качества сетевого соединения; передача голосовых данных создаёт риски для конфиденциальности пользователя. Единственным практически реализуемым вариантом для батарейных IoT-устройств остаётся выполнение распознавания ключевых слов (*Keyword Spotting*, KWS) непосредственно на микроконтроллере, то есть распознавание на периферии сети.

Степень разработанности. Задача распознавания ключевых слов на микроконтроллерах активно исследуется международным научным сообществом, и к настоящему моменту в литературе выделились три основных направления повышения энергоэффективности. Первое направление — разработка компактных нейросетевых архитектур, адаптированных к ресурсным ограничениям встраиваемых платформ [2; 3]. Второе — применение методов квантизации весов и активаций, обеспечивающих переход от вычислений с плавающей точкой к целочисленным операциям с использованием SIMD-расширений современных микроконтроллеров [4; 5]. Третье — построение каскадных схем детекции, в которых вычислительно затратный нейросете-

вой классификатор активируется не непрерывно, а лишь по событию, зафиксированному более дешёвым предварительным детектором речевой активности [6; 7]. Вопросы построения детекторов голосовой активности и компрессии моделей рассмотрены и в отечественной литературе [8—11]. Однако перечисленные направления в известной литературе изучаются преимущественно в изоляции друг от друга: их совместное применение на микроконтроллере общего назначения с количественной декомпозицией итогового энергобюджета по этапам обработки не охарактеризовано. Восполнение указанного пробела и составляет научно-техническую проблему настоящей работы.

Решаемая проблема. Практический выбор, перед которым оказывается разработчик автономного устройства с голосовым управлением, сводится к двум крайностям. Микроконтроллер общего назначения в режиме непрерывного мониторинга обеспечивает гибкость, доступность и развитую экосистему разработки, но демонстрирует потребление, неприемлемое для батарейного питания. Специализированный нейросетевой процессор (ASIC) демонстрирует энергоэффективность, превосходящую микроконтроллер на один-два порядка, но представляет собой закрытое решение с аппаратно фиксированной архитектурой модели, ограниченно доступное и не допускающее независимой модификации. Промежуточный вариант — каскадная архитектура с квантизованной моделью на микроконтроллере общего назначения — известен концептуально, однако количественно не охарактеризован: отдельный вклад каскадного управления режимами питания и квантизации модели в итоговое энергопотребление, как и вклад сервисных этапов цикла обработки (пробуждение, инициализация периферии, захват аудио, индикация результата) в общий бюджет энергии не определены.

Цель работы. Целью настоящей работы является снижение энергопотребления устройств интернета вещей с функцией голосового управления за счёт каскадного управления режимами работы микроконтроллера и применения квантизованных нейронных сетей для распознавания ключевых слов голосовых команд. Достижение указанной цели предполагает решение следующих задач:

- 1) проанализировать существующие методы и системы распознавания ключевых слов голосовых команд, выявить основные источни-

ки избыточного энергопотребления и сформулировать количественные требования к энергоэффективному решению по совокупности метрик (средняя мощность, энергия на детекцию, латентность, точность классификации, размер модели),

- 2) разработать каскадную архитектуру детекции из трёх этапов возрастающей вычислительной сложности с согласованным управлением режимами энергопотребления микроконтроллера, допускающую перенос на различные микроконтроллерные платформы,
- 3) исследовать влияние методов квантизации — посттренировочной (PTQ) и квантизации с дообучением (QAT) — на точность, размер и вычислительную сложность нейросетевой модели классификатора ключевых слов,
- 4) реализовать разработанную систему на платформе ESP32-S3 и провести её экспериментальную апробацию на лабораторном стенде с измерением тока и напряжения с разрешением единиц микроампер,
- 5) выполнить декомпозицию итогового энергобюджета цикла каскадной обработки по этапам и дать количественную оценку вклада каждого компонента архитектуры в общую экономию энергопотребления.

Объект и предмет. *Объектом* исследования является система распознавания ключевых слов голосовых команд на микроконтроллере общего назначения; *предметом* — методы снижения её энергопотребления за счёт каскадной архитектуры детекции и квантизации нейросетевой модели.

Научная новизна. Впервые получено количественное описание совместного применения каскадной архитектуры и квантизации нейросетевой модели на микроконтроллере общего назначения с отдельной оценкой вклада каждого метода в итоговое снижение энергопотребления. В отличие от существующих работ, рассматривающих указанные подходы изолированно, проведена покомпонентная декомпозиция энергобюджета цикла обработки на платформе ESP32-S3, позволяющая идентифицировать этапы с доминирующим вкладом в общий бюджет и определить приоритетные направления дальнейшей оптимизации. В рамках исследования впервые проведено сравнение методов посттренировочной квантизации и квантизации с дообучением применительно к архитектуре DS-CNN на платформе ESP32-S3 с одновре-

менной оценкой влияния метода квантизации на точность модели, её размер и латентность инференса в условиях реального аппаратного ускорения.

Основные результаты. В ходе работы получены следующие основные результаты:

- 1) разработана каскадная архитектура детекции из трёх ступеней возрастающей вычислительной сложности с согласованным управлением режимами питания микроконтроллера; экспериментально на платформе ESP32-S3 показано, что архитектура обеспечивает снижение средней потребляемой мощности в 4–9 раз относительно режима непрерывного мониторинга в зависимости от частоты голосовых событий;
- 2) установлено, что квантизация модели DS-CNN до целочисленного представления INT8 обеспечивает сжатие модели более чем в 3,5 раза при потере точности классификации, не превышающей статистической погрешности оценки на тестовой выборке; оба исследованных метода (PTQ и QAT) применимы к задаче распознавания ключевых слов на ресурсоограниченных платформах, а различия между ними лежат в пределах указанной погрешности;
- 3) выполнена покомпонентная декомпозиция энергобюджета каскадного цикла, установившая, что собственно вычислительные этапы (извлечение признаков и нейросетевой инференс) составляют менее четверти энергобюджета, тогда как доминирующий вклад вносят сервисные этапы; на основании декомпозиции определены приоритетные направления дальнейшей оптимизации системы;
- 4) разработан двухдоменный лабораторный измерительный стенд с электрической изоляцией измеряемой системы и системы измерений, обеспечивающий измерение энергопотребления с разрешением единиц микроампер и синхронизацию измерений с этапами работы программного обеспечения микроконтроллера.

Практическая значимость. Предложенная каскадная архитектура и методика измерения энергопотребления с микроамперным разрешением применимы при проектировании автономных IoT-устройств с голосовым управлением в задачах промышленного мониторинга (голосовое управление в условиях занятых рук оператора), носимой электроники (медицинские и

спортивные устройства) и беспроводных узлов умного дома. Архитектура не привязана к конкретной аппаратной платформе и переносима на любой микроконтроллер с поддержкой аппаратных режимов глубокого сна и внешнего пробуждения — семейства STM32, nRF52 и аналогичные. Результаты сравнительного исследования методов квантизации служат ориентиром при выборе стратегии оптимизации нейросетевых моделей для периферийных устройств, а методика двухдоменного измерительного стенда с изоляцией измеряемой системы и системы измерений применима для характеристики энергопотребления произвольных встраиваемых платформ.

Апробация и внедрение результатов. Предложенная каскадная архитектура и методика измерения энергопотребления доведены до работающего программно-аппаратного решения на платформе ESP32-S3 и охарактеризованы экспериментально на двухдоменном лабораторном стенде с разрешением единиц микроампер; методика и результаты приведены в разделах 3 и 4. Исходный код встроенного программного обеспечения и модулей обучения нейросетевых моделей размещён в открытом репозитории (приложение Г), что обеспечивает воспроизводимость полученных результатов.

Личный вклад автора. Все приведённые в работе результаты — проектирование каскадной архитектуры, программная реализация на платформе ESP32-S3, разработка лабораторного измерительного стенда, систематическое исследование архитектур нейросетевой модели и экспериментальная характеристика системы — получены автором самостоятельно.

1 Анализ предметной области

1.1 Задача распознавания ключевых слов для устройств интернета вещей

Голосовое управление устройствами интернета вещей (Internet of Things, IoT) предполагает наличие механизма обнаружения заранее определённых слов-команд в непрерывном аудиопотоке. В международной литературе данная задача обозначается термином *Keyword Spotting* (KWS) и представляет собой частный случай распознавания речи с ограниченным словарём — от единиц до нескольких десятков команд [2]. В отличие от полных систем автоматического распознавания речи, обрабатывающих произвольные фразы, системы KWS ориентированы на минимизацию вычислительных ресурсов и энергопотребления за счёт упрощения модели, что делает их пригодными для развёртывания непосредственно на ресурсоограниченных встраиваемых платформах [2; 12].

Практические сценарии применения KWS в IoT охватывают широкий класс задач: голосовая активация промышленных носимых устройств в условиях занятых рук оператора, речевое управление автономными роботизированными системами, активация охранных узлов по голосовой команде и управление бытовыми устройствами умного дома. Во всех перечисленных сценариях система должна находиться в состоянии постоянной готовности к приёму команды, что предполагает непрерывную работу звукового тракта: захват аудиосигнала, извлечение признаков и классификацию. При батарейном питании устройства данное требование вступает в прямое противоречие с ограниченным энергетическим бюджетом. Указанное противоречие между необходимостью непрерывного мониторинга и минимизацией среднего энергопотребления определяет актуальность разработки специализированных энергоэффективных архитектур KWS-систем.

В качестве стандартного бенчмарка для оценки и сравнения моделей KWS в научном сообществе используется открытый набор данных Google Speech Commands [13]. Набор версии 2 содержит около 105 000 однословных аудиозаписей длительностью в одну секунду, охватывающих 35 англоязычных команд, записанных более чем 2600 дикторами в различных акустических условиях. В стандартной конфигурации из 35 команд выделяют-

ся 10 целевых слов (*yes, no, up, down, left, right, on, off, stop, go*), отдельный класс тишины (*silence*) и класс неизвестных слов (*unknown*), формируя задачу классификации на 12 классов [13]. Именно данная конфигурация принята в бенчмарке MLPerf Tiny [14] в качестве эталонной для задачи KWS на микроконтроллерах и используется в настоящей работе.

1.2 Предварительная обработка аудиосигнала

Нейросетевые модели KWS, как правило, не обрабатывают сырой аудиосигнал непосредственно. Входным представлением служат спектрально-временные признаки, извлечённые из аудиопотока на этапе предобработки. Наиболее распространённым представлением являются мел-частотные кепстральные коэффициенты (Mel-Frequency Cepstral Coefficients, MFCC), предложенные Davis и Mermelstein в 1980 году [15] и до настоящего времени остающиеся основным признаковым пространством для задач распознавания речи на встраиваемых платформах.

Процедура извлечения MFCC включает сегментацию сигнала на перекрывающиеся фреймы, применение оконной функции, дискретное преобразование Фурье, проекцию амплитудного спектра на набор треугольных мел-фильтров, равномерно распределённых по психоакустически мотивированной шкале, логарифмирование энергий мел-фильтров и дискретное косинусное преобразование. Результатом является матрица фиксированного размера, в которой строки соответствуют временным фреймам, а столбцы — декоррелированным кепстральным коэффициентам. Подобная матрица может быть интерпретирована как одноканальное изображение, что обосновывает применение свёрточных нейронных сетей для последующей классификации. Конкретные параметры конвейера обработки (длина окна, шаг сегментации, количество мел-фильтров и сохраняемых коэффициентов), принятые в настоящей работе, рассматриваются в разделе 2.

Целесообразность совместного использования MFCC и свёрточных нейронных сетей подтверждается результатами в смежных задачах обработки речевого сигнала. В работе Муртазина и др. [9] представлен алгоритм выявления синтезированного голоса, основанный на сочетании кепстральных коэффициентов и свёрточной нейронной сети; авторы продемонстрировали, что подобная двухступенчатая комбинация — психоакустически мотивиро-

ванное преобразование признаков с последующей нейросетевой классификацией — обеспечивает высокую точность при ограниченной вычислительной сложности модели. Аналогичный подход экспериментально обоснован в работе Исхаковой и др. [10], где для задачи распознавания речевых эмоций показано, что снижение размерности признакового пространства с использованием свёрточной нейронной сети сохраняет основные классификационные свойства при существенном сокращении вычислительных затрат. Применительно к KWS данный подход является доминирующим, что обосновывает выбор пары MFCC и DS-CNN в настоящей работе.

Значимость этапа предобработки с точки зрения энергопотребления подчёркивается в работе Cerutti и др. [7]: экспериментально установлено, что в стандартном KWS-конвейере на микроконтроллере до 85% совокупной энергии расходуется на захват аудиосигнала и извлечение MFCC-признаков, тогда как на инференс нейронной сети приходится оставшиеся 15%. Данный результат указывает на то, что оптимизация предобработки может оказать большее влияние на энергоэффективность, чем оптимизация самой модели классификатора, и обосновывает необходимость анализа энергобюджета не на уровне модели, а на уровне полного цикла обработки — что и составляет одну из задач настоящего исследования.

1.3 Классификация подходов к реализации систем распознавания ключевых слов

Существующие подходы к построению систем KWS для IoT-устройств могут быть классифицированы по месту выполнения инференса (облако или периферия сети) и типу вычислительной платформы. Ниже рассматриваются основные варианты с акцентом на их пригодность для батарейных автономных устройств; сводное сравнение приведено в таблице 1.

1.3.1 Облачные решения

При облачном подходе устройство осуществляет захват аудиоданных и передачу их на удалённый сервер для инференса полноразмерной модели (Conformer, Whisper и подобные). К достоинствам относятся неограниченная сложность моделей, высокая точность и поддержка произвольного словаря. Вместе с тем для автономных IoT-устройств облачный подход об-

ладает существенными ограничениями. Передача аудиоданных по Wi-Fi требует поддержания активного радиоканала: согласно технической документации ESP32-S3 [1], потребление в пике передачи Wi-Fi достигает 240–355 мА при напряжении питания 3,3 В [1], что соответствует мгновенной мощности до 800 мВт; среднее потребление при активной передаче аудио составляет порядка 200–300 мВт. Помимо энергопотребления радиомодуля, облачный подход приносит сетевую латентность порядка 50–500 мс, зависит от наличия и качества сетевого подключения и ставит под вопрос приватность голосовых данных пользователя. По совокупности факторов облачный подход непригоден для батарейных автономных устройств и далее в работе не рассматривается.

1.3.2 Специализированные аппаратные платформы

ASIC (Application-Specific Integrated Circuit). Процессоры, разработанные исключительно для нейросетевого инференса, обеспечивают наивысшую энергоэффективность среди рассмотренных решений. Линейка Syntiant NDP100/NDP101/NDP120 реализует архитектуру постоянно активного прослушивания на базе специализированного вычислительного ядра. По данным бенчмарка MLPerf Tiny [14], данные процессоры демонстрируют потребление порядка десятков микроджоулей на один акт инференса, что на два-три порядка ниже, чем у микроконтроллеров общего назначения. Однако закрытая архитектура, невозможность модификации модели или аппаратной логики, высокая стоимость единичных образцов и ограниченная доступность делают ASIC неприменимыми в условиях независимого академического исследования, поскольку исключают возможность полного контроля над программно-аппаратной частью и независимого измерения вклада отдельных компонентов в энергобюджет.

FPGA (Field-Programmable Gate Array). Программируемые логические интегральные схемы позволяют реализовать аппаратные ускорители нейросетевых вычислений с полным контролем над микроархитектурой. В рамках бенчмарка MLPerf Tiny Duarte и др. [16] продемонстрировали развёртывание KWS-модели на FPGA-платформах Xilinx Pynq-Z2 и Arty A7-100T с использованием открытых фреймворков hls4ml и FINN, достигнув латентности менее 20 мкс и энергопотребления от 30 мкДж на один инференс. Сложность проектирования на ПЛИС иллюстрируется работой Ушениной и Да-

нилова [17], где даже разработка отдельного модуля искусственного нейрона для последовательно-параллельных архитектур нейронных сетей представляет самостоятельную инженерную задачу, требующую тщательной оптимизации последовательности операций умножения с накоплением и распределения ресурсов между арифметическими блоками и блочной памятью. При переходе от единичного нейрона к полной свёрточной сети уровня DS-CNN сложность проектирования возрастает на порядки. Кроме того, ПЛИС не содержат встроенных модулей беспроводной связности (Wi-Fi, Bluetooth), которые критически необходимы для IoT-устройств. Стандартные цифровые интерфейсы (I2S, I2C, SPI, UART) могут быть реализованы как конфигурируемые программные блоки («soft IP»), однако реализация полного стека Wi-Fi или Bluetooth на ПЛИС нецелесообразна по сложности и стоимости, что вынуждает использовать внешний микроконтроллер с радиомодулем — фактически возвращая в систему отдельную цифровую платформу общего назначения.

DSP (Digital Signal Processor). Цифровые сигнальные процессоры оптимизированы для потоковых вычислений: свёрток, быстрого преобразования Фурье и матричных операций с фиксированной точкой. Такие платформы (Cadence HiFi, TI C55xx, CEVA-BX2) активно применяются в коммерческих устройствах с интенсивной аудиообработкой — слуховых аппаратах, гарнитурах с активным шумоподавлением, мобильных SoC. Применительно к задаче KWS на одноканальном аудиовходе с батарейным питанием выделенный DSP избыточен по архитектуре: его сильные стороны проявляются на многоканальной потоковой обработке, тогда как одноканальный KWS не полностью утилизирует параллелизм DSP. Кроме того, аналогично ПЛИС, выделенный DSP не содержит встроенных модулей беспроводной связности и требует сопряжения с микроконтроллером, что увеличивает сложность системы. Современная тенденция — интеграция DSP-расширений (Cortex-M55 Helium, Xtensa LX7 PIE, RISC-V P-extension) непосредственно в основное ядро микроконтроллера — позволяет использовать DSP-функции без дополнительной микросхемы и в значительной мере покрывает потребности KWS-задач на встраиваемых платформах.

1.3.3 Микроконтроллеры общего назначения

Микроконтроллеры (MCU) с поддержкой целочисленных SIMD-расширений и аппаратных режимов энергосбережения представляют третий подход к реализации KWS на периферии сети. Инференс квантизованной нейросетевой модели и предобработка выполняются непосредственно на ЦПУ микроконтроллера. Среди платформ, применяемых для KWS, выделяются семейство ARM Cortex-M (STM32L4, STM32U3, STM32U5, STM32H7, nRF52840) с библиотекой CMSIS-NN для аппаратного ускорения целочисленных операций; платформы Xtensa (Espressif ESP32-S3) с двухъядерным процессором LX7, SIMD-регистрами шириной 128 бит и встроенными модулями Wi-Fi/BLE и аппаратным режимом глубокого сна с пробуждением по RTC GPIO [1]; платформы RISC-V (ESP32-C3, GreenWaves GAP8/GAP9) с открытым набором инструкций.

Типичное потребление микроконтроллера при активном инференсе KWS составляет 70–150 мВт [12], что на порядки выше показателей ASIC. Ключевым методом компенсации данного разрыва является каскадная архитектура: микроконтроллер большую часть времени находится в режиме глубокого сна (потребление порядка единиц микроампер), просыпается по внешнему событию и возвращается в сон сразу после обработки. Высокое мгновенное потребление в активном режиме компенсируется тем, что активный режим занимает малую долю общего времени работы устройства. Именно реализация такой компенсации составляет основное содержание проектной части настоящей работы.

Таблица 1 — Сравнение аппаратных платформ для реализации систем KWS

Платформа	Потребление при KWS	Гибкость и экосистема	Пригодность для автономного IoT
Облачные решения	до 800 мВт в пике передачи Wi-Fi [1]	неограниченная сложность моделей	непригодны: радиоканал, латентность, приватность

Платформа	Потребление при KWS	Гибкость и экосистема	Пригодность для автономного IoT
ASIC (Syntiant NDP120 и подобные)	десятки мкДж/-инференс	нет: модель аппаратно фиксирована	идеальная, но закрыта и недоступна
ПЛИС (Xilinx Artix-7, iCE40 UP)	десятки [16] мкДж/инференс	проектирование на HDL/HLS	нет встроенных Wi-Fi/BLE, требует внешнего MCU для стека связности
DSP (Cadence HiFi, TI C55xx)	десятки мВт в активном режиме	узкая ниша, ограниченное сообщество разработчиков	ориентированы на потоковую обработку, избыточны для одноканального KWS
MCU общего назначения	70–150 мВт при активном инференсе [12]	полная гибкость открытые SDK	пригоден при каскадной архитектуре

1.4 Каскадные подходы к снижению энергопотребления

Каскадная детекция разделяет звуковой тракт на несколько последовательных этапов с возрастающей вычислительной сложностью, каждый из которых выступает фильтром-предусловием для активации следующего. В периоды тишины работает только первый, наиболее дешёвый этап; нейросетевой инференс запускается лишь при обнаружении потенциального речевого события. Подобная архитектура реализует принцип, известный в коммерческих голосовых ассистентах (Amazon Echo, Google Nest) и в академической литературе под общим названием *always-on listening* — постоянно активного прослушивания при минимальном среднем энергопотреблении. Концептуальная схема каскада, принятого в настоящей работе, представлена в разделе 2 (рис. 1).

Ключевым компонентом каскадной архитектуры является детектор голосовой (или речевой) активности (Voice Activity Detector, VAD), задача которого — с минимальными вычислительными затратами отличить периоды тишины от периодов наличия звукового сигнала. Построение VAD является самостоятельным направлением исследований в области цифровой обработки сигналов и аналоговой схемотехники. В отечественной литературе вопросы построения детекторов VAD рассмотрены, в частности, в работе Стефаниди и др. [8], предложивших комбинированный детектор, объединяющий несколько признаков речевой активности — энергию сигнала, частоту пересечения нуля и спектральные характеристики. Подобная комбинация обеспечивает устойчивость к разнотипным акустическим помехам по сравнению с одноэлементными пороговыми решениями, однако требует определённых вычислительных ресурсов.

В международной литературе последних лет особый интерес представляют работы, посвящённые специализированным аналоговым реализациям VAD с сверхнизким энергопотреблением. Chen и др. [18] предложили аналоговый VAD на базе переключаемого конденсаторного TD-CNN (time-domain CNN) как пассивного аналогового извлекателя признаков с последующим бинарным нейросетевым классификатором; разработанная схема в техпроцессе 28 нм демонстрирует долю верных обнаружений (hit rate) 90% при потреблении 108 нВт и площади кристалла 0,8 мм². Дальнейшее развитие этой линии работ Lin и др. [19] реализовали с использованием короткого окна извлечения признаков (ST-CNN, 0,5 мс), рекуррентного классификатора и энергонезависимой памяти весов на конденсаторах (CAP-ROM), получив долю верных обнаружений 94% при потреблении 47 нВт. Эти результаты демонстрируют, что первая ступень каскадной KWS-архитектуры в принципе может быть сжата до десятков нановатт при использовании специализированной аналоговой схемотехники. Вместе с тем такие решения представляют собой заказные интегральные микросхемы, разработанные и изготовленные в конкретных техпроцессах, что не позволяет использовать их в качестве готовых компонентов при проектировании IoT-устройств на массовых платформах.

Предельным упрощением VAD-схемы, доступным разработчику без обращения к заказному кремнию, является аналоговый пороговый компаратор, фиксирующий лишь превышение огибающей сигнала заданного уров-

ня. Подобный детектор не отделяет речь от прочих громких звуков, но при потреблении в сотни микроватт служит эффективным «нулевым» фильтром каскада, отсекающим длительные периоды абсолютной тишины. Именно такая упрощённая аналоговая реализация применяется в настоящей работе как первый этап каскада; перенос алгоритмически сложных VAD на маломощный сопроцессор или специализированный аудио-фронтенд составляет самостоятельное направление развития предложенной системы.

Среди академических работ по каскадным KWS-системам на цифровых платформах выделяются следующие. Hardy и Badets [6] предложили двухуровневый каскад, в котором аналоговый детектор голосовой активности с потреблением порядка десятков микроватт управляет пробуждением основного микроконтроллера для запуска RNN-классификатора, и продемонстрировали устойчивость предложенной схемы к реальным акустическим условиям. Cerutti и др. [7] реализовали систему с аналоговым извлечением бинарных признаков и бинарной нейронной сетью на микроконтроллере, достигнув общего потребления менее 1 мВт в режиме непрерывного мониторинга. Замена цифровой предобработки аналоговым входным каскадом позволила сократить энергозатраты на этап захвата и извлечения признаков в 29 раз, снизив их долю с доминирующих 85% до 16% общего энергобюджета, — результат, подтверждающий, что основным источником энергопотребления в KWS-конвейере на микроконтроллере является не инференс нейронной сети, а предшествующая ему обработка аудиосигнала. Bushur и Chen [12] провели сравнительное исследование нейросетевых архитектур для KWS на различных периферийных платформах, оценивая компромисс между точностью, латентностью и размером модели.

Сопоставление перечисленных работ выявляет характерную особенность: каждая из них рассматривает лишь отдельный аспект проблемы энергоэффективности KWS. Работы Chen и др. [18] и Lin и др. [19] достигают рекордного потребления первой ступени, но требуют заказных аналоговых интегральных микросхем. Hardy и Badets исследуют каскадную архитектуру, но не применяют квантизацию модели и не проводят декомпозицию энергобюджета по компонентам. Cerutti и др. оптимизируют предобработку с помощью аналогового входного каскада, но не используют каскадное пробуждение из режима глубокого сна с многоступенчатой фильтрацией событий. Bushur и

Chen сравнивают платформы и архитектуры, но не рассматривают каскадный подход и не приводят покомпонентной декомпозиции энергопотребления. Решение, объединяющее каскадную архитектуру с управлением режимами питания, INT8-квантизацию обучаемой нейросетевой модели и количественную декомпозицию энергобюджета по этапам обработки на единой платформе, в открытой литературе представлено недостаточно. Данное наблюдение определяет позиционирование настоящей работы и обосновывает её научную новизну.

1.5 Архитектуры нейронных сетей для распознавания ключевых слов

Развитие нейросетевых подходов к задаче KWS прошло путь от полносвязных сетей через свёрточные к специализированным компактным архитектурам, оптимизированным для встраиваемых платформ. Систематизация существующих алгоритмических решений для развёртывания нейронных сетей на устройствах с ограниченными ресурсами проведена в обзорной работе Khan и др. [11], где выделены три взаимодополняющих направления адаптации моделей: оптимизация архитектуры (использование компактных топологий с уменьшенным числом параметров), посттренировочные преобразования (квантизация, обрезка соединений, дистилляция знаний) и аппаратно-программный кодизайн (специализированные ядра, эксплуатация SIMD-расширений). Ниже рассматриваются архитектуры, наиболее значимые для задачи KWS на микроконтроллерах.

Архитектура DS-CNN (Depthwise Separable CNN), предложенная Zhang и др. [2] в работе «Hello Edge», основана на замене стандартных свёрточных слоёв двухступенчатой декомпозицией: пространственная свёртка по каждому каналу независимо (depthwise convolution) и линейная комбинация каналов с ядром 1×1 (pointwise convolution). Данный приём, заимствованный из архитектуры MobileNet [20], сокращает количество параметров и операций умножения с накоплением в 8–10 раз при сопоставимой точности. На наборе Google Speech Commands архитектура DS-CNN достигает точности 95,4%, что на 10 процентных пунктов превышает показатели полносвязной сети DNN с аналогичным количеством параметров [2]. В бенчмарке MLPerf Tiny [14] архитектура DS-CNN принята как эталонная для задачи KWS: все

участники бенчмарка обязаны обеспечить точность не ниже 90% на наборе Google Speech Commands.

Архитектура TC-ResNet (Temporal Convolution ResNet), предложенная Choi и др. [3], применяет одномерные свёртки вдоль временной оси с остаточными соединениями. В отличие от DS-CNN, использующей двумерные свёртки по матрице MFCC-признаков, TC-ResNet обрабатывает MFCC-фреймы как одномерные временные ряды, что позволяет существенно сократить количество вычислений. Вместе с тем архитектура менее исследована в контексте квантизации и развёртывания через TensorFlow Lite Micro на платформах Xtensa, что снижает её привлекательность для настоящей работы.

Гибридная архитектура CRNN (Convolutional Recurrent Neural Network) комбинирует свёрточные слои для извлечения локальных спектрально-временных признаков с рекуррентными слоями (GRU или LSTM) для моделирования долгосрочных временных зависимостей. Архитектура обеспечивает высокую точность, однако рекуррентные слои существенно увеличивают требования к оперативной памяти из-за необходимости хранения скрытого состояния между фреймами, что ограничивает её применимость на микроконтроллерах с типичным объёмом SRAM 256–520 КБ.

Для настоящей работы выбрана архитектура DS-CNN. Выбор обусловлен совокупностью факторов: наличие эталонной реализации в бенчмарке MLPerf Tiny [14]; доказанная эффективность на микроконтроллерных платформах; масштабируемость — наличие готовых вариантов размером 80–500 КБ позволяет исследовать компромисс между размером модели и точностью; эффективная утилизация SIMD-инструкций архитектуры Xtensa LX7 за счёт того, что pointwise-свёртки сводятся к матричным умножениям INT8-элементов, при которых в один 128-битный SIMD-регистр помещаются 16 весов формата INT8 [1]. Конкретные параметры выбранной конфигурации модели рассматриваются в разделе 2.

1.6 Методы квантизации нейронных сетей

Квантизация нейронных сетей — процедура снижения разрядности представления весов и активаций модели — является одним из основных методов адаптации нейросетевых моделей к ресурсным ограничениям микроконтроллеров. Типичным является переход от 32-битных чисел с плавающей

точкой (FP32) к 8-битным целым числам (INT8) [4]. Применительно к задаче KWS квантизация обеспечивает несколько взаимосвязанных преимуществ, существенных в контексте энергоэффективности.

Во-первых, сокращение размера модели: при переходе от FP32 к INT8 каждый вес занимает 1 байт вместо 4, что обеспечивает теоретический коэффициент сжатия до $4\times$. Для модели DS-CNN в конфигурации Medium, занимающей около 960 КБ в формате FP32, размер после квантизации составляет порядка 260 КБ — существенно ниже объёма флеш-памяти типичных микроконтроллеров. Во-вторых, ускорение инференса: целочисленная арифметика INT8 нативно поддерживается SIMD-расширениями современных микроконтроллеров. На платформе ESP32-S3 библиотека ESP-NN реализует оптимизированные ядра свёрточных операций с использованием 128-битных векторных регистров архитектуры Xtensa LX7 [1], обеспечивая кратное ускорение по сравнению с операциями с плавающей точкой. В-третьих, снижение энергопотребления — как прямое (целочисленные операции потребляют меньше энергии на одну операцию), так и косвенное: сокращённое время инференса позволяет раньше вернуть микроконтроллер в режим глубокого сна, уменьшая среднее потребление в каскадной архитектуре.

В литературе выделяют два основных подхода к квантизации [4; 5]. Посттренировочная квантизация (Post-Training Quantization, PTQ) выполняется после завершения обучения модели: предобученная модель FP32 преобразуется к INT8-представлению на основании статистик распределения весов и активаций, собранных на репрезентативном калибровочном наборе данных. Для каждого тензора вычисляются масштаб (scale) и нулевая точка (zero point), определяющие линейное отображение из диапазона вещественных чисел в диапазон целых. Достоинством PTQ является простота: процедура не требует повторного обучения, выполняется за минуты и применима к произвольной предобученной модели. Недостатком — потенциально большая потеря точности, поскольку модель не компенсирует ошибки округления.

Квантизация с дообучением (Quantization-Aware Training, QAT) встраивает процедуру квантизации в цикл обучения. На этапе прямого прохода в граф вычислений вставляются узлы симуляции квантования (fake quantization nodes), моделирующие эффект округления активаций до INT8. Градиенты рассчитываются по приближению Straight-Through Estimator, что

позволяет оптимизатору компенсировать ошибки округления. QAT обеспечивает меньшую деградацию точности по сравнению с PTQ, что особенно заметно при квантизации ниже 8 бит [4]. Недостатком является необходимость повторного обучения и более сложная инфраструктура тренировочного конвейера. При экстремально низких разрядностях (1–2 бита) задача обеспечения стабильности обучения квантизованной сети сама по себе становится нетривиальной: дискретизация весов и активаций приводит к плохой обусловленности задачи оптимизации и к нестабильности градиентов. В работе Трусова и др. [21] предложен метод квантования на основе оценок неопределённости, стабилизирующий обучение бинарных нейронных сетей и обеспечивающий воспроизводимость результатов; для рассматриваемой в настоящей работе квантизации INT8 такие крайние эффекты не наблюдаются, что подтверждается результатами раздела 4.

Для задачи KWS с квантизацией до INT8 оба подхода в литературе обеспечивают потерю точности менее 1 процентного пункта [2; 4]. Тем не менее их сравнительный анализ на конкретной модели DS-CNN с развёртыванием на целевой платформе ESP32-S3 и оценкой влияния на латентность и энергопотребление инференса представляет самостоятельный исследовательский интерес и составляет одну из задач настоящей работы.

1.7 Фреймворки для развёртывания моделей на микроконтроллерах

Для выполнения инференса нейронных сетей на микроконтроллере применяются специализированные среды исполнения, адаптированные к ограничениям встраиваемых платформ: фиксированный объём оперативной и флеш-памяти, отсутствие операционной системы общего назначения и динамической аллокации.

TensorFlow Lite Micro (TFLM) [22] представляет собой облегчённую версию фреймворка TensorFlow для микроконтроллеров. Размер библиотеки составляет порядка 100–200 КБ машинного кода. TFLM использует статически выделяемую область памяти (tensor arena) для размещения всех промежуточных тензоров, что исключает необходимость динамической аллокации. Модели хранятся в формате FlatBuffers (.tflite), содержащем граф вычислений, параметры квантизации и массив весов. Фреймворк поддерживает полную INT8-квантизацию входных, выходных и промежуточных тензоров. Для

платформ Espressif доступна оптимизированная сборка esp-tflite-micro, интегрирующая библиотеку ESP-NN — набор ядер свёрточных операций, реализованных с использованием SIMD-инструкций архитектуры Xtensa LX7 [1].

ESP-SR (Espressif Speech Recognition) — проприетарная библиотека Espressif для задач распознавания речи, включающая модели WakeNet различных версий. Модели обучены с применением QAT и оптимизированы под архитектуру ESP32-S3, демонстрируя время инференса порядка единиц миллисекунд на фрейм. Основным ограничением является закрытость архитектуры и весов моделей, что исключает возможность независимой модификации, воспроизводимого исследования и сравнительного анализа. CMSIS-NN [23] — библиотека оптимизированных нейросетевых ядер для платформ ARM Cortex-M, разработанная Arm Ltd. Реализует свёрточные, полносвязные и pooling-операции с использованием SIMD-инструкций DSP Extension. Является стандартным решением для семейств STM32 и nRF52, однако неприменима к архитектуре Xtensa.

Для настоящей работы выбран TensorFlow Lite Micro с оптимизациями ESP-NN: данный стек обеспечивает развёртывание самостоятельно обученных квантизованных моделей DS-CNN на ESP32-S3 при сохранении аппаратного ускорения и полной воспроизводимости эксперимента.

1.8 Выбор целевой платформы

В качестве платформы для экспериментальной апробации выбран микроконтроллер ESP32-S3 производства Espressif Systems [1]. Выбор обусловлен совокупностью технических факторов: двухъядерный процессор Xtensa LX7 с тактовой частотой до 240 МГц и SIMD-расширением PIE, обеспечивающим аппаратное ускорение целочисленных нейросетевых операций; объёмы памяти (8 МБ флеш и 8 МБ внешней PSRAM в использованном модуле), достаточные для размещения встроенного программного обеспечения, библиотеки TFLM и квантизованной модели DS-CNN; встроенный аппаратный контроллер I2S для цифровых MEMS-микрофонов; поддержка режима глубокого сна с пробуждением по сигналу на выводах RTC GPIO, по таймеру RTC или по сигналу от сопроцессора ULP, с потреблением кристалла в данном режиме порядка 7–8 мкА [1]. Программная поддержка обеспечивается открытым фреймворком ESP-IDF [24], включающим компонент esp-

tfllite-micro с библиотекой ESP-NN. Доступность отладочных плат ESP32-S3 и низкая стоимость делают платформу пригодной для независимой академической работы, в отличие от ASIC-решений.

Следует подчеркнуть, что предлагаемая в настоящей работе каскадная архитектура не привязана к конкретной платформе. Принципы каскадного пробуждения с возрастающей вычислительной сложностью применимы к любому микроконтроллеру с аппаратной поддержкой режимов глубокого сна и внешнего пробуждения — STM32U5 с механизмом LPBAM, nRF52840 с функцией wake-on-sound и подобным. Выбор ESP32-S3 обусловлен задачей экспериментальной апробации и совокупностью практических факторов; обоснование выбора на уровне отдельных характеристик платформы (структура памяти, конкретные периферийные модули, особенности режимов сна) приведено в разделе 2.

1.9 Выводы и формулировка требований к решению

Проведённый анализ предметной области позволяет сформулировать следующие выводы.

- 1) Задача распознавания ключевых слов на автономных IoT-устройствах характеризуется фундаментальным противоречием между необходимостью непрерывного мониторинга аудиопотока и ограниченным энергетическим бюджетом устройства. Облачные решения непригодны для батарейных устройств вследствие высокого потребления радиомодуля при передаче аудиоданных, сетевой латентности и зависимости от инфраструктуры;
- 2) Из аппаратных платформ, рассматриваемых в литературе, специализированные ASIC обеспечивают наивысшую энергоэффективность, но закрыты и не допускают независимого исследования. ПЛИС демонстрируют низкую латентность и энергопотребление, но требуют специализированных компетенций и не обладают IoT-периферией. Выделенные DSP архитектурно устаревают на фоне интеграции DSP-функций в современные микроконтроллеры. Микроконтроллеры общего назначения обеспечивают оптимальный баланс гибкости, доступности и энергоэффективности при условии применения каскадной архитектуры с управлением режимами питания;

- 3) Принципиальная возможность реализации первой ступени каскадной KWS-системы с энергопотреблением в десятки нановатт подтверждена работами Chen и др. [18] и Lin и др. [19], однако опирается на специализированную аналоговую схемотехнику, недоступную при разработке IoT-устройств на массовых платформах. Настоящая работа исследует ортогональный подход — достижимое снижение энергопотребления в инфраструктуре цифровой микроконтроллерной платформы общего назначения;
- 4) Каскадная архитектура с этапами возрастающей вычислительной сложности позволяет снизить среднее потребление на порядки за счёт исключения нейросетевого инференса в периоды тишины. Существующие работы рассматривают каскадную архитектуру, квантизацию модели и декомпозицию энергобюджета преимущественно изолированно; их совместное применение с отдельной количественной оценкой вклада каждого компонента в открытой литературе охарактеризовано недостаточно;
- 5) Архитектура DS-CNN является эталонной для задачи KWS на микроконтроллерах, обеспечивая точность свыше 95% при размере модели 80–500 КБ в зависимости от варианта масштабирования и эффективную утилизацию SIMD-инструкций целевой платформы. Квантизация INT8 обеспечивает сжатие модели до $4\times$ и ускорение инференса при потере точности менее 1 процентного пункта; сравнение методов PTQ и QAT на конкретной модели и платформе сохраняет самостоятельный исследовательский интерес;
- 6) Для разворачивания моделей выбран открытый стек TensorFlow Lite Micro с оптимизациями ESP-NN, обеспечивающий аппаратное ускорение и полную воспроизводимость эксперимента. В качестве целевой платформы для апробации выбран микроконтроллер ESP32-S3; принципиальная архитектура решения переносима на другие платформы с аппаратными режимами глубокого сна.

На основании проведённого анализа сформулированы количественные и качественные требования к разрабатываемой системе, которым она должна удовлетворять для подтверждения исследовательской гипотезы о возможности достижения энергоэффективности на микроконтроллере общего назна-

чения за счёт каскадной архитектуры и квантизации.

- 1) *По средней потребляемой мощности.* Среднее энергопотребление системы в режиме ожидания при низкой частоте голосовых событий должно быть кратно ниже потребления в режиме непрерывного инференса на той же платформе.
- 2) *По латентности.* Время отклика системы от начала произнесения ключевого слова до выдачи решения классификатором должно соответствовать практическим требованиям интерактивного голосового интерфейса (порядок секунд, без жёстких реальных временных ограничений, характерных для управления критическими процессами).
- 3) *По точности классификации.* Точность распознавания на тестовой выборке стандартного набора Google Speech Commands [13] должна составлять не менее 90% (требование бенчмарка MLPerf Tiny [14]); потеря точности от квантизации модели не должна превышать одного процентного пункта.
- 4) *По размеру модели и оперативной памяти.* Размер квантизированной модели должен укладываться во флеш-память целевой платформы (порядка единиц мегабайт); размер промежуточных тензоров (tensor arena) должен укладываться в доступный объём оперативной памяти микроконтроллера.
- 5) *По воспроизводимости и переносимости.* Решение должно строиться на открытых программных компонентах, исключая закрытые проприетарные нейросетевые среды исполнения; архитектурно решение не должно быть привязано к конкретной аппаратной платформе и должно допускать перенос на другие микроконтроллеры с поддержкой режимов глубокого сна и внешнего пробуждения.
- 6) *По наблюдаемости.* Архитектура измерительного стенда должна обеспечивать возможность отдельного измерения энергопотребления каждого этапа цикла обработки (пробуждение, захват аудио, извлечение признаков, инференс, индикация результата, переход в сон), что необходимо для решения задачи декомпозиции энергобюджета.

Сформулированные требования определяют рамки проектирования

каскадной архитектуры, которому посвящён следующий раздел.

2 Проектирование каскадной архитектуры системы распознавания голосовых команд

В разделе 1 на основании анализа предметной области сформулирована гипотеза о возможности достижения энергоэффективности KWS-системы на микроконтроллере общего назначения за счёт совместного применения каскадной архитектуры детекции и квантизации нейросетевой модели, а также определены требования, которым должно удовлетворять проектируемое решение. Настоящий раздел посвящён формализации методологии каскадного управления режимами энергопотребления, выводу проектных критериев выбора параметров каскада, обоснованию архитектуры и гиперпараметров нейросетевого классификатора, а также описанию применяемых методов квантизации.

2.1 Принцип каскадной детекции и формализация энергобюджета

Идея каскадной детекции состоит в разделении задачи непрерывного мониторинга звукового потока на последовательность ступеней с возрастающей вычислительной сложностью и, как следствие, возрастающим мгновенным энергопотреблением. Каждая ступень выполняет функцию фильтра-предусловия: она активирует следующую ступень только при срабатывании по своему, более простому критерию. В периоды абсолютной тишины работает только первая, наиболее дешёвая ступень; вычислительно затратные операции — извлечение признаков и нейросетевой инференс — выполняются лишь по событию, потенциально содержащему голосовую команду.

Преимущество каскадной архитектуры перед режимом непрерывного инференса обусловлено асимметрией активности ступеней во времени. Пусть система состоит из K последовательных ступеней с мгновенным потреблением мощности $P_1, P_2, \dots, P_K$ соответственно ($P_1 \ll P_2 < \dots < P_K$). Обозначим α_k долю времени, в течение которой ступень k находится в активном состоянии. Первая ступень активна всегда: $\alpha_1 = 1$. Каждая последующая ступень активируется только при срабатывании предыдущей, поэтому $\alpha_{k+1} = \alpha_k \cdot p_k^{\text{trig}}$, где p_k^{trig} — вероятность активации ступени $k + 1$ по результату работы ступени k за единицу времени. Средняя потребляемая мощность системы выражается в виде

$$\bar{P} = \sum_{k=1}^K \alpha_k \cdot P_k, \quad (1)$$

где $\bar{P}$ — средняя мощность системы за длительный промежуток наблюдения; α_k — доля времени активности k -й ступени; P_k — мгновенная мощность k -й ступени в активном состоянии.

В вырожденном случае непрерывного инференса каскад состоит из одной ступени, постоянно активной, и средняя мощность равна P_K . В каскадной архитектуре при правильно выбранных порогах срабатывания α_k убывают с ростом k на порядки: для типичного бытового сценария ($K = 3$, редкие голосовые события) характерны значения $\alpha_2 \sim 10^{-2}$ и $\alpha_3 \sim 10^{-3}$. Подставляя эти значения в (1), нетрудно убедиться, что доминирующий вклад в среднюю мощность даёт первая ступень — даже если её мгновенное потребление мало по сравнению со всеми остальными.

Из формулы (1) непосредственно вытекают *два проектных критерия* каскадной архитектуры, которыми руководствуется настоящая работа.

Критерий разделения по сложности. Чтобы каскадная архитектура обеспечивала выигрыш относительно непрерывного инференса, мгновенное потребление каждой ступени должно быть кратно ниже потребления последующей: $P_k \ll P_{k+1}$. В противном случае энергия, расходуемая на постоянно активную дешёвую ступень, превзойдёт экономию от редкой активации дорогой. Применительно к настоящей работе: первая ступень (аналоговый детектор) должна потреблять не более единиц миллиампер при напряжении питания целевой платформы, тогда как ступень нейросетевого инференса допускает потребление в десятки миллиампер при условии её эпизодической активации.

Критерий частотного разделения. Чтобы каскад имел смысл, частота активации каждой последующей ступени должна быть существенно ниже частоты активации предыдущей: $\alpha_{k+1} \ll \alpha_k$. Это обеспечивается тем, что критерий срабатывания ступени k намеренно выбирается грубее критерия ступени $k + 1$: ступень k может допускать ложные срабатывания, но не должна давать ложных отказов. Все ложные срабатывания ступени k будут отфильтрованы на ступени $k + 1$, тогда как ложный отказ означает потерю команды и неприятен.

В настоящей работе принята архитектура с $K = 3$ ступенями. Выбор именно трёх ступеней обусловлен сочетанием технических ограничений платформы ESP32-S3 и характера задачи: одноступенчатая архитектура сводится к непрерывному инференсу; двухступенчатая (аналоговый VAD — нейросетевой инференс) объединяет в одной ступени захват аудио, предобработку и инференс, что не позволяет разделить их вклад в энергобюджет; трёхступенчатая архитектура разделяет потенциально наиболее затратный этап (захват и MFCC-предобработка) и собственно нейросетевой инференс, что соответствует наблюдению Cerutti и др. [7] о доминирующем вкладе предобработки в общий энергобюджет KWS-конвейера.

2.2 Архитектура каскада из трёх ступеней

Концептуальная схема предложенной архитектуры приведена на рис. 1. Каскад состоит из трёх последовательных ступеней с возрастающей вычислительной сложностью и согласованным управлением режимами питания микроконтроллера.

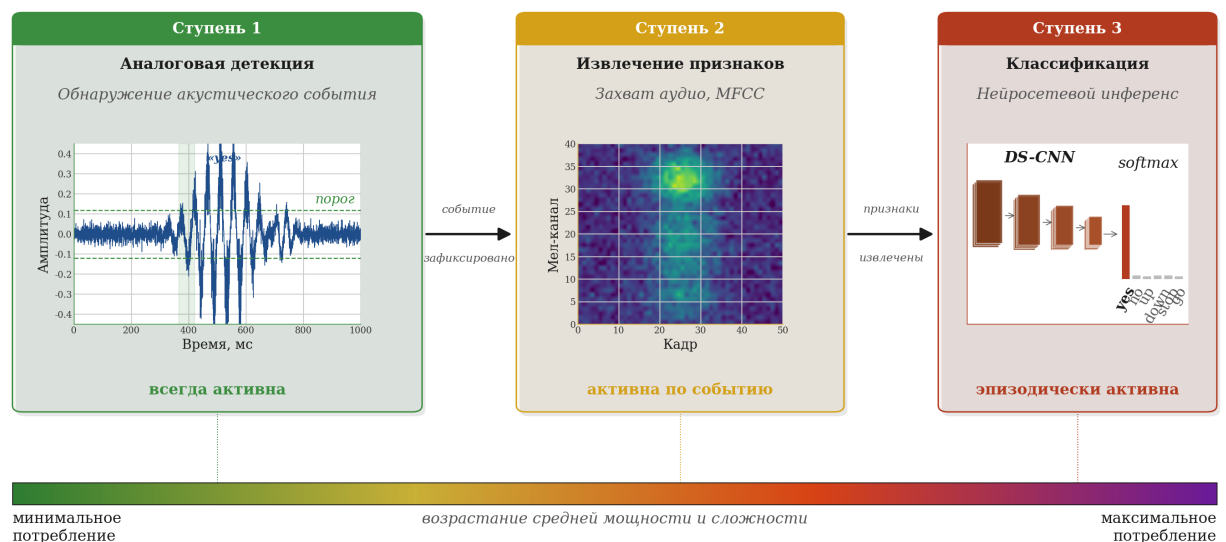


Рисунок 1 — Концептуальная схема каскадного пробуждения: три ступени возрастающей вычислительной сложности с переключением режимов питания микроконтроллера

Ступень 1 — аналоговая детекция акустического события — выполняет непрерывную пороговую обработку амплитуды сигнала с аналогового микрофона и принимает решение о наличии звукового события произвольной природы (речь, шум, удар). Реализована на маломощном компараторе,

потребление измеряется единицами сотен микроампер, активна постоянно. Положительное решение ступени формирует аппаратное прерывание, пробуждающее микроконтроллер из режима глубокого сна.

Ступень 2 — захват аудио и извлечение признаков — выполняет запись окна аудио заданной длительности с цифрового микрофона и расчёт MFCC-признаков на ЦПУ микроконтроллера в активном режиме. Потребление возрастает до десятков миллиампер, ступень активна по событию ступени 1 (доли процента общего времени работы).

Ступень 3 — нейросетевая классификация — выполняет инференс квантизованной DS-CNN с аппаратным SIMD-ускорением и возвращает индекс распознанного класса. Потребление сопоставимо со ступенью 2 (десятки миллиампер), ступень активна эпизодически по результату ступени 2.

Цветовая шкала на рисунке отражает рост средней мощности и вычислительной сложности от первой ступени к третьей. Указанное кратное разделение ступеней по мгновенному потреблению и по доле активности — два необходимых условия выигрыша каскадной архитектуры, формально получаемые из аналитической модели средней мощности (см. подраздел ??).

2.3 Критерий выбора порога ступени-предусловия

Каждая ступень-предусловие характеризуется порогом срабатывания, определяющим компромисс между двумя видами ошибок: ложными срабатываниями (срабатывание в отсутствие команды, приводящее к избыточному пробуждению последующих ступеней) и ложными отказами (несрабатывание при наличии команды, приводящее к её потере).

Для ступени 1 порог компаратора $V_{th}^{(1)}$ задаётся аппаратно с помощью подстроечного резистора. Выбор порога производится экспериментально из условия гарантированного срабатывания при произнесении команды на типичном для целевого сценария расстоянии от микрофона и одновременно минимизации срабатываний на фоновый акустический шум помещения. Поскольку аналоговый компаратор не различает тип звукового события, ложные срабатывания на бытовые шумы (хлопки дверей, шаги, посторонняя речь) принципиально неустранимы на данной ступени; их фильтрация возложена на ступень 3.

Для ступени 3 порог уверенности $\tau_{conf}^{(3)}$ устанавливается на этапе раз-

вёртывания модели в виде минимального значения логита (или, эквивалентно, вероятности после softmax-нормализации) победившего класса. Поднятие порога снижает долю ложных срабатываний (FAR, false acceptance rate), но увеличивает долю ложных отказов (FRR, false rejection rate); опускание порога даёт обратный эффект. В контексте каскадной архитектуры с батарейным питанием приоритет отдан минимизации ложных срабатываний: каждое ложное срабатывание ступени 3 приводит к ошибочной активации управляющего действия (например, включение исполнительного устройства), тогда как ложный отказ требует от пользователя лишь повторить команду. Тем самым проектное правило формулируется следующим образом: порог $\tau_{\text{conf}}^{(3)}$ выбирается из условия $\text{FAR} \rightarrow \min$ при ограничении FRR сверху заданным значением, не превышающим ожидаемой комфортной частоты повторов пользователем.

2.4 Архитектура нейросетевой модели и обоснование гиперпараметров

В разделе 1 в качестве архитектуры классификатора обоснована DS-CNN [2] — свёрточная сеть с разделимыми по глубине свёртками. В настоящем подразделе обосновывается конкретный выбор гиперпараметров модели в рамках предложенного авторами Hello Edge [2] семейства конфигураций.

В исходной работе [2] предложены три конфигурации, привязанные к ресурсным ограничениям целевых платформ:

- DS-CNN-S (Small): порядка 80 КБ памяти и 6 миллионов операций умножения с накоплением на инференс;
- DS-CNN-M (Medium): порядка 200 КБ памяти и 20 миллионов операций умножения с накоплением на инференс;
- DS-CNN-L (Large): порядка 500 КБ памяти и 80 миллионов операций умножения с накоплением на инференс.

Для настоящей работы выбрана конфигурация DS-CNN в варианте Medium. Конфигурация Small демонстрирует точность ниже 94% на наборе Google Speech Commands [2], что недостаточно для практического применения с приоритетом минимизации ложных срабатываний; конфигурация Large обеспечивает прирост точности менее одного процентного пункта относительно Medium при четырёхкратном увеличении вычислительной слож-

ности, что в каскадной архитектуре прямо транслируется в увеличение длительности и энергопотребления ступени 3. Конфигурация Medium обеспечивает баланс между точностью и вычислительной стоимостью.

Структурная схема выбранной модели приведена на рис. 2. Входным тензором является матрица MFCC-признаков размером $49 \times 10 \times 1$. Сеть включает первичный свёрточный слой (Conv2D, ядро 10×4 , 172 фильтра, шаг $(2, 2)$), выполняющий начальное извлечение крупномасштабных спектрально-временных паттернов; шесть блоков разделимой свёртки (DS-Conv), каждый из которых состоит из depthwise-свёртки с ядром 3×3 , нормализации (batch normalization), активации ReLU, pointwise-свёртки с ядром 1×1 , повторной нормализации и ReLU; глобального усреднения по пространственным размерностям (Global Average Pooling), обеспечивающего инвариантность к малым временным сдвигам произнесения команды; полносвязного классификационного слоя 172×12 . Общее число параметров модели составляет порядка 300 тысяч, что в формате INT8 после квантизации соответствует размеру TFLite-файла порядка 260 КБ.

Выбор числа фильтров (172) и количества DS-Conv-блоков (шесть) непосредственно следует из эталонной конфигурации DS-CNN-M [2]. Сохранение указанных параметров обеспечивает воспроизводимость результатов и прямую сопоставимость с MLPerf Tiny [14], в котором DS-CNN-M выступает эталонной моделью для задачи KWS. Реализация архитектуры модели на фреймворке TensorFlow и полный код обучения приведены в приложении E (модули ds_cnn.py и train.py).

2.5 Методы квантизации применяемой модели

Сформулированные в разделе 1 требования к решению включают укладывание размера модели во флеш-память платформы и сокращение времени инференса для уменьшения вклада ступени 3 в общий энергобюджет. Оба требования удовлетворяются применением квантизации модели из формата FP32 в INT8. Согласно (1), сокращение длительности ступени 3 при неизменной мгновенной мощности эквивалентно снижению α_3 и прямо влияет на $\bar{P}$.

В настоящей работе исследуются оба общепринятых метода квантизации — посттренировочная (PTQ) и квантизация с дообучением (QAT). Целевая разрядность выбрана как INT8: данный уровень обеспечивает четырёх-

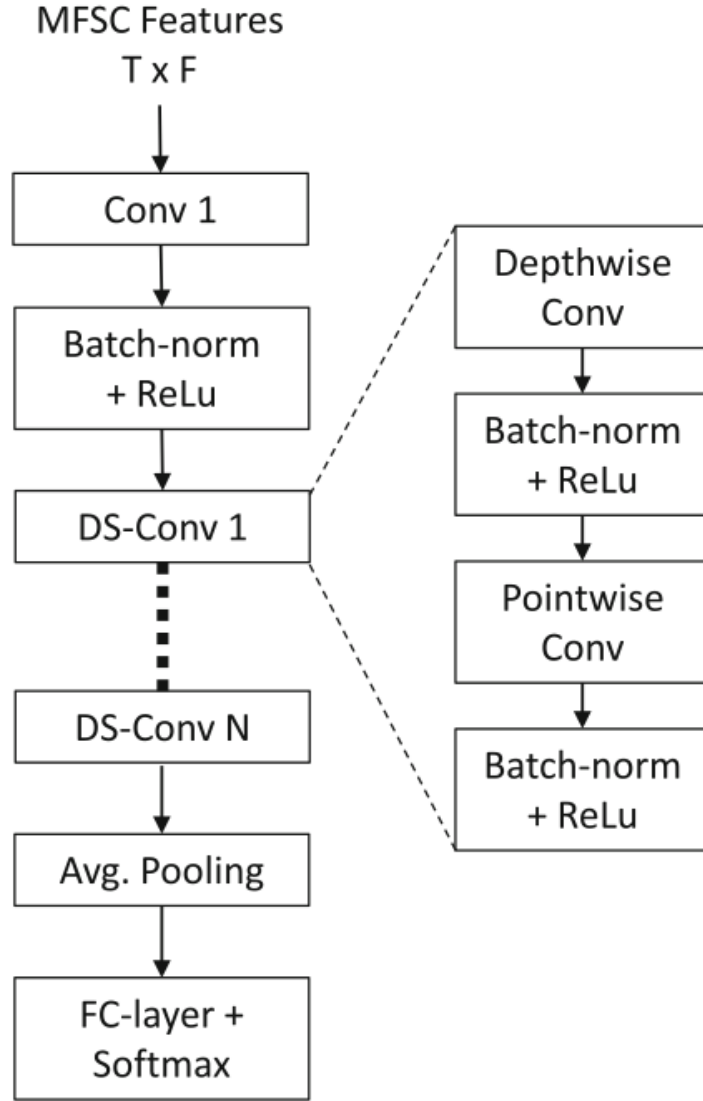


Рисунок 2 — Структурная схема модели DS-CNN-M с указанием размерностей входа, выхода и параметров слоёв

кратное сокращение размера модели при сохранении точности классификации и не требует специальных приёмов стабилизации обучения, характерных для более агрессивной (бинарной или тернарной) квантизации и рассмотренных, в частности, в работе Трусова и др. [21]. Цель сравнения, как сформулировано в задаче 3 введения — эмпирическая характеристика различий двух методов на конкретной модели и платформе по совокупности метрик: точность классификации, размер итогового файла модели, время инференса при аппаратном ускорении ESP-NN.

Линейная аффинная квантизация, применяемая в обоих методах, отображает вещественное значение r в целочисленное q по правилу

$$q = \text{clamp}\left(\left\lfloor \frac{r}{s} \right\rfloor + z, q_{\min}, q_{\max}\right), \quad (2)$$

где q — результирующее квантованное значение; r — исходное вещественное число; s — масштабный коэффициент (scale); z — нулевая точка (zero point), целочисленный сдвиг, обеспечивающий представление вещественного нуля; $[q_{\min}, q_{\max}] = [-128, 127]$ — диапазон знакового 8-битного целого. Обратное преобразование (деквантизация) определяется как $\hat{r} = s \cdot (q - z)$ и используется при обратной интерпретации результата на этапе сравнения с выходом FP32-модели.

Посттренировочная квантизация (PTQ). Применяется к предобученной FP32-модели без модификации её весов в процессе обучения. Параметры квантизации s и z для каждого тензора (весов слоёв, активаций промежуточных тензоров) определяются на основании статистик распределения значений, собранных на репрезентативном калибровочном наборе. Для весов статистики извлекаются непосредственно из обученной модели; для активаций — из выходов соответствующих слоёв при пропуске через сеть калибровочных образцов. Достоинством метода является отсутствие необходимости в повторном обучении: процедура выполняется за минуты средствами TensorFlow Lite Converter [22]. Реализация процедуры приведена в приложении E (модуль `quantize_ptq.py`).

Квантизация с дообучением (QAT). Параметры квантизации определяются совместно с весами в процессе обучения. На этапе прямого прохода в граф вычислений вставляются узлы симуляции квантования (fake quantization nodes), реализующие операцию

$$\hat{r} = s \cdot (\text{clamp}(\lfloor r/s \rfloor + z, q_{\min}, q_{\max}) - z), \quad (3)$$

моделирующую ошибку округления, которую внесёт целевое INT8-представление. Поскольку операция округления не дифференцируема, при обратном распространении ошибки используется приближение Straight-Through Estimator: градиент функции потерь $\mathcal{L}$ по непрерывной переменной r оценивается как

$$\frac{\partial \mathcal{L}}{\partial r} \approx \frac{\partial \mathcal{L}}{\partial \hat{r}} \cdot \mathbf{1}_{[r_{\min}, r_{\max}]}(r), \quad (4)$$

где $\mathbf{1}_{[r_{\min}, r_{\max}]}(r)$ — индикаторная функция, обращающаяся в единицу внутри диапазона квантования и в ноль вне его. Тем самым опти-

мизатор получает возможность сдвигать веса так, чтобы компенсировать ошибку округления, накапливающуюся при последующем переводе модели в целочисленный формат. Реализация выполнена средствами библиотеки `tensorflow_model_optimization` [5], автоматически вставляющей узлы симуляции в слои `Conv2D`, `DepthwiseConv2D` и `Dense`. Дообучение проводилось на протяжении 25 эпох с пониженной скоростью обучения относительно исходного цикла обучения FP32-модели. Реализация приведена в приложении Е (модуль `quantize_qat.py`); особенностью реализации является предварительное слияние слоёв нормализации (`BatchNorm`) в предшествующие свёрточные слои перед квантизацией, что устраняет числовую нестабильность при симуляции квантования.

Сравнительные результаты применения обоих методов к модели DS-CNN-M, включая измерения точности на тестовой выборке, размер итоговых файлов и время инференса на платформе ESP32-S3, приведены в разделе 4. В настоящем разделе фиксируется лишь процедура их получения.

2.6 Выводы по разделу

В настоящем разделе сформулирована методология каскадного управления режимами энергопотребления KWS-системы на микроконтроллере общего назначения. Получены следующие основные результаты.

- 1) Выведена аналитическая модель средней потребляемой мощности каскадной системы (1), связывающая энергобюджет с мгновенными потреблением ступеней и долями их активности. Из модели вытекают два проектных критерия — кратное разделение ступеней по мгновенному потреблению и по частоте активации, — следование которым является необходимым условием выигрыша каскадной архитектуры перед режимом непрерывного инференса;
- 2) Предложена трёхступенчатая архитектура каскада: аналоговая детекция звукового события на компараторе LM393, захват аудио и извлечение MFCC-признаков на ЦПУ микроконтроллера в активном режиме, нейросетевой инференс квантизированной модели DS-CNN на ЦПУ с аппаратным ускорением ESP-NN. Архитектура удовлетворяет обоим выведенным проектным критериям;
- 3) Сформулировано проектное правило выбора порогов срабатывания

ступеней-предусловий: для каждой ступени-предусловия порог выбирается из условия минимизации ложных срабатываний при ограничении ложных отказов сверху заданным значением; при этом приоритет в каскадной батарейной системе отдан минимизации ложных срабатываний;

- 4) Обоснован выбор архитектуры нейросетевого классификатора DS-CNN в конфигурации Medium как обеспечивающей оптимальный баланс точности и вычислительной сложности для ресурсов целевой платформы ESP32-S3 и для использования в составе ступени 3 каскада. Зафиксированы конкретные гиперпараметры: 172 фильтра, шесть DS-Conv-блоков, входной тензор $49 \times 10 \times 1$;
- 5) Описаны и формализованы оба исследуемых метода квантизации модели — посттренировочная (PTQ) и с дообучением (QAT). Приведены математические формулировки прямого и обратного квантизационного преобразования, а также приёма Straight-Through Estimator, применяемого при обратном распространении ошибки в QAT.

Сформулированная методология и принятые проектные решения определяют содержание следующего раздела, в котором рассматриваются практическая реализация системы на платформе ESP32-S3, разработка лабораторного измерительного стенда и методика проведения экспериментальной апробации.

3 Реализация системы и методика экспериментальной апробации

В разделе 2 предложена каскадная архитектура из трёх ступеней, обоснован выбор класса нейросетевого классификатора DS-CNN и формализованы методы квантизации модели. Настоящий раздел посвящён реализации описанной системы на платформе ESP32-S3, методике систематического исследования архитектур нейросетевого классификатора, разработке лабораторного измерительного стенда, обеспечивающего регистрацию энергопотребления с разрешением единиц микроампер, и методике экспериментальной апробации. Изложение построено в той же последовательности, что и работа системы: сначала рассматривается реализация встроенного программного обеспечения и тракта обработки звука, затем — методика обучения и квантизации множества нейросетевых моделей, далее — измерительная инфраструктура, и в завершение — методика эксперимента и формулы обработки полученных данных.

3.1 Программная архитектура и реализация каскада

3.1.1 Конечный автомат каскада

Логика работы каскадной системы реализована на платформе ESP32-S3 в виде конечного автомата (Finite State Machine, FSM), состояния которого соответствуют этапам цикла обработки, описанным в разделе 2. Переходы между состояниями инициируются либо аппаратными событиями (пробуждение по сигналу EXT1, завершение DMA-передачи аудиоданных), либо программными решениями (положительная классификация нейросетевой моделью, истечение временного окна захвата). Состояния автомата перечислены в таблице 2, диаграмма переходов автомата приведена в приложении Б (рис. 21).

Таблица 2 — Состояния конечного автомата каскада

Состояние	Назначение	Условие выхода
DEEP_SLEEP	глубокий сон, ожидание звукового события	пробуждение по EXT1 от компаратора LM393

Продолжение таблицы 2

Состояние	Назначение	Условие выхода
HW_BOOT	холодный старт ядра, инициализация подсистем питания и тактирования	передача управления приложению
RECORD	захват аудиоданных через I2S с микрофона INMP441	заполнение буфера, эквивалентного окну захвата
MFCC	вычисление матрицы MFCC-признаков из захваченного аудио	завершение конвейера предобработки
INFERENCE	выполнение инференса DS-CNN через TFLite Micro	возврат результата интерпретатором
RESULT	интерпретация результата, индикация классифицированной команды	завершение времени индикации
SHUTDOWN	деинициализация периферии, изоляция GPIO, подготовка к глубокому сну	вызов <code>esp_deep_sleep_start()</code>

Для синхронизации профиля энергопотребления с состояниями автомата два вывода GPIO микроконтроллера кодируют текущее состояние FSM в виде двухбитного значения и опрашиваются логгером измерительного стенда параллельно с регистрами датчика тока. Данный механизм позволяет в ходе постобработки CSV-файла измерений однозначно сопоставить участки профиля мгновенного тока с конкретными этапами обработки и тем самым реализовать декомпозицию энергобюджета, составляющую одну из задач работы.

Реализация конечного автомата выполнена в виде последовательной программы на языке C с использованием фреймворка ESP-IDF версии 5.4 [24]. Переход в режим глубокого сна включает четыре последовательные операции, порядок которых существенен. Сначала выполняется деинициализация контроллера I2S и освобождение DMA-буферов для исключения паразитного потребления неактивной периферией. Затем выводы RTC GPIO, управляющие индикацией, устанавливаются

ся в состояние LOW и фиксируются вызовом `rtc_gpio_hold_en()` для сохранения уровня в режиме сна. Далее конфигурируется источник пробуждения EXT1 функцией `esp_sleep_enable_ext1_wakeup()` с указанием вывода, подключённого к выходу компаратора LM393, и активного уровня `ESP_EXT1_WAKEUP_ANY_LOW`. Завершающий вызов `esp_deep_sleep_start()` переводит микроконтроллер в глубокий сон. Порядок операций критичен: деинициализация периферии и изоляция GPIO предшествуют настройке источника пробуждения, в противном случае возможно ложное срабатывание EXT1 из-за переходных процессов на выводах RTC GPIO в момент перехода в режим сна. Полный исходный код модуля управления режимом глубокого сна доступен в репозитории программного обеспечения (см. приложение Г).

3.1.2 Декомпозиция программных модулей

Программное обеспечение структурировано по принципу разделения ответственности: управление режимом сна, захват аудио, извлечение признаков, инференс нейросети и индикация результата выделены в независимые программные модули. Компонентная диаграмма программной системы представлена на рис. 3 в нотации C4 (уровень компонентов).

Главный модуль `main.c` реализует оркестрацию состояний конечного автомата; модуль `vad_sleep` управляет переходами в режим глубокого сна и обработкой пробуждения по EXT1; модули `i2s_mic`, `mfcc` и `kws` реализуют соответственно захват аудиосигнала, извлечение MFCC-признаков и инференс нейросетевой модели; модуль `voice_engine_kws` объединяет компоненты конвейера распознавания ключевых слов голосовых команд в единый цикл обработки. Вспомогательные модули `sync_gpio`, `profile_storage`, `display`, `st7789` и `ws2812_led` обеспечивают синхронизацию с измерительным стендом, локальное хранение профилей и визуальную индикацию состояния системы. Все модули опираются на стандартные подсистемы фреймворка ESP-IDF и внешние библиотеки ESP-DSP (для аппаратно-ускоренных операций БПФ), TensorFlow Lite Micro [22] и ESP-NN [1] (для нейросетевого инференса).

Полный исходный код программного обеспечения доступен в репозитории автора по ссылке, приведённой в приложении Г. Ключевые фрагменты, иллюстрирующие реализацию пробуждения по EXT1 и конфигурации интерпретатора TensorFlow Lite Micro, приведены в приложении Д.

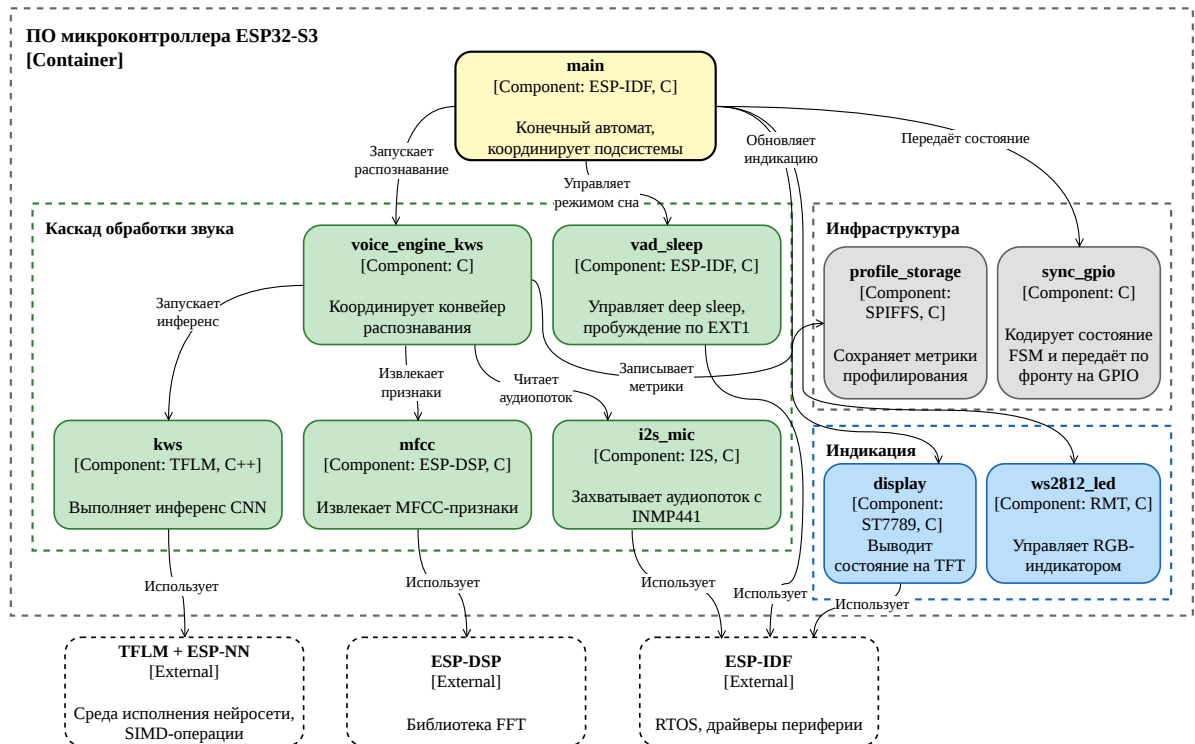


Рисунок 3 — Компонентная диаграмма программной системы микроконтроллера в нотации C4

3.1.3 Аппаратные регистры и режимы питания ESP32-S3

Описание архитектуры доменов питания, регистров управления энергосберегающими режимами и логики работы сопроцессора ULP, на которые опирается реализация переходов между состояниями, приведено в техническом справочнике на платформу [25]. В режиме глубокого сна основные ядра Xtensa LX7, цифровая периферия, модули Wi-Fi и Bluetooth обесточены; активными остаются домен RTC и сопроцессор ULP. Согласно паспортным характеристикам кристалла, потребление в данном режиме составляет порядка 7–8 мкА [1], тогда как фактическое потребление модуля ESP32-S3-Zero, использованного в работе, существенно выше из-за вклада компонентов обвязки; обсуждение этого расхождения проводится в разделе 4.

3.2 Реализация тракта обработки звука

Тракт обработки звука ступени 2 каскада реализован программно на основном ядре микроконтроллера и выполняется в состоянии MFCC конечного автомата. Конкретные параметры конвейера предобработки (частота дискретизации $f_s = 16$ кГц, длительность фрейма $T_w = 40$ мс, шаг сегментации

$T_s = 20$ мс, число мел-фильтров $M = 40$ в диапазоне 20–4000 Гц, число сохраняемых кепстральных коэффициентов $C = 10$) выбраны в соответствии с конфигурацией, основанной на эталонной конфигурации Hello Edge [2], и обоснованы в разделе 2.

Полный конвейер предобработки от исходного звукового сигнала до матрицы MFCC-признаков, подаваемой на вход нейросетевой модели, представлен на рис. 4 на примере одного произнесения слова «yes».

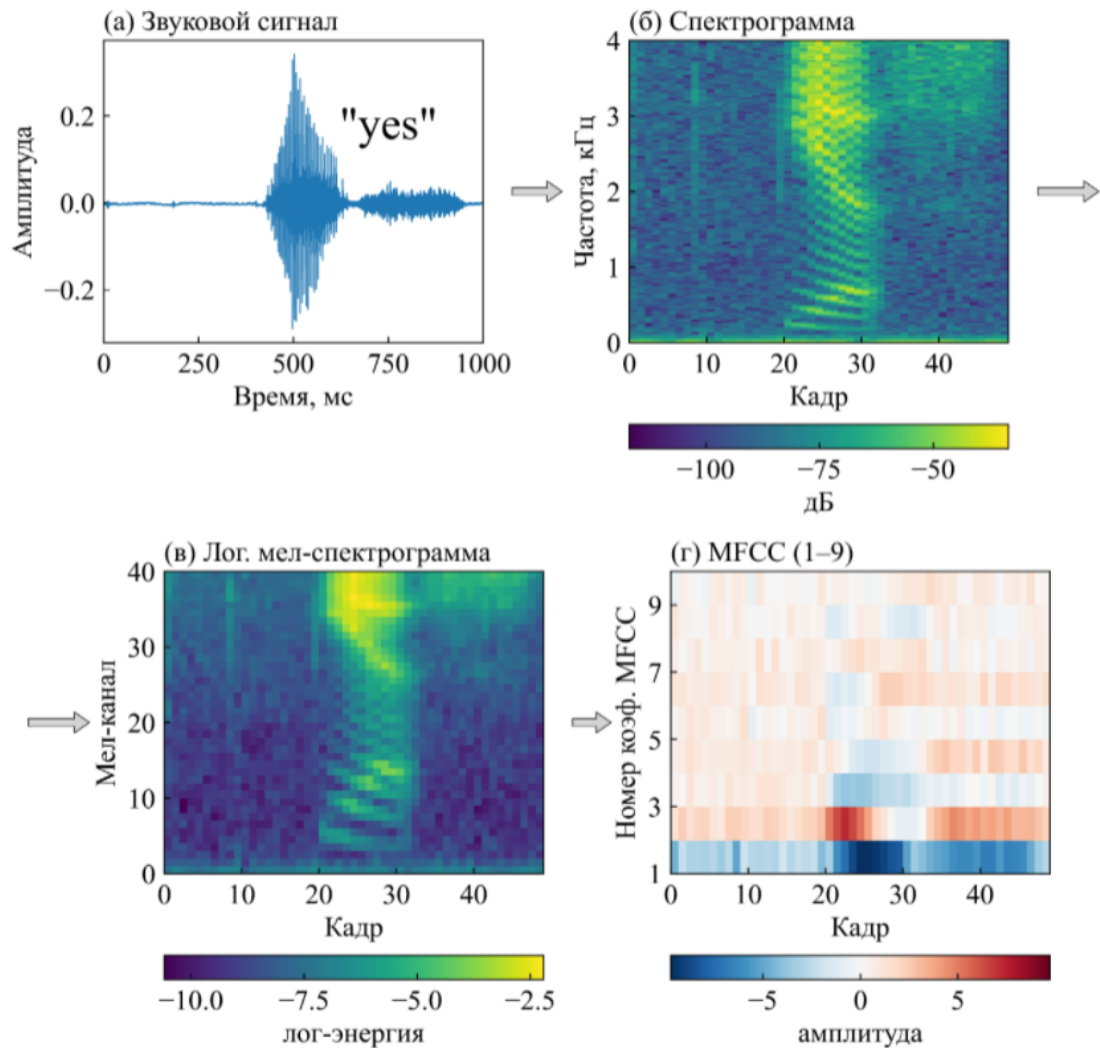


Рисунок 4 — Конвейер извлечения MFCC-признаков: исходный звуковой сигнал, его спектрограмма, логарифмическая мел-спектрограмма и итоговая матрица кепстральных коэффициентов

3.2.1 Захват аудио

Аудиосигнал поступает с цифрового MEMS-микрофона INMP441 [26] через аппаратный контроллер I2S микроконтроллера в режиме Philips: разрядность слота 32 бита, монорежим, частота дискретизации $f_s = 16$ кГц. Из

32-битного представления полезные 18 бит АЦП микрофона расположены в старших разрядах; приведение к 16-битному представлению выполняется арифметическим сдвигом вправо на 14 разрядов, что эквивалентно усечению младших разрядов шума. Захват выполняется в кольцевой буфер DMA, что обеспечивает работу аудиотракта параллельно с вычислительными задачами на основном ядре.

3.2.2 Сегментация и оконная функция

Одна секунда записи содержит $N = f_s \cdot 1 \text{ с} = 16000$ отсчётов. При длине фрейма в отсчётах $L = T_w \cdot f_s = 640$ и шаге $S = T_s \cdot f_s = 320$ количество фреймов в одной секунде аудио составляет

$$F = \left\lfloor \frac{N - L}{S} \right\rfloor + 1 = 49, \quad (5)$$

что в точности соответствует размерности входного тензора DS-CNN (49×10), зафиксированной в разделе 2. К каждому фрейму применяется оконная функция Хэмминга

$$w[n] = 0,54 - 0,46 \cdot \cos\left(\frac{2\pi n}{L - 1}\right), \quad n = 0, 1, \dots, L - 1, \quad (6)$$

назначение которой — подавление спектральной утечки на границах фрейма, неизбежной при прямоугольной сегментации непрерывного сигнала. Выбор оконной функции Хэмминга (а не Ханна или Блэкмана) обусловлен балансом между уровнем боковых лепестков спектральной характеристики окна и шириной главного лепестка, оптимальным для частотного разрешения речевых формант.

3.2.3 Дискретное преобразование Фурье

Для приведения к ближайшей сверху степени двойки размер БПФ выбран $N_{\text{FFT}} = 1024$, отсчёты фрейма дополняются нулями до этой длины. Разрешение БПФ по частоте составляет

$$\Delta f = \frac{f_s}{N_{\text{FFT}}} \approx 15,6 \text{ Гц}, \quad (7)$$

что обеспечивает достаточное частотное разрешение для разделения формант речи в диапазоне до 4 кГц. БПФ реализуется через функ-

цию `dsps_fft2r_fc32` библиотеки ESP-DSP, оптимизированную под векторные SIMD-инструкции архитектуры Xtensa LX7. Аппаратное ускорение БПФ сокращает время выполнения данного этапа кратно по сравнению со скалярной реализацией, что напрямую влияет на длительность ступени 2 и, как следствие, на её вклад в энергобюджет цикла.

3.2.4 Мел-фильтрация, логарифмирование, ДКП

Спектр мощности фрейма проецируется на $M = 40$ треугольных мел-фильтров, равномерно распределённых по психоакустически мотивированной мел-шкале

$$\text{mel}(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right), \quad (8)$$

где f — циклическая частота в герцах. Использование мел-шкалы вместо линейной отражает нелинейность восприятия частоты человеческим слухом: разрешающая способность слуха выше в низкочастотной области и снижается с ростом частоты, что соответствует характеру речевого сигнала, в котором фонемоносная информация сосредоточена в нижней части спектра.

К энергиям мел-фильтров применяется натуральный логарифм с защитной аддитивной добавкой $\varepsilon = 10^{-10}$ для исключения числовой неопределённости при нулевой энергии:

$$E_m^{\log} = \ln(E_m + \varepsilon), \quad m = 1, 2, \dots, M. \quad (9)$$

Логарифмирование выполняет двойную функцию: во-первых, обеспечивает динамическое сжатие спектра, переводя громкость в перцептивно-линейную шкалу (закон Вебера–Фехнера); во-вторых, разделяет в кепстральной области свёрточные компоненты сигнала — источник возбуждения и амплитудно-частотную характеристику речевого тракта.

К вектору логарифмических энергий мел-фильтров применяется дискретное косинусное преобразование II типа, из результата которого сохраняются первые $C = 10$ коэффициентов:

$$c_k = \sqrt{\frac{2}{M}} \sum_{m=1}^M E_m^{\log} \cdot \cos \left[\frac{\pi k}{M} \left(m - \frac{1}{2} \right) \right], \quad k = 0, 1, \dots, C - 1. \quad (10)$$

Сохранение лишь первых десяти коэффициентов мотивировано тем, что младшие кепстральные коэффициенты описывают грубую огибающую спектра речевого тракта (формантную структуру), наиболее информативную для задачи фонемной классификации, тогда как старшие коэффициенты описывают мелкомасштабные спектральные детали, связанные преимущественно с источником возбуждения и индивидуальными особенностями диктора. Аналогичный принцип отсечения коэффициентов экспериментально обоснован в смежной задаче снижения размерности признакового пространства для распознавания речевых эмоций в работе Исхаковой и др. [10].

Результирующая матрица 49×10 образует входной тензор DS-CNN. На рис. 5 приведены примеры MFCC-матриц для четырёх ключевых слов из набора Google Speech Commands. Визуально различимая кепстральная структура для разных классов команд подтверждает, что выбранное признаковое пространство сохраняет информацию, необходимую для последующей классификации; одновременно близость кепстральных паттернов внутри пар акустически близких слов («no»/«go») объясняет характер ошибок классификатора, рассматриваемых в разделе 4.

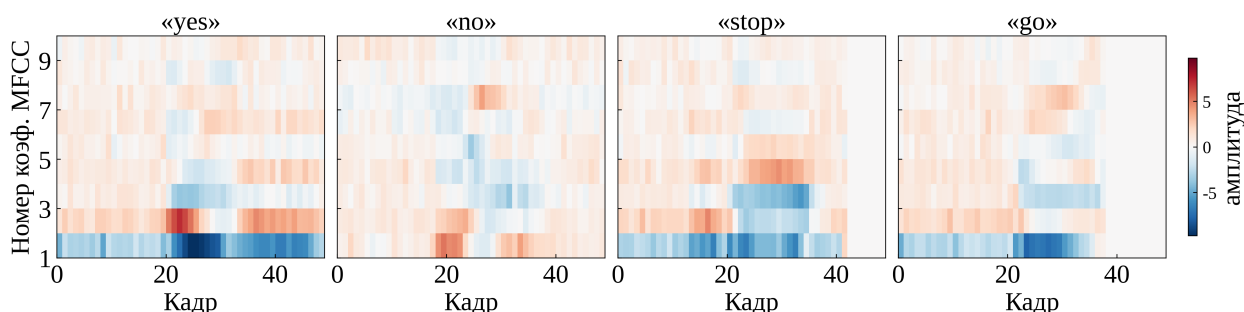


Рисунок 5 — Матрицы MFCC-признаков для четырёх ключевых слов из набора Google Speech Commands: «yes», «no», «stop», «go»

3.3 Реализация инференса нейросетевой модели

Инференс квантизированной модели DS-CNN выполняется на основном ядре ESP32-S3 фреймворком TensorFlow Lite Micro [22] с аппаратным ускорением через библиотеку ESP-NN, реализующую ядра свёрточных операций на SIMD-инструкциях Xtensa LX7.

Инициализация интерпретатора TFLM включает три этапа. Первый — загрузка модели из массива `g_model_data`, размещённого во флеш-памяти

и получаемого экспортом обученной модели в формат .tflite с последующим преобразованием в массив `const uint8_t`. Второй — регистрация используемых операторов в регистраторе операторов (op-resolver): Conv2D, DepthwiseConv2D, FullyConnected, Mean (для глобального усреднения), Reshape, Quantize, Dequantize и Softmax. Третий — аллокация арены тензоров, статически выделяемой области оперативной памяти, используемой TFLM для размещения промежуточных тензоров без динамической аллокации.

В реализации настоящей работы арена тензоров размером 140 КБ располагается во внутреннем SRAM микроконтроллера (а не во внешней PSRAM), поскольку аппаратное SIMD-ускорение библиотеки ESP-NN корректно работает только с памятью, физически доступной по шине внутреннего SRAM.

Подача данных на вход интерпретатора требует квантизации матрицы MFCC-признаков из формата FP32 в формат INT8 с использованием масштабного коэффициента и нулевой точки, считанных из метаданных входного тензора модели, согласно формуле (2.2) раздела 2. После выполнения инференса (вызов `Interpreter::Invoke()`) выходной вектор логитов в формате INT8 деквантизуется обратно в вещественные значения, и в качестве решения классификатора выбирается класс с максимальным значением логита. Ключевой фрагмент кода инференса приведён в приложении Д, полный код — в репозитории программного обеспечения (приложение Г).

3.4 Методика исследования пространства архитектур

В разделе 2 в качестве класса нейросетевого классификатора выбрана архитектура DS-CNN; конкретные значения гиперпараметров определяются по результатам систематического экспериментального исследования, методика которого описана в настоящем подразделе.

3.4.1 Сетка обучаемых архитектур

Семейство архитектур DS-CNN определяется двумя ключевыми гиперпараметрами: числом фильтров в свёрточных слоях F и числом блоков разделимой свёртки B . Для систематического исследования зависимости точности классификации от этих параметров обучено множество моделей, образующих прямоугольную сетку в пространстве (F, B) :

- значения F : 16, 24, 32, 40, 48, 56, 64, 72, 80, 88, 96, 104, 112, 128, 160, 168, 172, 176, 184, 192, 224 (всего 21 значение, шаг 8 в основном диапазоне);
- значения B : 2, 3, 4, 5, 6, 7, 8 (всего 7 значений).

Диапазон F выбран с шагом 8, кратным размеру SIMD-регистра архитектуры Xtensa LX7 (128 бит, что соответствует 16 элементам INT8 для весов pointwise-свёрток). Базовое значение $F = 172$ эталонной конфигурации Hello Edge [2] не кратно 8 и сохранено в сетке как референсная точка для сопоставления с литературой. Диапазон B от 2 до 8 охватывает мелкие архитектуры с минимальной глубиной до моделей, в полтора раза более глубоких, чем эталонная (где $B = 6$).

Из полной декартовой сетки исключены архитектуры с очевидно недостаточной ёмкостью ($F = 16$, $B = 2$ — менее 2000 параметров) и архитектуры, которые в предварительных запусках демонстрировали расходимость обучения. Итоговое число обученных архитектур составило 134.

3.4.2 Методика обучения

Все архитектуры обучены по единой методике, описанной в разделе 3.4 (приведена ниже в подразделе 3.4.3 для удобства изложения). Общий конвейер обучения и квантизации представлен на рис. 6: набор Google Speech Commands v2 [13] разделяется на обучающую, валидационную и тестовую выборки в соотношении 80:10:10 при условии, что записи одного диктора попадают только в одну из выборок.

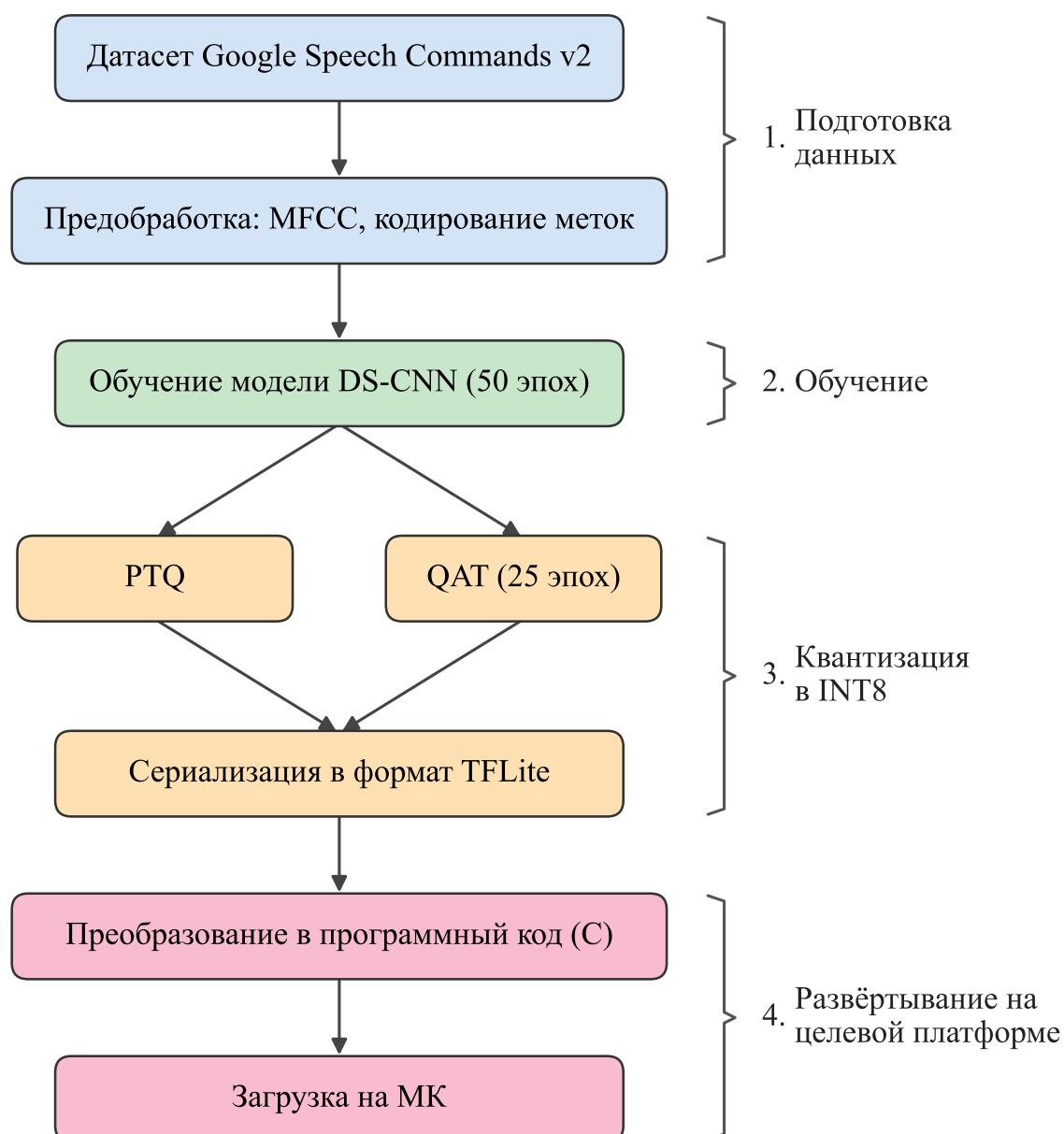


Рисунок 6 — Конвейер обучения и квантизации нейросетевой модели

Каждая FP32-модель обучается в течение 50 эпох оптимизатором Adam с косинусным расписанием снижения скорости обучения от 10^{-3} до 10^{-5} , размером мини-батча 100, функцией потерь — категориальной перекрёстной энтропией с применением сглаживания меток (label smoothing, коэффициент 0,1). Аугментация обучающих данных включает случайный временной сдвиг произнесения в пределах ± 100 мс и наложение фоновых шумов из набора Speech Commands.

Для каждой архитектуры выполняется квантизация обученной FP32-модели методом PTQ, а для подмножества архитектур (105 моделей) допол-

нительно методом QAT с дообучением на протяжении 25 эпох с пониженной скоростью обучения 10^{-4} . Тем самым общее число оцениваемых вариантов в исследовании составляет 239 точек (134 FP32, 134 PTQ, 105 QAT).

Полные исходные тексты модулей обучения и квантизации приведены в приложении Е; ключевые фрагменты — архитектура `ds_cnn.py`, цикл обучения `train.py`, конвертация PTQ и QAT — доступны в репозитории автора (приложение Г).

3.4.3 Метрики и статистическая значимость

Для каждой архитектуры в исследовании сохраняются: точность классификации FP32-модели и каждой из квантизованных моделей на тестовой выборке; размер итогового файла `.tflite`; число параметров модели; время обучения; служебные метаданные (дата обучения, конфигурация, флаг SIMD-выравнивания).

Тестовая выборка Google Speech Commands v2 содержит $n_{\text{test}} = 4154$ образца. Точность классификации на выборке такого размера измеряется с конечной статистической погрешностью. Доверительный интервал точности рассчитывается по формуле Уилсона для биномиальной пропорции:

$$p \in \left[\frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}; \frac{\hat{p} + \frac{z^2}{2n} + z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \right], \quad (11)$$

где $\hat{p}$ — наблюдаемая доля верных классификаций; n — объём тестовой выборки; z — критическое значение стандартного нормального распределения для заданного уровня доверия. Для уровня 95% ($z \approx 1,96$) и $\hat{p} \approx 0,95$, $n = 4154$, ширина доверительного интервала составляет приблизительно $\pm 0,65$ п.п. Различия в точности между моделями, не превышающие указанной величины, не являются статистически значимыми и в дальнейшем интерпретируются как шум.

Графическое представление результатов обучения всей сетки архитектур, а также Парето-граница в координатах (точность, размер модели INT8), составляют содержание подраздела 4.1.

3.5 Лабораторный измерительный стенд

Для количественной оценки энергопотребления каскадной системы разработан двухдоменный лабораторный стенд, изолирующий измеряемую систему и систему измерений в независимые электрические домены. Подобное разделение обеспечивает корректную регистрацию потребления в диапазоне от единиц микроампер (в режиме глубокого сна) до сотен миллиампер (при активном инференсе) без внесения паразитных токов системы измерений в шину питания исследуемой платы. Принципиальная схема стенда приведена в приложении А, общий вид — в приложении В, диаграмма последовательности сбора данных — в приложении И.

3.5.1 Домен 1: измеряемая система

Домен 1 включает компоненты, потребление которых является объектом измерения, и питается от литий-ионного аккумулятора 18650 номинальным напряжением 3,7 В и ёмкостью 2600 мА·ч с модулем защиты и зарядки TP4056. Состав домена:

- микроконтроллер: модуль на базе ESP32-S3 с 8 МБ флеш-памяти и 8 МБ внешней PSRAM, выведенными на разъёмы 11 пользовательскими GPIO, из которых пять поддерживают функцию RTC GPIO и могут быть использованы как источники пробуждения EXT0/EXT1 из режима глубокого сна;
- микрофон: цифровой MEMS-микрофон INMP441 производства InvenSense (TDK) [26] с интерфейсом I2S, отношением сигнал/шум 61 дБА, динамическим диапазоном 87 дБ, частотным диапазоном 60 Гц — 15 кГц и типовым потреблением 1,4 мА в активном режиме при напряжении питания 3,3 В;
- детектор звука первой ступени каскада: модуль на базе компаратора LM393 с электретным микрофонным капсулем и подстроечным резистором настройки порога; выход подключён к выводу RTC GPIO микроконтроллера для пробуждения по EXT1;
- индикация результата: светодиод, подключённый к выводу RTC GPIO для визуальной индикации обнаруженной команды;
- дисплей: TFT-дисплей на базе контроллера ST7789, подключённый по SPI, для отображения состояния системы и результата классифи-

кации в режиме отладки.

Между аккумулятором и шиной питания домена 1 в разрыв включён датчик тока и мощности — микросхема Texas Instruments INA228 в качестве чувствительного элемента стенда.

3.5.2 Датчик тока INA228

INA228 [27] представляет собой монитор мощности с 20-битным сигма-дельта-АЦП, диапазоном измерения тока от единиц микроампер до сотен миллиампер и интерфейсом I2C. В стенде использован шунтовой резистор $R_{sh} = 100$ мОм, включённый в разрыв линии питания между аккумулятором и шиной домена 1. Согласно документации производителя [27], наименьший значащий разряд канала измерения напряжения на шунте составляет 312,5 нВ, что при $R_{sh} = 100$ мОм соответствует разрешению по току на уровне единиц микроампер — величине, необходимой для регистрации потребления в режиме глубокого сна.

Калибровочный регистр SHUNT_CAL датчика программируется согласно методике производителя [27] исходя из выбранного сопротивления шунта R_{sh} и максимального ожидаемого тока I_{max} :

$$CURRENT_LSB = \frac{I_{max}}{2^{19}}, \quad (12)$$

$$SHUNT_CAL = 13107200 \cdot CURRENT_LSB \cdot R_{sh}, \quad (13)$$

где CURRENT_LSB — наименьший значащий разряд канала тока в амперах; I_{max} — максимальный ожидаемый ток в амперах; R_{sh} — сопротивление шунта в омах; числовой множитель $13,1072 \cdot 10^6$ — внутренняя константа АЦП INA228 в режиме ADCRANGE = 0. Для условий настоящей работы выбраны $R_{sh} = 0,1$ Ом и $I_{max} = 0,5$ А, что даёт $CURRENT_LSB \approx 0,954$ мкА на отсчёт и охватывает диапазон от глубокого сна до пиков активного инференса с запасом.

Период опроса регистров INA228 (VBUS, CURRENT, VSHUNT) выбран равным 200 мс. Обоснование выбора периода приведено в подразделе 3.7. Данные передаются логгеру по шине I2C.

3.5.3 Домен 2: система измерений (логгер)

Домен 2 питается от отдельного аккумулятора 18650 и электрически изолирован от домена 1; токи, потребляемые компонентами домена 2, не влияют на показания INA228. Состав домена:

- микроконтроллер логгера: ESP32-C3 Mini — одноядерный процессор RISC-V с тактовой частотой 160 МГц; через шину I2C опрашивает регистры INA228 с периодом 200 мс;
- канал передачи данных: USB-UART-мост, через который логгер транслирует пакеты измерений на персональный компьютер в формате CSV со столбцами timestamp, bus_v, current_ma, shunt_uv, power_mw, а также два бита состояния FSM, опрашиваемые через дополнительные GPIO для синхронизации с этапами обработки;
- постобработка: Python-скрипт на ПК принимает CSV-поток, строит профили мгновенного тока и сохраняет данные для последующего анализа.

Временная последовательность процессов при сборе измерительных данных приведена в приложении И: микроконтроллер ESP32-S3 кодирует состояние FSM на двух выводах GPIO; логгер ESP32-C3 параллельно опрашивает эти GPIO и читает регистры INA228 по I2C; результаты передаются на ПК через USB-UART в виде CSV-пакетов.

Принципиальная схема разделения доменов и подключения компонентов приведена в приложении А; фотография собранного стенда — в приложении В.

3.6 Методика экспериментальной апробации

Методика эксперимента определяется задачами, поставленными в разделе 1: охарактеризовать предложенную каскадную архитектуру по совокупности заявленных метрик (точность классификации, средняя потребляемая мощность, энергия на детекцию, время инференса, размер модели) и выполнить декомпозицию энергобюджета каскадного цикла по этапам обработки. Эксперимент состоит из двух последовательных частей.

3.6.1 Характеризация моделей в программной среде

В программной среде (TensorFlow 2 на персональном компьютере) для каждой из 134 обученных архитектур измеряются: итоговая точность клас-

сификации FP32-модели на тестовой выборке Google Speech Commands; точность каждой из квантизованных моделей (PTQ INT8 и, где применимо, QAT INT8), получаемая запуском квантизованной модели на той же тестовой выборке через интерпретатор TensorFlow Lite на центральном процессоре; размер итоговых файлов .tflite; число параметров. Результаты этой части эксперимента приведены в разделе 4 (подразделы 4.1 и 4.2).

3.6.2 Характеризация системы на платформе ESP32-S3

На платформе ESP32-S3 с собранным измерительным стендом для множества архитектур из Парето-границы (отбор которого обоснован в разделе 4) измеряются: профиль мгновенного тока полного цикла каскадной обработки от пробуждения по EXT1 до возврата в режим глубокого сна; время выполнения ступени 3 каскада (инференс через TFLM с ESP-NN), измеренное программно через `esp_timer_get_time()`; потребление системы в режиме глубокого сна, измеренное на протяжении длительного интервала наблюдения для усреднения шумов.

Для корреляции профиля потребления с состояниями конечного автомата используется синхронизация через GPIO, описанная в подразделе 3.1: два вывода микроконтроллера кодируют текущее состояние FSM и опрашиваются логгером одновременно с показаниями INA228. В постобработке участки CSV-файла, соответствующие каждому состоянию FSM, выделяются автоматически и обрабатываются отдельно. Результаты этой части эксперимента приведены в разделе 4 (подразделы 4.3–4.6).

3.6.3 Формулы обработки измерений

Энергия, расходуемая на одно полное прохождение цикла каскадной обработки, вычисляется численным интегрированием мгновенной мощности по времени цикла. На промежутках между двумя последовательными измерениями ($\Delta t = 200$ мс) мощность считается постоянной, что соответствует приближению прямоугольных импульсов:

$$E_{\text{cycle}} = \int_{t_0}^{t_1} P(t) dt \approx \sum_k P_k \cdot \Delta t, \quad (14)$$

где E_{cycle} — энергия одного цикла в джоулях; P_k — мгновенная мощность, зарегистрированная INA228 в k -м отсчёте, в ваттах; $\Delta t = 0,2$ с — период опроса. Энергобюджет цикла далее декомпозируется по этапам обра-

ботки разбиением суммы (14) на участки, соответствующие отдельным состояниям FSM, что и составляет содержание задачи декомпозиции, поставленной в разделе 1.

Энергия одного нейросетевого инференса, выделяемая в отдельную метрику для сопоставления моделей различной архитектуры, рассчитывается как

$$E_{\text{inf}} = \bar{I}_{\text{INFERENCE}} \cdot \bar{V} \cdot t_{\text{inf}}, \quad (15)$$

где $\bar{I}_{\text{INFERENCE}}$ — средний ток в состоянии INFERENCE (в амперах); $\bar{V}$ — среднее напряжение на шине питания (в вольтах); t_{inf} — длительность инференса (в секундах).

Согласно модели (2.1) раздела 2, средняя потребляемая мощность системы определяется выражением

$$\bar{P} = P_{\text{sleep}} + f_{\text{event}} \cdot E_{\text{cycle}}, \quad (16)$$

где $\bar{P}$ — средняя потребляемая мощность системы в ваттах; P_{sleep} — мощность в режиме глубокого сна; f_{event} — ожидаемая частота голосовых событий (число пробуждений каскада в секунду). Формула (16) представляет собой частный случай формулы (2.1) для трёхступенчатого каскада, в котором ступени 2 и 3 объединены в одно состояние полного рабочего цикла и активны только при срабатывании ступени 1.

Расчётное время автономной работы устройства от батареи ёмкостью C оценивается из соотношения

$$T_{\text{auto}} = \frac{C}{\bar{I}}, \quad (17)$$

где T_{auto} — время непрерывной работы в часах; C — ёмкость аккумулятора в миллиампер-часах; $\bar{I}$ — средний ток потребления системы в миллиамперах, рассчитанный из $\bar{P}$ и номинального напряжения питания. Формула (17) используется в разделе 4 для пересчёта экспериментально измеренных значений $\bar{P}$ в производный показатель, имеющий прямое практическое значение для оценки применимости системы.

3.6.4 Обоснование выбора периода опроса 200 мс

Период опроса регистров INA228, равный 200 мс, выбран как компромисс между двумя противоположными требованиями. Меньший период позволил бы разрешить более короткие переходные процессы, однако привёл бы к существенному увеличению объёма CSV-файла и нагрузке на канал передачи данных USB-UART, ограниченный пропускной способностью 115 200 бод. Больший период увеличил бы погрешность интегрирования энергии на коротких этапах каскада (в первую очередь на этапе MFCC длительностью порядка сотен миллисекунд).

При выбранном периоде 200 мс на каждом из основных этапов каскада (длительностью от 500 мс до 3 с) укладывается от трёх до пятнадцати отсчётов, что обеспечивает приемлемую точность интегрирования энергии методом прямоугольных импульсов. Внутреннее усреднение INA228 на окне $\approx 4,2$ мс сглаживает короткие пики переходных процессов, а энергия микросекундных переходов между состояниями пренебрежимо мала на фоне многосекундных рабочих этапов. Тем самым выбранный период обеспечивает корректную интегральную оценку энергобюджета цикла при сохранении приемлемого объёма данных.

3.7 Выводы по разделу

В настоящем разделе описана практическая реализация каскадной KWS-системы на платформе ESP32-S3, методика систематического исследования архитектур нейросетевого классификатора и методика экспериментальной апробации. Получены следующие результаты.

- 1) Реализован конечный автомат каскада из семи состояний с аппаратно-программной синхронизацией через GPIO, обеспечивающей возможность отдельного измерения энергопотребления каждого этапа цикла обработки. Реализация выполнена средствами фреймворка ESP-IDF; программная архитектура декомпозирована на независимые модули в соответствии с компонентной диаграммой рис. 3;
- 2) Реализован тракт обработки звука ступени 2 каскада, включающий захват аудиосигнала через I2S с цифрового MEMS-микрофона INMP441, сегментацию сигнала на фреймы по 40 мс с шагом 20 мс,

применение оконной функции Хэмминга, дискретное преобразование Фурье размером 1024 точки с аппаратным ускорением библиотеки ESP-DSP, проекцию на 40 треугольных мел-фильтров, логарифмирование и дискретное косинусное преобразование с сохранением 10 кепстральных коэффициентов (рис. 4);

- 3) Реализован инференс квантизованной модели DS-CNN через фреймворк TensorFlow Lite Micro с аппаратным ускорением библиотеки ESP-NN. Арена тензоров размером 140 КБ размещена во внутреннем SRAM микроконтроллера для обеспечения корректной работы SIMD-ускорения;
- 4) Выполнено систематическое исследование пространства архитектур DS-CNN: обучено 134 архитектуры в сетке $F \in \{16, \dots, 224\}$, $B \in \{2, \dots, 8\}$, для каждой выполнена квантизация PTQ, для 105 архитектур дополнительно QAT, итого получено 239 оцениваемых вариантов (рис. 7, 8);
- 5) Сформулирована методика статистической оценки точности с использованием доверительного интервала Уилсона (11), обеспечивающая содержательную интерпретацию различий между моделями;
- 6) Разработан двухдоменный лабораторный измерительный стенд (приложения А, В), изолирующий измеряемую систему и систему измерений в независимые электрические домены. В качестве чувствительного элемента использован монитор мощности INA228 с шунтом 100 мОм, обеспечивающий разрешение по току на уровне единиц микроампер и пригодный для регистрации потребления от глубокого сна до пиков активного инференса;
- 7) Сформулирована методика экспериментальной апробации, включающая характеризацию моделей в программной среде и характеризацию системы на платформе ESP32-S3, а также приведены формулы обработки экспериментальных данных — расчёта энергии цикла методом численного интегрирования (14), расчёта энергии одного инференса (15), оценки средней потребляемой мощности (16) и расчётного времени автономной работы (17).

Результаты применения описанной методики и интерпретация полученных данных составляют содержание следующего раздела.

4 Анализ результатов

В настоящем разделе изложены результаты экспериментальной апробации системы, реализованной по методике раздела 3, и их интерпретация в терминах задач, поставленных во введении. Структура раздела построена зеркально к перечню задач: подраздел 4.1 посвящён результатам систематического исследования архитектур DS-CNN и Парето-анализу по размеру модели; подраздел 4.2 излагает сравнительный анализ методов квантизации (задача 3); подраздел 4.3 содержит анализ узких мест времени инференса на платформе ESP32-S3; подраздел 4.4 излагает обоснование выбора финальной модели по Парето-критерию «точность $\times$ энергия инференса»; подраздел 4.5 представляет оценку качества классификации выбранной модели; подразделы 4.6–4.8 излагают результаты измерений на платформе ESP32-S3, декомпозицию энергобюджета каскадного цикла и сравнительную оценку автономности (задачи 4 и 5); подраздел 4.9 обсуждает дополнительные наблюдения и направления дальнейшего развития работы.

4.1 Систематическое исследование архитектур DS-CNN

4.1.1 Точность в пространстве гиперпараметров

По методике, описанной в подразделе 3.4, обучено 134 архитектуры DS-CNN в сетке $F \in \{16, \dots, 224\}$, $B \in \{2, \dots, 8\}$; для каждой выполнена квантизация PTQ, для 105 архитектур дополнительно QAT. Результаты обучения в пространстве гиперпараметров представлены тепловой картой точности INT8-моделей на рис. 7.

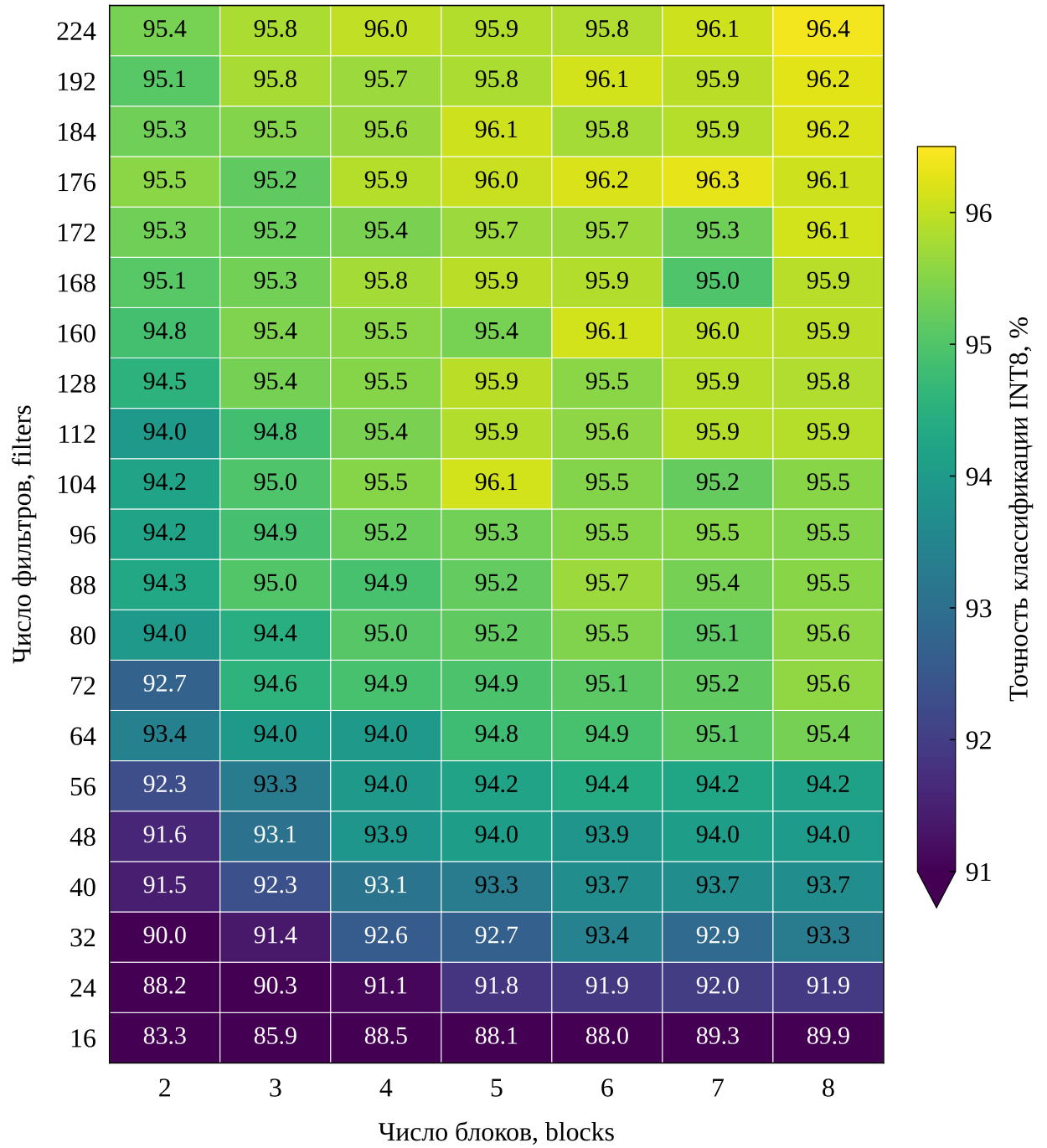


Рисунок 7 — Тепловая карта точности INT8-моделей DS-CNN в пространстве гиперпараметров (filters × blocks)

Анализ тепловой карты выявляет две закономерности. *Первая* — точность выходит на статистически значимое плато при $F \geq 80$ и $B \geq 4$, в пределах которого изменение гиперпараметров не приводит к различиям в точности, превышающим доверительный интервал Уилсона $\pm 0,65$ п.п. для тестовой выборки $n = 4154$ (формула 11 раздела 3). *Вторая* — для архитектур с малым числом фильтров ($F < 64$) глубина сети оказывает существенное влияние на точность, тогда как для $F \geq 80$ зависимость точности от глу-

бины сглаживается. Содержательная интерпретация этих наблюдений: при ограниченной ёмкости свёрточных слоёв сеть выигрывает от дополнительных уровней абстракции; при достаточной ёмкости дополнительная глубина не даёт значимого выигрыша.

4.1.2 Парето-граница по размеру модели

Парето-граница архитектур в координатах «точность × размер модели INT8» по 134 обученным архитектурам и 239 оцениваемым точкам (FP32, PTQ, QAT) приведена на рис. 8.

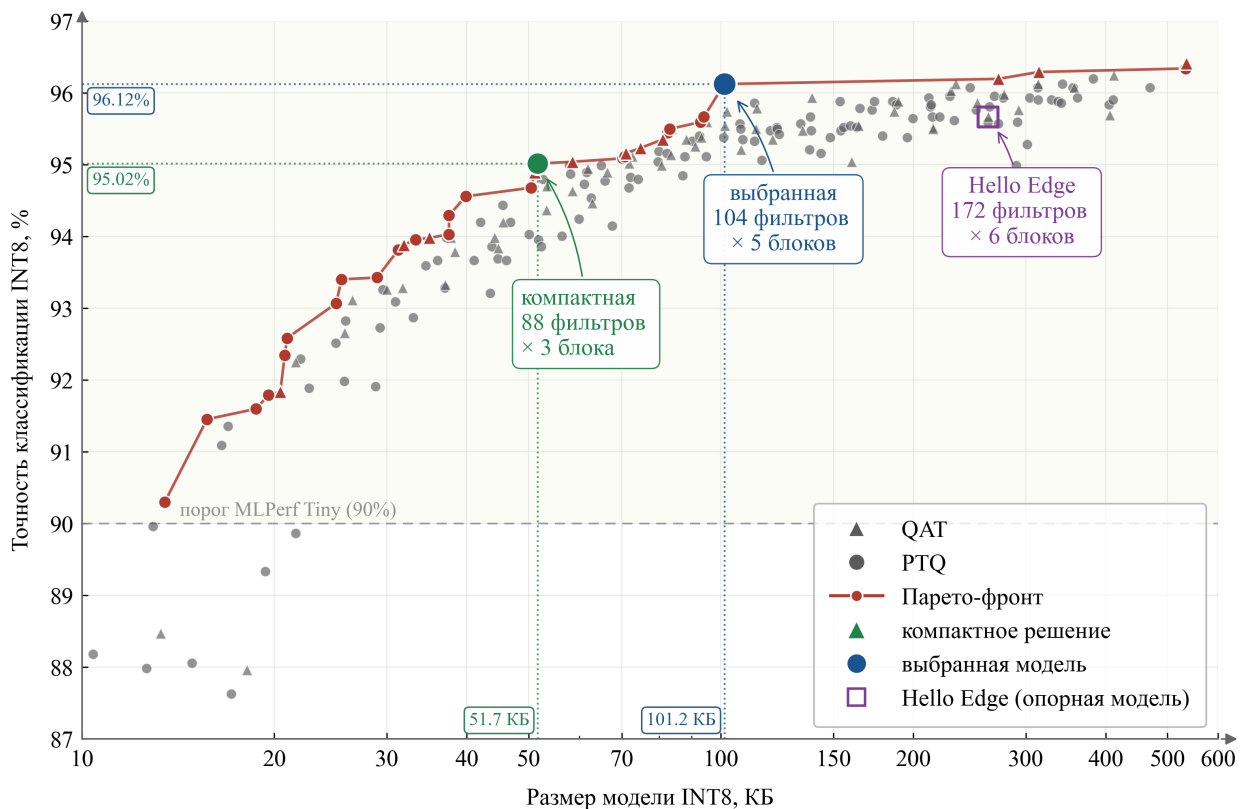


Рисунок 8 — Парето-граница архитектур DS-CNN в координатах точность INT8 × размер модели

Анализ Парето-границы позволяет выделить четыре характерные зоны.

Зона ультракомпактных моделей (< 20 КБ, точность менее 92%) — архитектуры с предельно малым числом фильтров, не удовлетворяющие порогу точности 90% бенчмарка MLPerf Tiny [14] либо находящиеся на его границе. Применимы только при крайне строгих ограничениях на флеш-память.

Зона компромисса (20–80 КБ, точность 92–95%) — архитектуры с умеренным числом фильтров, обеспечивающие быстрый рост точности при увеличении размера модели. Содержательный градиент Парето-границы в этой

зоне — порядка 0,1 п.п. точности на каждый дополнительный килобайт размера модели.

Зона оптимума (80–110 КБ, точность 95–96%) — архитектуры, для которых точность достигает значений в пределах 1 п.п. от верхней границы исследованного диапазона. Парето-кнее данной зоны находится в области моделей с $F \approx 100$, $B = 5$, и обоснование выбора финальной модели именно из этой зоны проводится в подразделе 4.4.

Зона насыщения (более 110 КБ) — архитектуры, в которых дальнейшее увеличение размера не приводит к статистически значимому росту точности. Эталонная конфигурация Hello Edge DS-CNN-M [2] ($F = 172$, $B = 6$, размер ≈ 260 КБ) располагается в этой зоне и не является Парето-оптимальной для исследованной задачи на платформе ESP32-S3.

4.2 Сравнение методов квантизации PTQ и QAT

В соответствии с задачей 3 введения проведено сравнение методов квантизации PTQ и QAT. Сопоставление выполнено на двух уровнях обобщения. На первом уровне — детальное сравнение для четырёх представительных архитектур, перекрывающих весь диапазон исследованных размеров: компактная f_{24_b5} (20 КБ), средняя f_{72_b4} (51 КБ), эталонная Hello Edge f_{176_b6} (272 КБ) и выбранная финальная f_{104_b5} (101 КБ). На втором уровне — агрегированный статистический анализ парных различий точности $\Delta = \text{асс}_{\text{QAT}} - \text{асс}_{\text{PTQ}}$ по всем 63 архитектурам, для которых проведены оба метода. Результаты обоих уровней сравнения представлены на рис. 9.

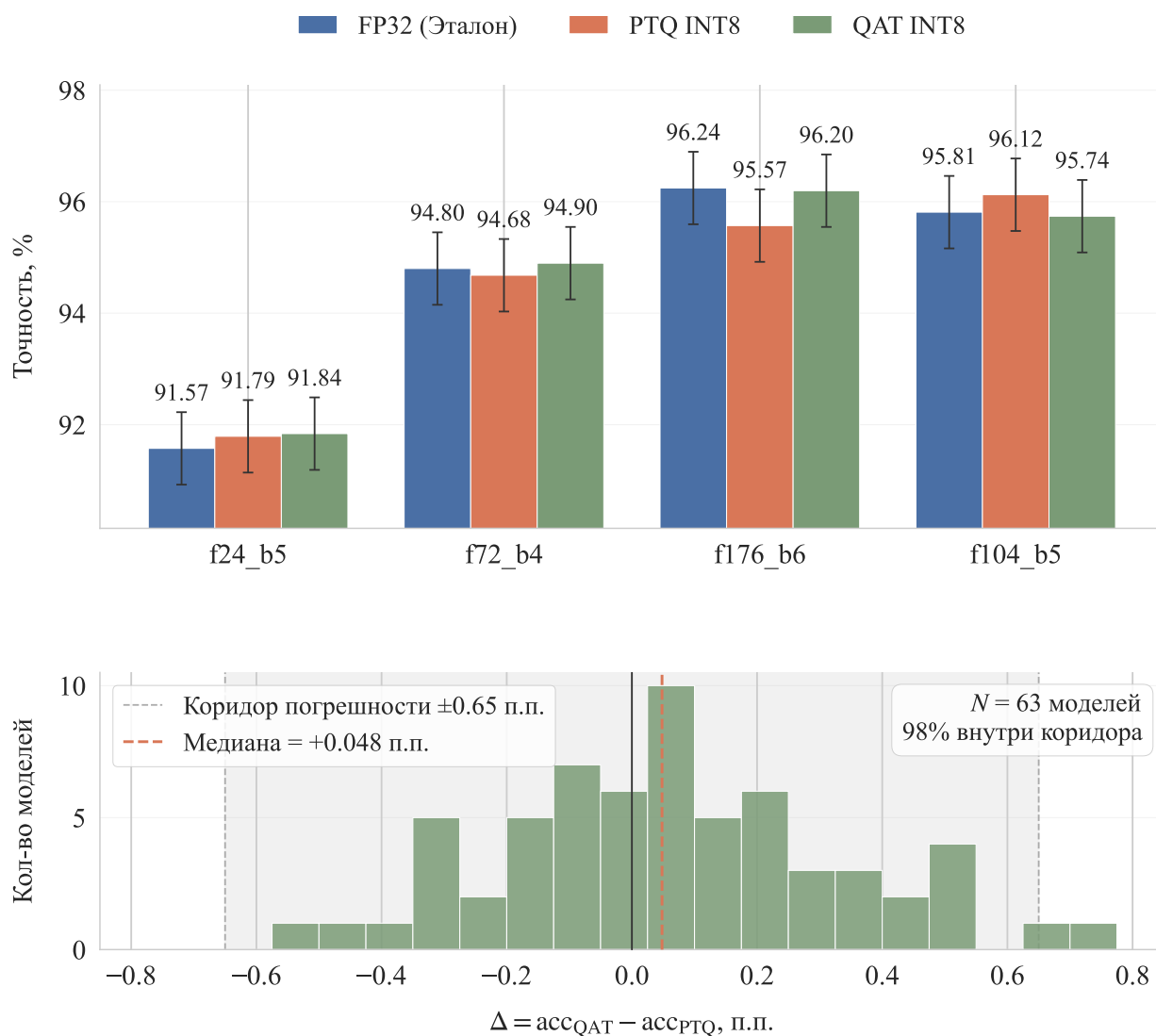


Рисунок 9 — Сравнение методов квантизации PTQ и QAT. Сверху: точность FP32, PTQ INT8 и QAT INT8 для четырёх представительных архитектур с доверительными интервалами Уилсона $\pm 0,65$ п.п. Снизу: распределение парных различий $\Delta = \text{acc}_{\text{QAT}} - \text{acc}_{\text{PTQ}}$ по 63 архитектурам с обоими методами квантизации; серой полосой обозначен коридор статистической погрешности

Из приведённого сравнения следуют три наблюдения.

Первое. На всех четырёх представительных архитектурах различия в точности между методами PTQ и QAT не превышают 0,67 п.п. (максимальное различие 0,63 п.п. для *f176_b6*: 95,57% PTQ против 96,20% QAT) и в большинстве случаев существенно меньше: для *f24_b5* различие составляет 0,05 п.п., для *f72_b4* — 0,22 п.п., для *f104_b5* — 0,38 п.п. Все различия лежат в пределах либо непосредственно у границы доверительного интервала Уилсона ($\pm 0,65$ п.п.) и не являются статистически значимыми.

Второе. Агрегированная статистика по 63 парам PTQ/QAT подтверждает наблюдение первого уровня: 98% парных различий Δ лежат внутри коридора $\pm 0,65$ п.п. при медиане $+0,048$ п.п. Распределение различий симметрично относительно нуля, что указывает на отсутствие систематического преимущества одного метода над другим для исследованного класса задач квантизации DS-CNN до INT8.

Третье. В отдельных случаях точность квантизованной модели превышает точность исходной FP32-модели на величину порядка 0,3 п.п. (для архитектуры $F = 104$, $B = 5$ — 96,12% PTQ INT8 против 95,81% FP32; для $F = 176$, $B = 6$ — 96,24% FP32 против 96,20% QAT INT8 и 95,57% PTQ INT8). Подобное наблюдение объясняется статистическим характером оценки и согласуется с независимыми результатами Сёренсена и др. [28], где для аналогичной архитектуры на платформе Cortex-M4 также отмечались случаи точности INT8-моделей не ниже FP32. Содержательно эффект интерпретируется как регуляризационное действие квантизации: округление весов и активаций вносит шум, который для умеренно переобученной FP32-модели играет роль сглаживающего фактора [4].

Вывод по задаче 3 введения. Применительно к задаче квантизации DS-CNN до INT8 на платформе ESP32-S3 методы PTQ и QAT обеспечивают статистически неразличимую точность. Преимущество PTQ состоит в существенно меньшей вычислительной стоимости — отсутствие цикла дообучения сокращает время получения квантизованной модели с десятков минут до единиц секунд при сопоставимой точности. Это делает PTQ предпочтительным методом при условии достаточности INT8-разрядности для целевой задачи.

4.3 Анализ времени инференса на платформе ESP32-S3

Послойная декомпозиция времени инференса, полученная программным профилированием через `esp_timer_get_time()` для подмножества архитектур разного размера, приведена на рис. 10 (в абсолютных единицах) и рис. 11 (в относительных долях).

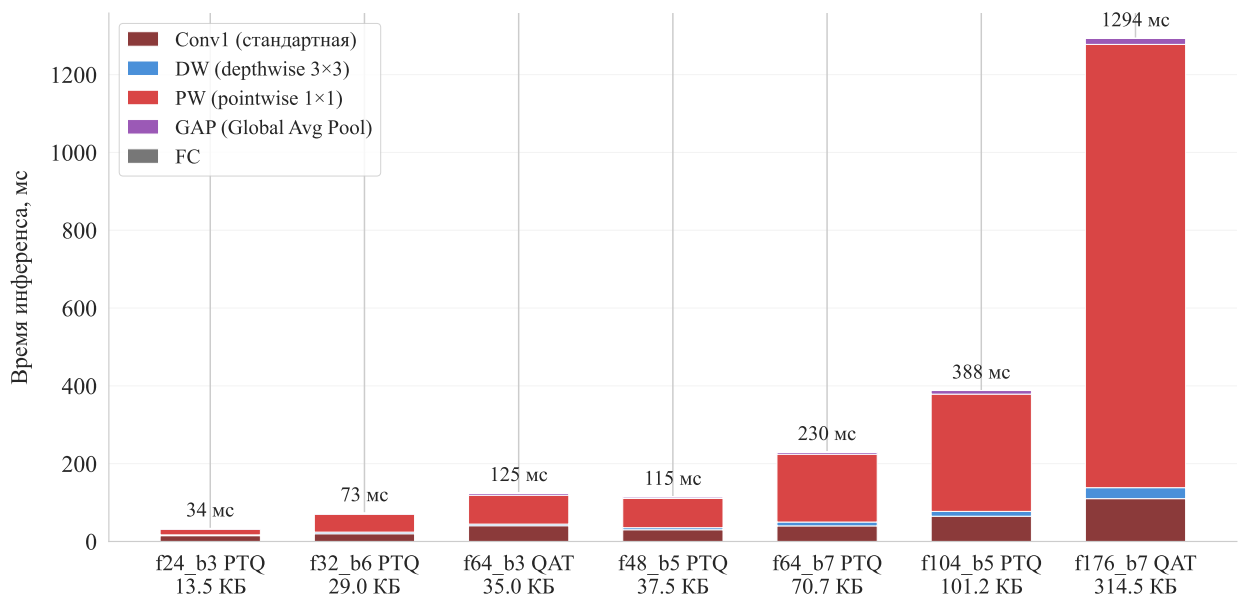


Рисунок 10 — Постлойная декомпозиция времени инференса DS-CNN на ESP32-S3 для архитектур различного размера, абсолютные значения

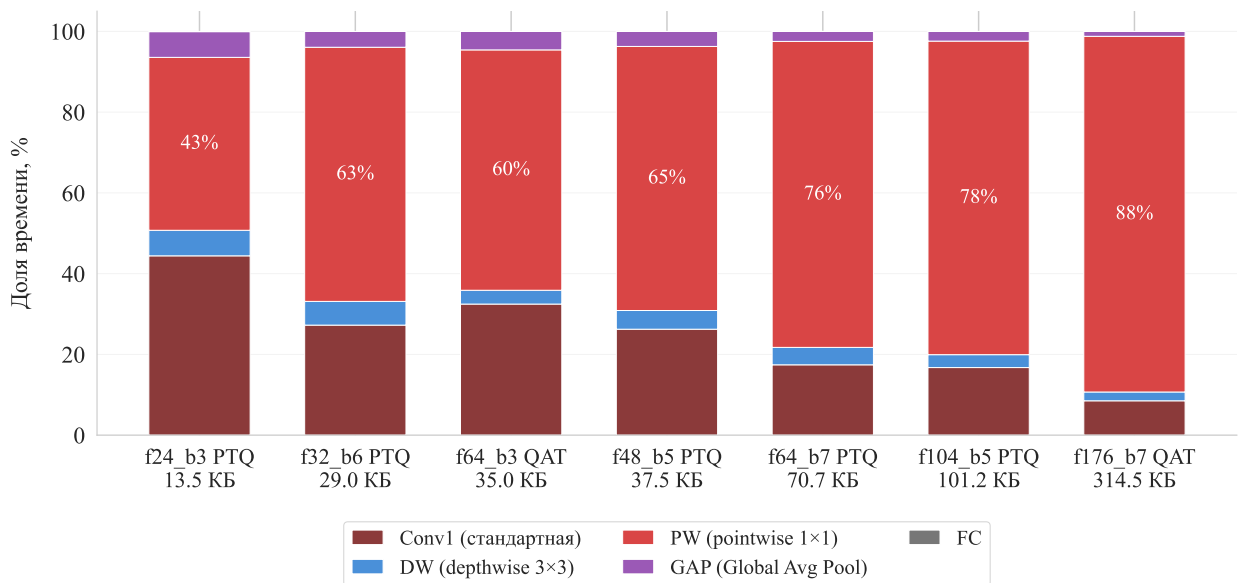


Рисунок 11 — Постлойная декомпозиция времени инференса DS-CNN на ESP32-S3 для архитектур различного размера, относительные доли

Из приведённой декомпозиции следуют три инженерных вывода.

Первый. На всех исследованных архитектурах доминирующий вклад в время инференса вносят pointwise-свёртки — от примерно 50% общего времени для компактных архитектур ($F < 48$) до более 85% для архитектур с $F > 128$. Стандартная свёртка первого слоя (stem_conv), depthwise-свёртки и финальные операции (Mean, FullyConnected) в сумме составляют от четверти до половины времени инференса для компактных моделей и менее одной

шестой для крупных.

Второй. С ростом числа фильтров F время выполнения pointwise-свёрток растёт сверхлинейно (фактор $\sim F^2$, поскольку pointwise-свёртка содержит $F \times F$ умножений на выходной пиксель), тогда как остальные операции масштабируются линейно или сублинейно. Это объясняет наблюдаемую квадратичную зависимость полного времени инференса от числа фильтров и полностью согласуется с положением финальной модели *f104_b5* в области умеренных F , обеспечивающей баланс между точностью и временем инференса.

Третий. Подтверждение использования аппаратного SIMD-ускорения для pointwise-свёрток получено инструментальным контролем: внутри conv.cc библиотеки ESP-NN зафиксирована активация оптимизированного «fast path» кода для всех протестированных конфигураций (число каналов кратно 8). Несмотря на это, доминирование pointwise-свёрток сохраняется, что указывает на ограничение, накладываемое самим объёмом вычислений, а не эффективностью их аппаратного ускорения.

Практическое следствие. Оптимизация энергопотребления нейросетевого инференса на платформе общего назначения требует прежде всего сокращения числа фильтров архитектуры, а не углубления сети либо переноса вычислений на альтернативные аппаратные ускорители. Этот вывод послужил дополнительным обоснованием выбора финальной модели с умеренным числом фильтров $F = 104$ при увеличенной глубине $B = 5$ относительно компромисса «глубже — уже».

4.4 Выбор финальной модели по Парето-критерию энергия $\times$ точность

Размер модели характеризует архитектуру в отрыве от целевой платформы. Для финального выбора модели, развёртываемой на ESP32-S3, основным критерием служит энергия одного нейросетевого инференса, рассчитываемая по формуле (15) раздела 3.

4.4.1 Методика расчёта энергии инференса

Энергия одного нейросетевого инференса определяется как интеграл мгновенной мощности по интервалу состояния INFERENCE цикла каскад-

ной обработки:

$$E_{\text{инф}} = \int_{t_0}^{t_1} U(t) \cdot I(t) dt, \quad (18)$$

где $U(t)$ и $I(t)$ — мгновенные значения напряжения и тока шины питания микроконтроллера, измеренные монитором мощности INA228 [27] с периодом дискретизации, согласованным с программными метками состояний; t_0 и t_1 — моменты входа и выхода из состояния INFERENCE соответственно. Численное интегрирование выполняется методом трапеций по дискретным отсчётам (U_k, I_k, t_k) , что компенсирует изменение мгновенного тока в пределах состояния и даёт более точную оценку, чем произведение средних значений.

Для подавления случайной составляющей измерения каждый цикл каскадной обработки выполняется на лабораторном стенде троекратно подряд; в качестве оценки энергии инференса принято среднее арифметическое по трём реализациям, в качестве оценки погрешности — выборочное стандартное отклонение.

4.4.2 Энергия инференса по подмножеству Парето-границы

Для подмножества архитектур Парето-границы по размеру модели (рис. 8) проведены прямые измерения энергии инференса по формуле (18). Дополнительно в выборку включены архитектуры с шестью depthwise-separable блоками ($B = 6$) различной ширины ($F \in \{64, 96, 128, 160, 172, 176\}$), позволяющие охарактеризовать поведение системы в диапазоне размеров модели от 100 до 270 КБ. Результаты сведены в табл. 3 и графически представлены на рис. 12.

Таблица 3 — Энергетические характеристики архитектур подмножества Парето-границы

Арх-ра	Метод квантиз.	Размер, КБ	Время, мс	Ток, мА	Энергия, мДж	Точность, %
f24_b5	QAT	20.4	120	81.2	39.78 ± 0.41	91.84
f24_b3	PTQ	13.5	120	81.5	40.09 ± 1.01	90.30
f40_b2	PTQ	15.7	120	82.1	40.50 ± 1.67	91.45
f32_b4	PTQ	20.9	120	82.1	40.87 ± 0.88	92.59

Продолжение таблицы 3

Арх-ра	Метод квантиз.	Размер, КБ	Время, мс	Ток, мА	Энергия, мДж	Точность, %
f32_b6	PTQ	29.0	140	82.6	46.57 ± 11.71	93.43
f48_b3	PTQ	25.0	139	83.3	47.81 ± 11.59	93.07
f48_b4	QAT	31.9	160	84.3	54.91 ± 10.28	93.89
f64_b2	PTQ	25.5	180	82.8	60.76 ± 1.85	93.40
f48_b5	PTQ	37.5	180	83.5	62.04 ± 0.50	94.03
f64_b3	QAT	35.0	200	82.8	66.74 ± 11.50	93.98
f72_b3	PTQ	39.9	220	83.5	74.02 ± 11.75	94.56
f72_b4	QAT	51.1	240	83.0	82.16 ± 0.18	94.90
f88_b3	PTQ	51.7	280	83.1	96.02 ± 12.43	95.02
f64_b7	PTQ	70.7	300	83.1	102.01 ± 1.62	95.11
f80_b5	QAT	71.0	340	84.2	115.27 ± 11.08	95.16
f88_b6	PTQ	94.0	417	83.3	142.64 ± 8.92	95.67
f104_b5	PTQ	101.2	440	83.2	149.64 ± 9.52	96.12
f176_b7	QAT	314.5	1347	82.9	451.22 ± 7.58	96.29

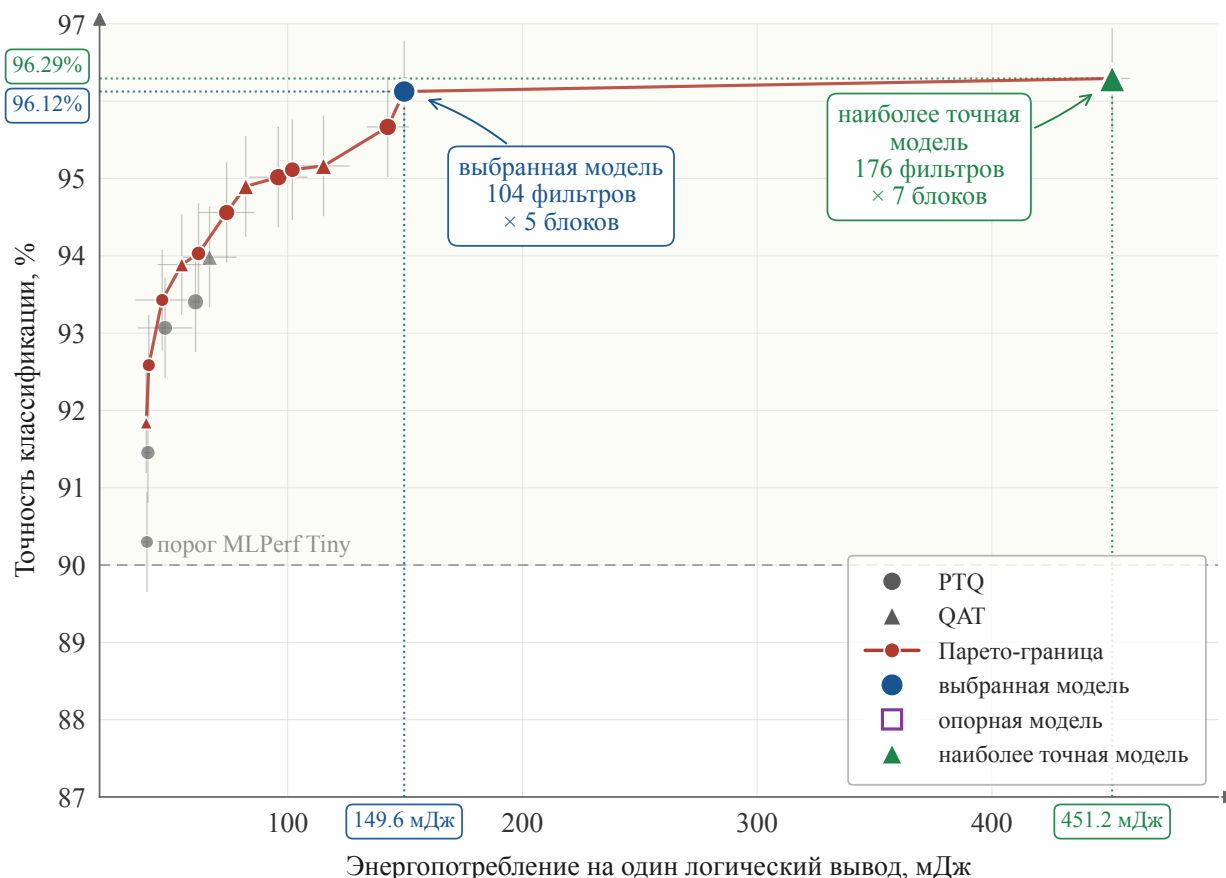


Рисунок 12 — Парето-граница архитектур DS-CNN в координатах точность классификации × энергия одного нейросетевого инференса на платформе ESP32-S3. Серыми маркерами показаны архитектуры, доминируемые по совокупности двух критериев; красной линией соединены архитектуры Парето-границы. Подсвечены выбранная финальная модель f_{104_b5} (синий) и архитектура с максимальной достижимой точностью в исследованном диапазоне f_{176_b7} (зелёный)

Полученный график иллюстрирует характерную форму границы Парето в задачах оптимизации нейросетевых архитектур: резкий прирост точности в области малых значений энергии (от 25 до 100 мДж) сменяется протяжённым участком насыщения, на котором дальнейшее увеличение вычислительных затрат сопровождается прибавкой в точности классификации, не превышающей единиц десятых процентного пункта. Точка излома границы Парето локализуется в окрестности 150 мДж на инференс, что соответствует выбранной финальной модели f_{104_b5} .

4.4.3 Обоснование выбора финальной модели

Из множества архитектур, удовлетворяющих жёстким критериям (точность $\geq 90\%$ из требования MLPerf Tiny [14]; размер модели в пределах внутреннего SRAM платформы для совместимости с SIMD-ускорением библио-

теки ESP-NN; время инференса в пределах разумной интерактивной задержки), в качестве финальной модели для развёртывания выбрана архитектура DS-CNN с $F = 104$, $B = 5$, квантизованная методом PTQ (далее — f104_b5 PTQ).

Обоснование выбора по совокупности критериев:

- *Положение в точке излома границы Парето.* Выбранная модель располагается на верхне-левой огибающей графика рис. 12 в точке излома, после которой граница переходит в режим насыщения по точности. Энергия одного инференса составляет 149,6 мДж — в 3 раза меньше по сравнению с архитектурой максимальной точности f176_b7 (451,2 мДж) при разнице в точности 0,17 п.п. (96,12% против 96,29%), не превышающей статистической погрешности оценки.
- *Точность классификации* 96,12% превышает требование MLPerf Tiny (90%) на 6,1 п.п. и отличается от максимальной достижимой в исследованном диапазоне точности 96,29% (архитектура f176_b7 QAT) на 0,17 п.п., что находится в пределах статистической погрешности оценки точности на тестовой выборке.
- *Превосходство над опорной моделью Hello Edge [2].* Выбранная архитектура достигает точности, не уступающей опорной модели, при существенно меньшем размере (101 КБ против ≈ 272 КБ) и меньшей энергии одного инференса. Указанное соотношение демонстрирует, что использованный подход к выбору архитектуры обеспечивает результат, превосходящий типовое решение литературы по совокупности рассматриваемых метрик.
- *Метод квантизации PTQ* обеспечивает достижение указанной точности при сжатии модели в 3,73 раза относительно FP32-варианта (377 КБ FP32 $\rightarrow$ 101 КБ INT8) без использования цикла дообучения. Согласно результатам подраздела 4.2, различие между PTQ и QAT для рассматриваемой архитектуры не превышает погрешности измерения точности на тестовой выборке, что делает PTQ предпочтительным по совокупности факторов сложности реализации и стоимости разработки.

Совокупность приведённых критериев позволяет считать архитектуру

f104_b5 PTQ обоснованным выбором финальной модели для развёртывания на платформе ESP32-S3 в составе разрабатываемой системы распознавания ключевых слов.

4.5 Качество классификации выбранной модели

4.5.1 Кривые обучения

Динамика обучения FP32-варианта выбранной модели — кривые функции потерь и точности классификации на обучающей и валидационной выборках — представлена на рис. 13.

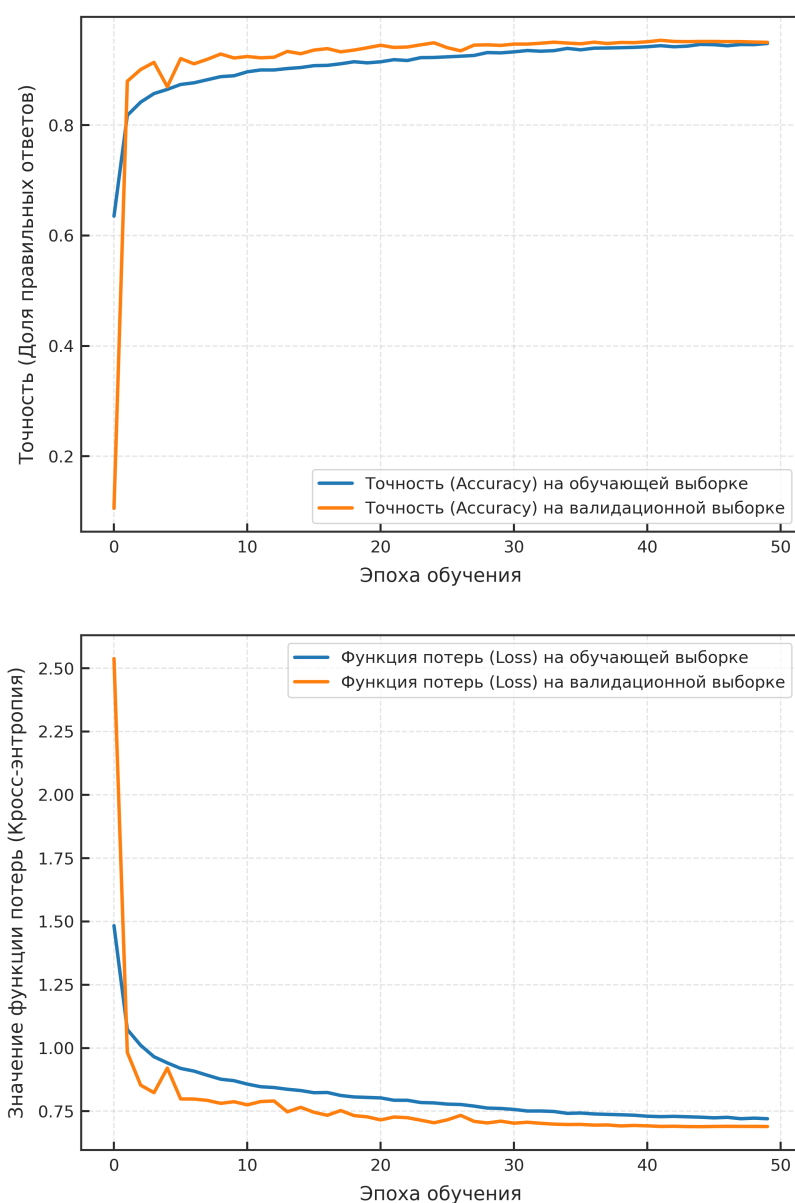


Рисунок 13 — Кривые обучения модели $f104_b5$ (FP32): функция потерь и точность классификации на обучающей и валидационной выборках по эпохам

Валидационная точность стабилизируется в пределах последних эпох обучения, разрыв между обучающей и валидационной точностью сохраняется в пределах нескольких процентных пунктов, что свидетельствует о сходимости обучения без признаков существенного переобучения — эффект сглаживания меток (label smoothing) и аугментации данных временным сдвигом обеспечивает необходимую регуляризацию.

4.5.2 Метрики качества классификации по классам

Подробные метрики качества классификации квантизованного варианта PTQ INT8 выбранной модели на тестовой выборке Google Speech Commands v2 приведены в таблице 4.

Таблица 4 — Метрики качества классификации модели *f104_b5* PTQ INT8 на тестовой выборке Google Speech Commands v2

Класс	Precision	Recall	F1-score	Support
yes	0,9647	0,9785	0,9716	419
no	0,9429	0,9778	0,9600	405
up	0,9543	0,9341	0,9441	425
down	0,9348	0,9532	0,9439	406
left	0,9780	0,9733	0,9757	412
right	0,9949	0,9848	0,9898	396
on	0,9642	0,9520	0,9581	396
off	0,9500	0,9453	0,9476	402
stop	0,9781	0,9781	0,9781	411
go	0,9370	0,9254	0,9312	402
unknown	1,0000	1,0000	1,0000	40
silence	0,7368	0,7000	0,7179	40
Macro avg	0,9446	0,9419	0,9432	4154
Weighted avg	0,9581	0,9581	0,9580	4154

Поклассовый анализ выявляет следующее.

Класс silence демонстрирует ощутимо более низкие метрики ($F1 = 0,72$) при минимальном размере выборки данного класса (40 образцов из 4154). Низкое качество классификации класса *silence* объясняется методикой обучения, в которой образцы тишины генерировались искусственно из фоновых шумов набора Google Speech Commands; в реальных условиях работы системы класс *silence* играет роль фильтра ложных срабатываний и не критичен для качества основной задачи распознавания команд.

Класс unknown демонстрирует точность 100%, что объясняется харак-

тером класса: он объединяет все нецелевые слова из набора и для классификатора представляет более выраженный паттерн отличия от целевых команд.

Целевые команды демонстрируют F1 в диапазоне 0,93–0,99 с macro avg F1 = 0,9432 (по 12 классам) и weighted avg F1 = 0,9580. Минимальные значения F1 наблюдаются на парах акустически близких слов (*no/go*, *on/off*) — ошибки концентрируются именно на этих парах, что подтверждается анализом матриц ошибок (рис. 14–16).

4.5.3 Матрицы ошибок

Матрицы ошибок FP32, PTQ INT8 и QAT INT8 вариантов выбранной модели на тестовой выборке представлены на рис. 14–16.

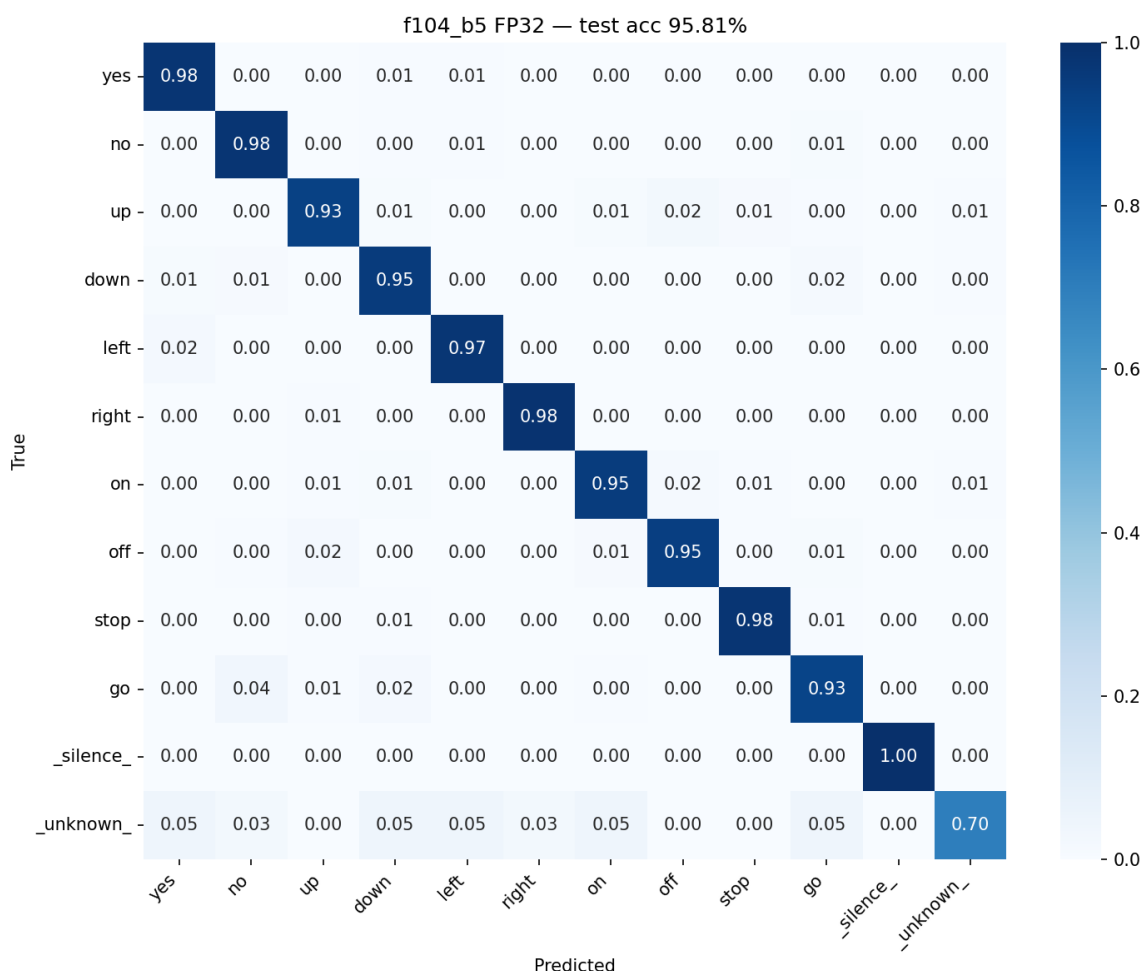


Рисунок 14 — Матрица ошибок базовой модели *f104_b5* (FP32)



Рисунок 15 — Матрица ошибок квантованной модели $f104_b5$ (PTQ INT8)



Рисунок 16 — Матрица ошибок квантованной модели $f104_b5$ (QAT INT8)

Визуальное сравнение матриц подтверждает количественный вывод подраздела 4.2: структура ошибок сохраняется при переходе от FP32 к INT8 (как PTQ, так и QAT). Основная масса ошибок концентрируется в нескольких внедиагональных ячейках, соответствующих парам акустически близких слов; квантизация не вносит качественно новых паттернов ошибок, а лишь равномерно снижает уверенность классификатора на тех же примерах, на которых модель ошибалась в FP32-варианте.

При анализе результатов обнаруживается характерный феномен: точность квантованной модели f_{104_b5} после применения процедуры PTQ INT8 (96,12%) оказалась незначительно выше исходного варианта FP32 (95,81%). Различие в 0,31 п.п. не является статистически значимым и полностью находится в пределах доверительного интервала Уилсона ($\pm 0,65$ п.п.) для используемого объёма тестовой выборки при $n = 4154$. Подобное поведение, при котором целочисленная модель показывает точность на уровне или несколько выше исходной, согласуется с независимыми результатами Сёренсена и др. [28].

С содержательной инженерной точки зрения данный эффект объясняется тем, что квантование весов и активаций до 8-битного представления вносит контролируемый шум округления, который действует как форма регуляризации. Для исходной FP32-модели, обладающей некоторой избыточной ёмкостью, этот сглаживающий фактор позволяет снизить слабую степень переобучения и незначительно улучшить обобщающую способность на независимой тестовой выборке. Подобный регуляризационный эффект дискретизации весов подробно описан в работе Jacob и др. [4].

4.6 Профиль энергопотребления полного цикла каскадной обработки

Профиль мгновенного тока, регистрируемый INA228 в течение полного цикла каскадной обработки от пробуждения по сигналу EXT1 до возврата в режим глубокого сна, для выбранной модели f_{104_b5} PTQ представлен на рис. 17. На профиле визуально различимы все состояния конечного автомата, что подтверждает корректность схемы GPIO-синхронизации, описанной в подразделе 3.1.

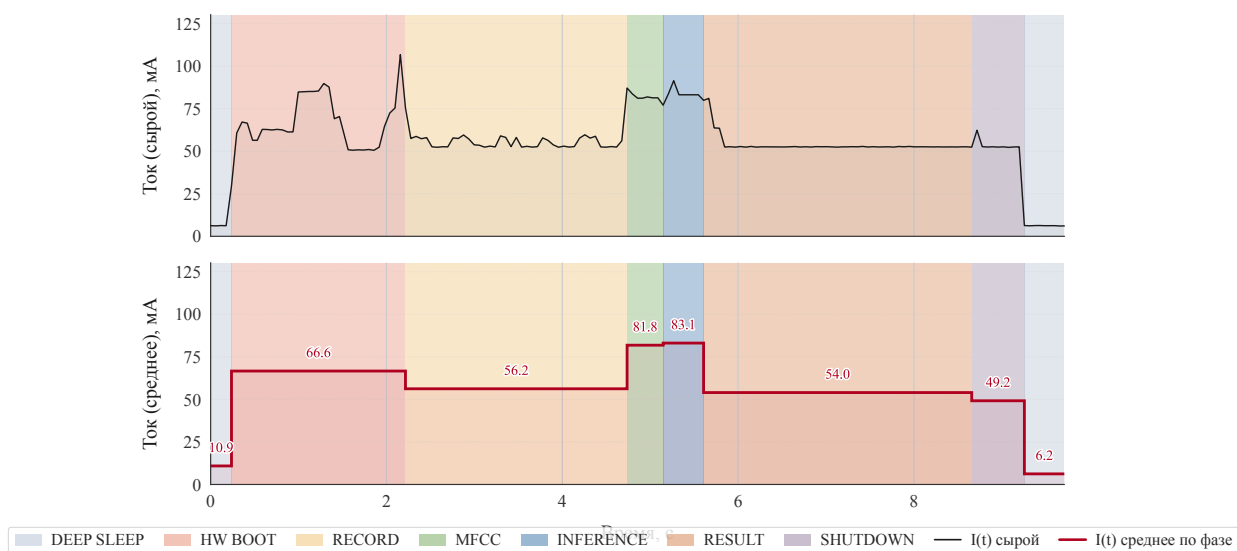


Рисунок 17 — Профиль мгновенного тока полного цикла каскадной обработки выбранной модели $f104_b5$ PTQ на платформе ESP32-S3. Сверху: исходный сигнал тока с детальной разметкой состояний FSM. Снизу: усреднённые по состояниям значения тока с численными метками

Длительности и средние значения тока на каждом участке, полученные автоматическим разделением CSV-файла измерений по биту состояния FSM, сведены в табл. 5.

Таблица 5 — Длительности и средние значения тока на этапах каскадной обработки для модели $f104_b5$ PTQ

Этап (состояние FSM)	Длительность, мс	Средний ток, мА
Глубокий сон (DEEP_SLEEP)	—	6,2
Инициализация (HW_BOOT)	1982	66,6
Захват аудио (RECORD)	2519	56,2
Извлечение MFCC (MFCC)	410	81,8
Инференс DS-CNN (INFERENCE)	460	83,1
Завершение (SHUTDOWN)	597	49,2

4.7 Декомпозиция энергобюджета цикла

Энергия, расходуемая на одно полное прохождение каскадного цикла, вычислена по формуле (14) численным интегрированием мгновенной мощности по показаниям INA228. Декомпозиция энергобюджета по состояниям конечного автомата приведена в табл. 6 и графически — на рис. 18.

Таблица 6 — Декомпозиция энергопотребления одного цикла каскадной обработки для модели $f104_b5$ PTQ

Этап	Длит., мс	Энергия, мДж	Доля бюджета, %
Инициализация (HW_BOOT)	1982	532,57	34,9
Захват аудио (RECORD)	2519	569,74	37,4
Извлечение MFCC (MFCC)	410	137,19	9,0
Инференс DS-CNN (INFERENCE)	460	155,65	10,2
Завершение (SHUTDOWN)	597	130,21	8,5
Итого за цикл	5968	1525,4	100,0

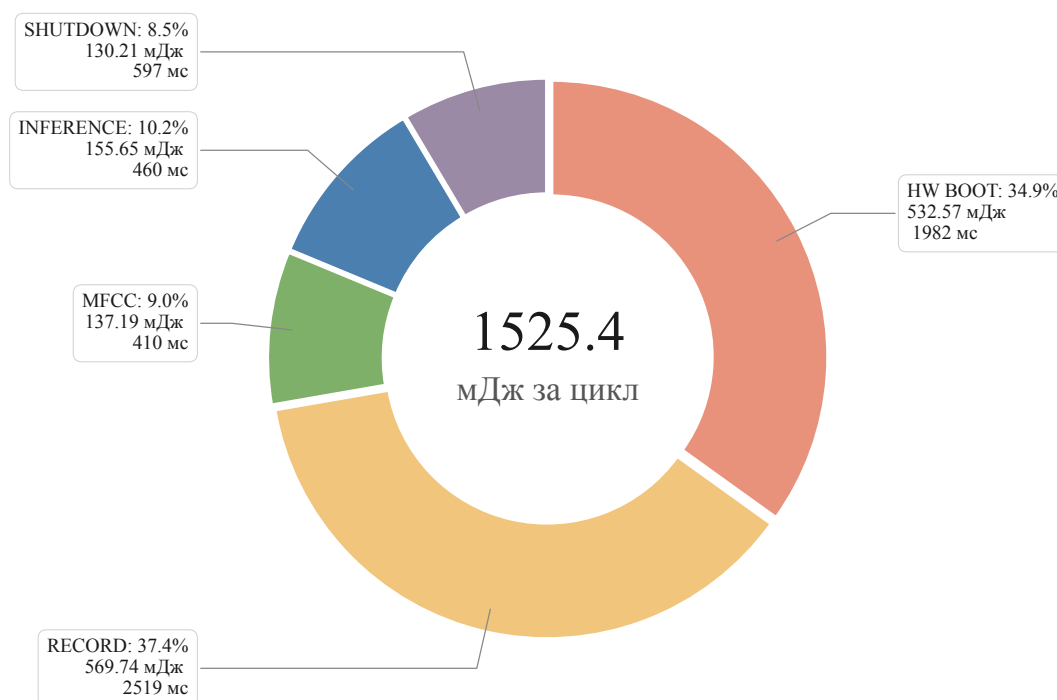


Рисунок 18 — Распределение энергопотребления одного цикла каскадной обработки по этапам конечного автомата для выбранной модели *f104_b5* PTQ. Полная энергия цикла составляет 1525,4 мДж

Анализ декомпозиции приводит к ключевому выводу настоящей работы. Собственно вычислительные этапы — извлечение MFCC-признаков (9,0%) и нейросетевой инференс (10,2%) — в сумме составляют 19,2% общего энергобюджета цикла, тогда как доминирующий вклад вносят *сервисные* этапы: аппаратная инициализация микроконтроллера после пробуждения (34,9%), захват аудио (37,4%) и подготовка к переходу в глубокий сон (8,5%). Данное наблюдение качественно согласуется с независимым выводом Cerutti и др. [7] о доминирующем вкладе предобработки в общий энергобюджет KWS-конвейера, однако дополняется следующей особенностью каскадных архитектур: при переключении микроконтроллера между режимами питания значимыми компонентами энергобюджета становятся также инициализация подсистем после пробуждения и подготовка к переходу в глубокий сон. Эти этапы отсутствуют в традиционных конвейерах непрерывного мониторинга и идентифицируются как самостоятельные значимые компоненты энергобюджета именно в каскадных архитектурах с активным управлением режимами питания.

Тем самым задача 5 введения выполнена. Проведена покомпонентная

декомпозиция энергобюджета каскадного цикла, выявившая доминирующий вклад сервисных этапов и определяющая приоритетные направления дальнейшей оптимизации (см. подраздел 4.9).

4.8 Сравнительная оценка автономности

Согласно формуле (16) раздела 3, средняя потребляемая мощность системы определяется энергией цикла и частотой голосовых событий. Расчётное время автономной работы от аккумулятора ёмкостью 2500 мА·ч по формуле (17) рассчитано для трёх сценариев эксплуатации, отличающихся частотой голосовых событий, и для трёх вариантов аппаратной реализации, отличающихся током покоя в режиме глубокого сна. Результаты приведены на рис. 19.

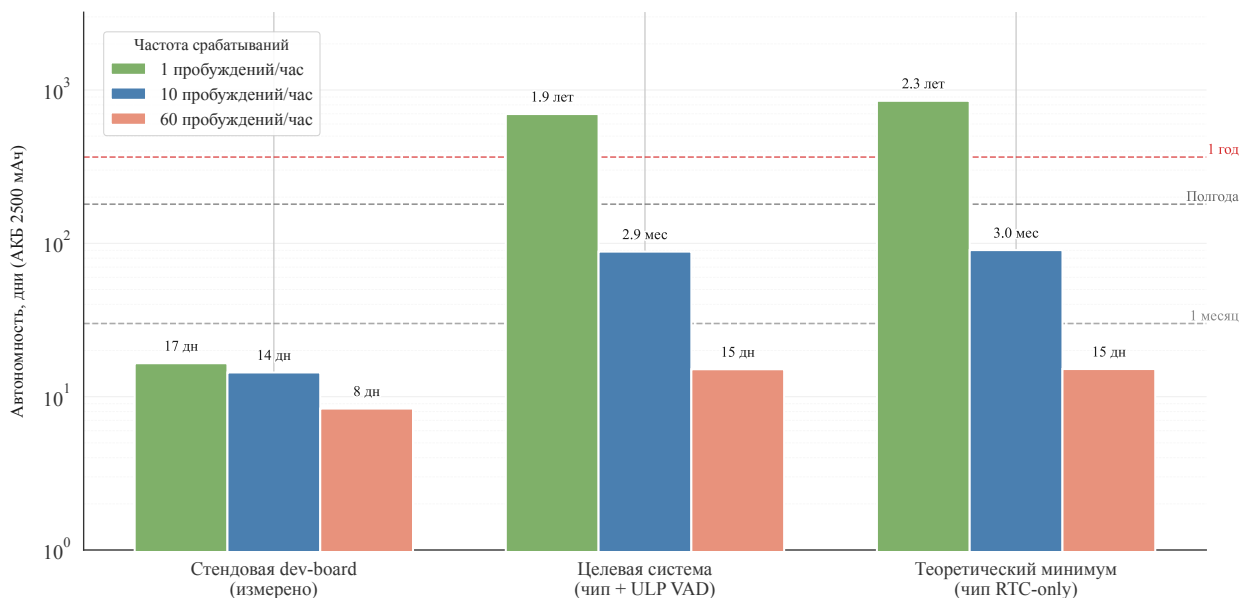


Рисунок 19 — Расчётное время автономной работы системы от аккумулятора ёмкостью 2500 мА·ч в зависимости от частоты голосовых событий для трёх вариантов аппаратной реализации

Три варианта реализации соответствуют различным уровням оптимизации аппаратной части системы при сохранении неизменной каскадной архитектуры программного обеспечения.

Вариант 1 — стендовая отладочная плата (измерено) — текущая реализация на модуле ESP32-S3-Zero с обвязкой отладочной платы, включающей линейный стабилизатор, USB-UART-преобразователь, индикаторные светодиоды и аналоговый модуль детектора звука LM393. Измеренный ток

покоя в режиме глубокого сна составляет 6,3 мА. Расчётная автономность — от 8 до 17 суток в зависимости от частоты срабатываний.

Вариант 2 — целевая система с ULP VAD — промышленная реализация с собственной печатной платой, применением LDO ультранизкого тока покоя и заменой аналогового компаратора на специализированный MEMS-сенсор с интегрированной функцией пробуждения по звуку (wake-on-sound). Расчётный ток покоя — порядка 30 мкА. Автономность увеличивается до 15 суток при 60 пробуждениях в час, до 2,9 месяцев при 10 пробуждениях в час и до 1,9 лет при 1 пробуждении в час.

Вариант 3 — теоретический минимум — кристалл ESP32-S3 в режиме deep sleep с активным RTC GPIO без дополнительной обвязки. Согласно паспортным характеристикам [1], ток покоя составляет около 10 мкА. Автономность — 2,3 года при 1 пробуждении в час.

Сопоставление трёх вариантов показывает, что каскадная архитектура программного обеспечения в сочетании с оптимизацией аппаратной части позволяет достичь автономности порядка лет при типичной частоте срабатываний бытового устройства. Переход от отладочной платы к промышленной реализации обеспечивает увеличение автономности более чем в 40 раз в режиме 1 пробуждение в час и сохраняет двукратное преимущество даже при интенсивных 60 срабатываниях в час. *Тем самым подтверждено достижение цели ВКР* в части снижения энергопотребления за счёт каскадного управления режимами работы микроконтроллера: при ожидаемой частоте голосовых событий типового бытового устройства предложенная архитектура обеспечивает автономную работу длительностью от недель до лет в зависимости от уровня аппаратной оптимизации.

4.9 Дополнительные наблюдения и направления развития работы

4.9.1 Энергопотребление модуля в режиме глубокого сна

Измеренное потребление модуля ESP32-S3-Zero в режиме глубокого сна составило 6,3 мА, что превышает паспортное потребление кристалла ESP32-S3 (7–8 мкА согласно технической документации платформы [1]) примерно в 800 раз. Расхождение объясняется суммой токов покоя компонентов обвязки модуля: линейный стабилизатор напряжения, микросхемы флеш-памяти и PSRAM в дежурном режиме, а также подключённый модуль детек-

тора звука LM393. Данное наблюдение представляет самостоятельное инженерное значение: оно демонстрирует ценность измерений с микроамперным разрешением на лабораторном стенде для характеристики реального энергопотребления готового устройства и подтверждает, что разрыв между паспортным потреблением кристалла и фактическим потреблением серийного модуля не является ограничением каскадной архитектуры как принципа, а определяется обвязкой выбранной отладочной платы.

4.9.2 Проекция на промышленную реализацию

Декомпозиция энергобюджета настоящей работы выполнена на отладочной плате со всеми присущими ей паразитными вкладами. Покомпонентная проекция энергобюджета режима глубокого сна на промышленную реализацию приведена в табл. 7.

Таблица 7 — Покомпонентная проекция потребления в режиме глубокого сна для промышленной реализации

Компонент	Потребление, мкА
ESP32-S3 в режиме глубокого сна с активным RTC IO [1]	~ 10
LDO с ультранизким током покоя (HT7333, MCP1700 и подобные)	~ 4
MEMS-микрофон с интегрированным wake-on-sound (Vesper VM1010 или аналог)	~ 10
Цифровой микрофон INMP441, обесточенный через MOSFET-ключ	0
Подтягивающие резисторы, делители напряжения, паразитные токи	~ 5
Расчётное итоговое потребление в режиме ожидания	~ 30

Расчётное потребление в режиме глубокого сна для промышленной реализации лежит в диапазоне 30–70 мкА в зависимости от выбора конкретных

компонентов, что соответствует расчётной автономности порядка нескольких месяцев при типичной частоте срабатываний (см. рис. 19, средний столбец). Тем самым принципиальная достижимость заявленных во введении показателей энергоэффективности подтверждена покомпонентной декомпозицией, проведённой на основании экспериментальных измерений.

4.9.3 Направления дальнейшего развития

Покомпонентная декомпозиция энергобюджета непосредственно определяет приоритеты дальнейшего развития системы.

Уменьшение длительности этапа инициализации (HW BOOT, 34,9% энергобюджета цикла) — наиболее значимое направление оптимизации. Возможные подходы: перенос части инициализации в RTC SRAM, сохраняющую содержимое при глубоком сне; применение режима лёгкого сна вместо глубокого для уменьшения времени повторной инициализации периферии; вынос инициализации драйверов на сопроцессор ULP, способный работать параллельно с пробуждением основного ядра.

Сокращение окна захвата аудио (RECORD, 37,4%) — применение скользящего кольцевого буфера, заполняемого микрофоном в фоновом режиме при работе сопроцессора ULP, позволит сократить активное окно захвата основным ядром с 2,5 секунд до фактической длительности произнесения команды.

Оптимизация обвязки модуля — проектирование собственной печатной платы с применением LDO с ультранизким током покоя, замена аналогового VAD на специализированный MEMS-сенсор с интегрированной функцией пробуждения по звуку (wake-on-sound). Подобный подход реализован в академических работах по сверхнизкопотребляющим аналоговым VAD [18; 19].

Дополнительная архитектурная оптимизация модели — структурированное разрежение pointwise-свёрток, дистилляция знаний из крупной модели в более компактную, что может дать дополнительное снижение энергии нейросетевого инференса при сохранении точности.

Верификация подхода на других платформах — семейства STM32U5 с механизмом LPBAM, nRF52840 с функцией wake-on-sound и аналогичные платформы с поддержкой режимов глубокого сна и внешнего пробуждения. Каскадная архитектура и методика декомпозиции энергобюджета не привя-

заны к конкретной аппаратной платформе.

Экспериментальное исследование стратегий постобработки — агрегирование решений по скользящему окну, применение корректора ложных отказов [29] для снижения частоты ложных отказов без увеличения энергопотребления модели.

4.10 Выводы по разделу

В настоящем разделе представлены результаты экспериментальной апробации предложенной каскадной KWS-системы на платформе ESP32-S3 и их интерпретация в терминах задач, поставленных во введении. Получены следующие основные результаты.

- 1) Систематическое исследование 134 архитектур DS-CNN в пространстве гиперпараметров ($\text{filters} \times \text{blocks}$) выявило плато точности при $F \geq 80$, $B \geq 4$ в пределах статистической погрешности оценки. Построенная Парето-граница в координатах «точность $\times$ размер модели» (рис. 8) показала, что эталонная конфигурация Hello Edge DS-CNN-M не является Парето-оптимальной для исследованной задачи на платформе ESP32-S3.
- 2) Сравнение методов квантизации PTQ и QAT, проведённое на двух уровнях обобщения (детально на 4 архитектурах и агрегированно на 63 парах), показало, что для разрядности INT8 98% парных различий между методами укладываются в коридор статистической погрешности $\pm 0,65$ п.п. при медиане $+0,048$ п.п. Метод PTQ является предпочтительным как обеспечивающий сопоставимую точность при существенно меньшей вычислительной стоимости. *Задача 3 введения выполнена.*
- 3) Послойная декомпозиция времени инференса на ESP32-S3 выявила доминирующий вклад pointwise-свёрток (50–86% времени инференса в зависимости от размера модели) с сверхлинейной зависимостью от числа фильтров, что определяет приоритеты архитектурной оптимизации.
- 4) По Парето-критерию «точность $\times$ энергия инференса» выбрана финальная модель f104_b5 PTQ (96,12% точности, 101 КБ размер, 149,6 мДж/инференс), обеспечивающая компромисс между точно-

стью, размером и энергопотреблением и превосходящая эталонную Hello Edge конфигурацию по совокупности критериев.

- 5) Качество классификации выбранной модели охарактеризовано по совокупности метрик precision, recall, F1-score по каждому классу (табл. 4); macro avg F1 = 0,9432, weighted avg F1 = 0,9580. Структура ошибок концентрируется на парах акустически близких слов и сохраняется при квантизации, что подтверждает корректность процедуры квантизации.
- 6) Выполнена покомпонентная декомпозиция энергобюджета каскадного цикла. Установлено, что собственно вычислительные этапы составляют 19,2% энергобюджета, тогда как доминирующими являются сервисные этапы (инициализация 34,9%, захват аудио 37,4%, завершение 8,5%). Идентификация инициализации и завершения как самостоятельных значимых компонентов энергобюджета, не учитываемых в моделях непрерывного мониторинга, составляет содержание главного исследовательского результата работы. *Задача 5 введения выполнена.*
- 7) Сравнительная оценка показала, что предложенная каскадная архитектура программного обеспечения в сочетании с переходом от отладочной платы к промышленной аппаратной реализации обеспечивает увеличение расчётного времени автономной работы от 8–17 суток до 2,9 месяцев — 1,9 года в зависимости от частоты голосовых событий, что подтверждает достижение цели работы в части снижения энергопотребления за счёт каскадного управления режимами работы микроконтроллера.
- 8) Определены приоритетные направления дальнейшего развития системы: уменьшение длительности этапа инициализации, использование скользящего кольцевого буфера на сопроцессоре ULP для сокращения окна захвата аудио, оптимизация обвязки модуля с применением LDO ультранизкого тока покоя, замена аналогового VAD на специализированный MEMS-сенсор, верификация подхода на других микроконтроллерных платформах.

ЗАКЛЮЧЕНИЕ

В настоящей работе решена задача снижения энергопотребления устройств интернета вещей с функцией голосового управления за счёт каскадного управления режимами работы микроконтроллера и применения квантизованных нейронных сетей для распознавания ключевых слов. Цель работы достигнута; все поставленные во введении задачи выполнены.

В соответствии с задачей 1 проведён системный анализ предметной области, включающий рассмотрение облачных голосовых решений, специализированных аппаратных платформ (ASIC, ПЛИС, DSP) и микроконтроллеров общего назначения. Систематизация подходов к снижению энергопотребления систем распознавания ключевых слов показала, что в литературе три ключевых направления — компактные нейросетевые архитектуры, методы квантизации моделей и каскадные схемы детекции — рассматриваются преимущественно изолированно, без отдельной количественной оценки их совместного вклада в итоговый энергобюджет. На основании анализа сформулированы количественные и качественные требования к разрабатываемой системе по совокупности метрик: средней потребляемой мощности, латентности, точности классификации, размеру модели, воспроизводимости и наблюдаемости.

В соответствии с задачей 2 разработана каскадная архитектура детекции, состоящая из трёх ступеней возрастающей вычислительной сложности: аналоговой детекции звукового события на компараторе, захвата аудио с извлечением MFCC-признаков на ЦПУ микроконтроллера в активном режиме и нейросетевого инференса квантизованной модели DS-CNN с аппаратным ускорением. Выведена аналитическая модель средней потребляемой мощности каскадной системы, связывающая энергобюджет с мгновенными потреблением ступеней и долями их активности; из модели получены два проектных критерия — кратное разделение ступеней по мгновенному потреблению и по частоте активации, следование которым является необходимым условием выигрыша каскадной архитектуры перед режимом непрерывного инференса. Архитектура не привязана к конкретной аппаратной платформе и переносима на любой микроконтроллер с поддержкой аппаратных режимов глубокого сна и внешнего пробуждения.

В соответствии с задачей 3 проведено систематическое исследование

пространства архитектур DS-CNN: обучено 134 архитектуры в сетке гиперпараметров $F \in \{16, \dots, 224\}$ фильтров и $B \in \{2, \dots, 8\}$ блоков разделимой свёртки; для каждой выполнена квантизация методом PTQ, для 105 архитектур дополнительно методом QAT — итого 239 оцениваемых точек. Сравнение методов квантизации, выполненное на двух уровнях обобщения (детальное сравнение на четырёх представительных архитектурах и агрегированное сравнение по 63 парам PTQ/QAT), показало, что 98% парных различий точности $\Delta = \text{acc}_{\text{QAT}} - \text{acc}_{\text{PTQ}}$ укладываются в коридор статистической погрешности $\pm 0,65$ п.п. при медиане $+0,048$ п.п. Это позволяет считать выбор между методами PTQ и QAT не критическим для исследованной задачи квантизации до INT8; PTQ предпочтителен по соображениям вычислительной стоимости.

В соответствии с задачей 4 разработан двухдоменный лабораторный измерительный стенд с электрической изоляцией измеряемой системы и системы измерений, обеспечивающий регистрацию мгновенного тока с разрешением единиц микроампер при помощи 20-битного монитора мощности INA228. Реализация системы выполнена на платформе ESP32-S3 на базе фреймворка ESP-IDF; конечный автомат каскада из семи состояний синхронизирован с логгером измерительного стенда через GPIO, что обеспечило однозначное сопоставление участков профиля потребления с этапами цикла обработки. Экспериментальная апробация подтвердила работоспособность всех ступеней каскада в их взаимодействии.

По результатам исследования архитектур и сравнения методов квантизации в качестве финальной модели для развёртывания выбрана архитектура DS-CNN с $F = 104$ фильтрами и $B = 5$ блоками, квантизованная методом PTQ. Выбранная модель обеспечивает точность классификации 96,12% на тестовой выборке Google Speech Commands при размере 101 КБ и энергии одного инференса 149,6 мДж на платформе ESP32-S3. По совокупности критериев (точность, размер, энергия инференса) выбранная модель превосходит эталонную конфигурацию Hello Edge DS-CNN-M [2] ($F = 172$, $B = 6$, размер ≈ 272 КБ, энергия инференса в 3 раза выше) при сопоставимой точности (различие в пределах статистической погрешности).

В соответствии с задачей 5 выполнена покомпонентная декомпозиция энергобюджета цикла каскадной обработки. Установлено, что энергия полного цикла составляет 1525,4 мДж; распределение по этапам имеет следую-

щий вид: инициализация микроконтроллера — 34,9%, захват аудио — 37,4%, извлечение признаков MFCC — 9,0%, нейросетевой инференс — 10,2%, завершение и подготовка к глубокому сну — 8,5%. Главный исследовательский результат декомпозиции — собственно вычислительные этапы (MFCC и инференс) составляют лишь 19,2% общего энергобюджета цикла, тогда как доминирующими являются *сервисные* этапы — инициализация, захват аудио и завершение, в сумме составляющие 80,8% энергобюджета. Идентификация инициализации и завершения как самостоятельных значимых компонентов энергобюджета, не учитываемых в моделях непрерывного мониторинга и становящихся видимыми именно в каскадных архитектурах с активным управлением режимами питания, составляет содержание новизны работы. Сравнительная оценка автономности показала, что предложенная каскадная архитектура программного обеспечения в сочетании с переходом от отладочной платы к промышленной аппаратной реализации обеспечивает увеличение расчётного времени автономной работы от единиц суток до месяцев и лет в зависимости от частоты голосовых событий.

Научная новизна полученных результатов состоит в следующем. Во-первых, впервые на единой платформе общего назначения получено количественное описание совместного применения каскадной архитектуры, квантизации нейросетевой модели и систематического поиска оптимальной архитектуры с отдельной оценкой вклада каждого метода в итоговое снижение энергопотребления. Во-вторых, проведено систематическое исследование пространства архитектур DS-CNN (134 архитектуры, 239 точек квантизации) с оценкой статистической значимости различий точности по доверительному интервалу Уилсона, что позволило строго охарактеризовать поведение методов квантизации PTQ и QAT для целевой задачи. В-третьих, компонентная декомпозиция энергобюджета цикла обработки, выполненная на платформе ESP32-S3, позволила идентифицировать этапы с доминирующим вкладом в общий бюджет и определить приоритетные направления дальнейшей оптимизации; в открытой литературе декомпозиция подобного уровня детализации, выделяющая сервисные этапы каскадного цикла как самостоятельные значимые компоненты энергобюджета, ранее не приводилась.

В ходе апробации получено самостоятельное инженерное наблюдение

ние — разрыв между паспортным потреблением кристалла ESP32-S3 в режиме глубокого сна (7–8 мкА) и фактическим потреблением серийного модуля ESP32-S3-Zero (6,3 мА), составляющий порядка 800 раз. Разрыв обусловлен вкладом обвязки отладочной платы и не является ограничением каскадной архитектуры как принципа. Покомпонентная проекция энергобюджета на промышленную реализацию — с заменой компаратора LM393 на специализированный MEMS-микрофон с функцией пробуждения по звуку (wake-on-sound), применением LDO с ультранизким током покоя и обесточиванием цифрового микрофона через MOSFET-ключ — показывает достижимость потребления в режиме ожидания на уровне 30–70 мкА, что обеспечивает расчётное время автономной работы порядка от нескольких месяцев до двух лет в зависимости от частоты голосовых событий и подтверждает принципиальную способность предложенной каскадной архитектуры удовлетворять требованиям к энергоэффективным IoT-устройствам.

Практическая значимость работы состоит в том, что предложенный каскадный метод, методика систематического исследования пространства архитектур с применением статистической оценки значимости различий, методика декомпозиции энергобюджета и методика измерения энергопотребления с микроамперным разрешением на двухдоменном стенде применимы при проектировании автономных IoT-устройств с голосовым управлением в задачах промышленного мониторинга, носимой электроники и беспроводных узлов умного дома. Метод не привязан к конкретной аппаратной платформе и переносим на микроконтроллеры семейств STM32, nRF52 и аналогичные. Результаты сравнительного исследования методов квантизации служат ориентиром при выборе стратегии оптимизации моделей для периферийных устройств. Методика двухдоменного измерительного стенда с электрической изоляцией применима для характеристики энергопотребления произвольных встраиваемых платформ.

Направлениями дальнейшего развития работы являются: проектирование собственной печатной платы с применением LDO ультранизкого тока покоя; замена аналогового компаратора первой ступени на специализированный MEMS-микрофон с пробуждением по звуку; применение скользящего кольцевого буфера на сопроцессоре ULP для сокращения окна активного захвата аудио; уменьшение длительности этапа инициализации средства

ми переноса части инициализации в RTC SRAM либо использования режима лёгкого сна; верификация каскадного метода на платформах STM32U5 и nRF52840 для подтверждения переносимости подхода; экспериментальное исследование стратегий постобработки выходов классификатора (агрегирование решений по скользящему окну, применение корректора ложных отказов) для снижения частоты ложных отказов без увеличения энергопотребления модели.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Espressif Systems. ESP32-S3 Datasheet. Version 1.7. — 2024. — [Электронный ресурс]. URL: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf (дата обращения: 01.04.2026).
2. Hello Edge: Keyword Spotting on Microcontrollers / Y. Zhang, N. Suda, L. Lai, V. Chandra // arXiv preprint arXiv:1711.07128. — 2018. — С. 12.
3. Temporal Convolution for Real-time Keyword Spotting on Mobile Devices / S. Choi, S. Seo, B. Shin [и др.] // Proc. Interspeech. — 2019. — С. 3372—3376.
4. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference / B. Jacob, S. Kligys, B. Chen [и др.] // Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2018. — С. 2704—2713.
5. Google LLC. TensorFlow Model Optimization Toolkit — Quantization Aware Training. — 2024. — [Электронный ресурс]. URL: https://www.tensorflow.org/model_optimization/guide/quantization/training (дата обращения: 01.04.2026).
6. E. Hardy, F. Badets. An Ultra-low Power RNN Classifier for Always-On Voice Wake-Up Detection Robust to Real-World Scenarios // Proc. tinyML Research Symposium. — 2021. — С. 7.
7. Sub-mW Keyword Spotting on an MCU: Analog Binary Feature Extraction and Binary Neural Networks / G. Cerutti, L. Cavigelli, R. Andri [и др.] // IEEE Transactions on Circuits and Systems I: Regular Papers. — 2022. — Т. 69, № 5. — С. 2002—2012.
8. Комбинированный детектор голосовой активности / А. Ф. Стефаниди, А. Л. Приоров, А. И. Топников, В. В. Хрящев // Цифровая обработка сигналов. — 2023. — № 1. — С. 62—67.
9. Алгоритм выявления синтезированного голоса на основе кепстральных коэффициентов и сверточной нейронной сети / Р. А. Муртазин, А. Ю. Кузнецов, Е. А. Фёдоров [и др.] // Научно-технический вестник информационных технологий, механики и оптики. — 2021. — Т. 21, № 4. — С. 545—552.
10. А. О. Исхакова, Д. А. Вольф, Р. В. Мещеряков. Способ снижения размерности пространства признаков при распознавании речевых эмоций с использованием СНС // Автоматика и телемеханика. — 2022. — № 11. — С. 147—161.
11. Review of algorithmic solutions for deployment of neural networks on lite devices / S. A. Khan, S. Shulepina, D. Shulepin, R. A. Lukmanov // Компьютерные исследования и моделирование. — 2024. — Т. 16, № 7. — С. 1601—1619.

12. J. Bushur, C. Chen. Neural Network Exploration for Keyword Spotting on Edge Devices // *Future Internet*. — 2023. — Т. 15, № 6. — С. 219.
13. P. Warden. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition // *arXiv preprint arXiv:1804.03209*. — 2018. — С. 11.
14. MLPerf Tiny Benchmark / C. Banbury, V. J. Reddi, P. Torelli [и др.] // *Proc. Neural Information Processing Systems Datasets and Benchmarks Track*. — 2021. — С. 15.
15. S. Davis, P. Mermelstein. Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences // *IEEE Transactions on Acoustics, Speech, and Signal Processing*. — 1980. — Т. 28, № 4. — С. 357—366.
16. Open-source FPGA-ML Codesign for the MLPerf Tiny Benchmark / J. Duarte, M. Trahms, H. Borrás [и др.] // *Proc. Machine Learning and Systems (MLSys)*. — 2022. — С. 6.
17. И. В. Ушенина, Е. А. Данилов. Реализация на ПЛИС модуля искусственного нейрона для последовательно-параллельных архитектур нейронных сетей с прямой связью // *Цифровая обработка сигналов*. — 2025. — № 1. — С. 78—84.
18. A 108-nW 0.8-mm² Analog Voice Activity Detector Featuring a Time-Domain CNN With Sparsity-Aware Computation and Sparsified Quantization in 28-nm CMOS / F. Chen, K.-F. Un, W.-H. Yu [и др.] // *IEEE Journal of Solid-State Circuits*. — 2022. — Т. 57, № 11. — С. 3288—3297.
19. A 47-nW Voice Activity Detector (VAD) Featuring a Short-Time CNN Feature Extractor and an RNN-Based Classifier With a Non-Volatile CAP-ROM / J. Lin, K.-F. Un, W.-H. Yu [и др.] // *IEEE Journal of Solid-State Circuits*. — 2023. — Т. 58, № 11. — С. 3020—3029.
20. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications / A. G. Howard, M. Zhu, B. Chen [и др.] // *arXiv preprint arXiv:1704.04861*. — 2017. — С. 9.
21. А. В. Трусов, Д. Н. Путинцев, Е. Е. Лимонова. Uncertainty-based quantization method for stable training of binary neural networks // *Компьютерная оптика*. — 2024. — Т. 48, № 4. — С. 573—581.
22. TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems / R. David, J. Duke, A. Jain [и др.] // *Proceedings of Machine Learning and Systems*. Т. 3. — 2021. — С. 800—811.

23. L. Lai, N. Suda, V. Chandra. CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs // arXiv preprint arXiv:1801.06601. — 2018. — С. 6.
24. Espressif Systems. ESP-IDF Programming Guide. Version 5.4. — 2026. — [Электронный ресурс]. URL: <https://docs.espressif.com/projects/esp-idf/en/v5.4/esp32s3/> (дата обращения: 01.04.2026).
25. Espressif Systems. ESP32-S3 Technical Reference Manual. Version 1.2. — 2024. — [Электронный ресурс]. URL: https://www.espressif.com/documentation/esp32-s3_technical_reference_manual_en.pdf (дата обращения: 02.04.2026).
26. InvenSense (TDK). INMP441 Omnidirectional Microphone with Bottom Port and I2S Digital Output. Datasheet. — 2023. — [Электронный ресурс]. URL: <https://invensense.tdk.com/products/analog/inmp441/> (дата обращения: 01.04.2026).
27. Texas Instruments. INA228 85-V, 20-Bit, Ultra-Precise Power/Energy/Charge Monitor. Datasheet. — 2022. — [Электронный ресурс]. URL: <https://www.ti.com/lit/ds/symlink/ina228.pdf> (дата обращения: 02.04.2026).
28. P. M. Sørensen, B. Epp, T. May. A Depthwise Separable Convolutional Neural Network for Keyword Spotting on an Embedded System // EURASIP Journal on Audio, Speech, and Music Processing. — 2020. — Т. 2020, № 1. — С. 10.
29. А. М. Голубков, Е. В. Шуранов. Применение FN-корректора с целью повышения качества классификации аудиособытий // Научно-технический вестник информационных технологий, механики и оптики. — 2022. — Т. 22, № 4. — С. 708—715.

Приложение А

Принципиальная электрическая схема лабораторного стенда

Полная принципиальная электрическая схема двухдоменного измерительного стенда, разработанная в среде KiCad, приведена на рис. 20. Схема отражает разделение измеряемой системы (домен 1) и системы измерений (домен 2) с включением монитора мощности INA228 в разрыв линии питания домена 1, описанное в подразделе 3.5.

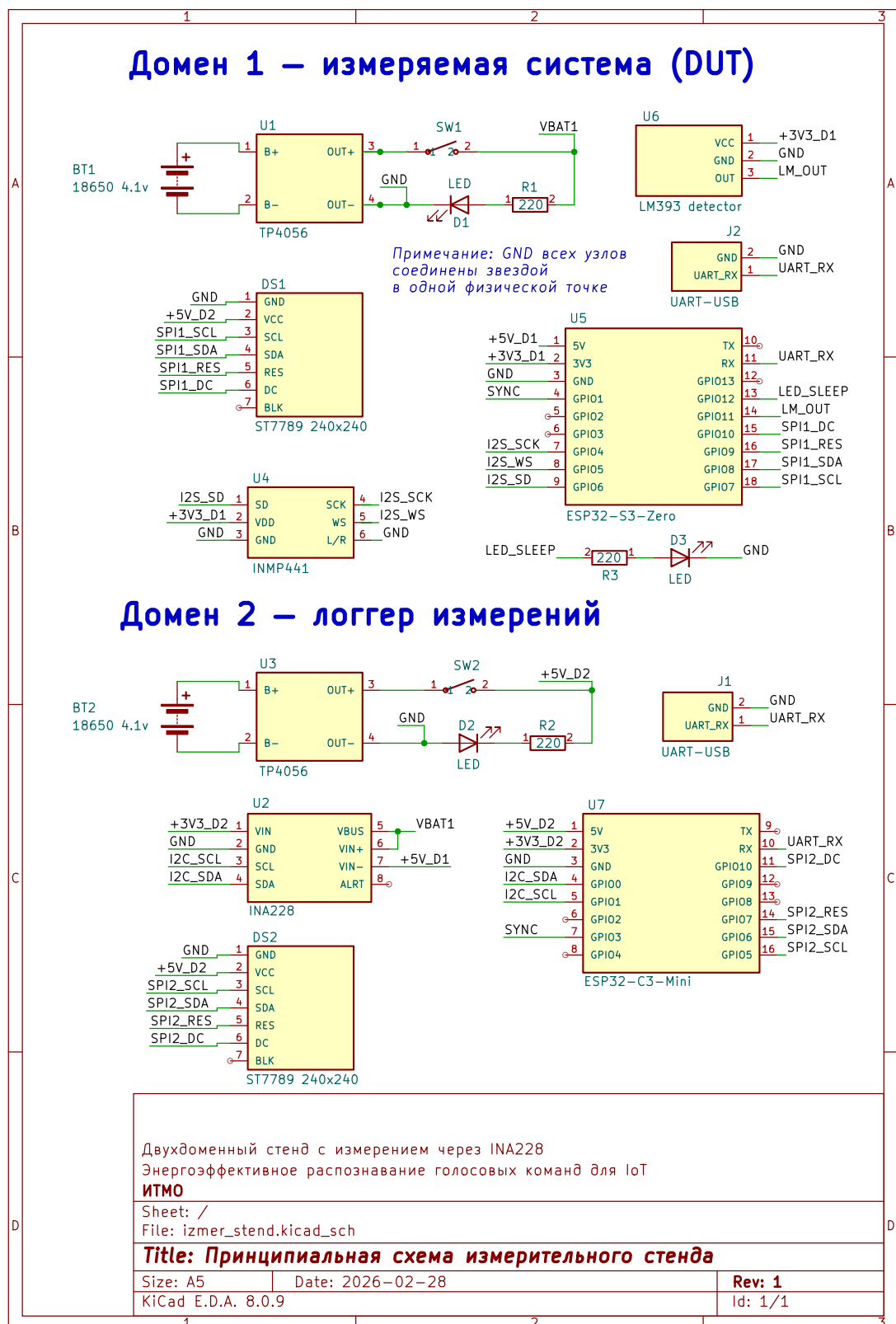


Рисунок 20 — Принципиальная электрическая схема двухдоменного измерительного стенда

Приложение Б

Диаграмма состояний конечного автомата

Диаграмма состояний конечного автомата каскадной обработки, описанного в подразделе 3.1, приведена на рис. 21. Состояния автомата соответствуют строкам таблицы 2; переходы между состояниями инициируются аппаратными событиями (пробуждение по EXT1, завершение DMA-передачи) или программными решениями (положительная классификация, истечение временных интервалов).

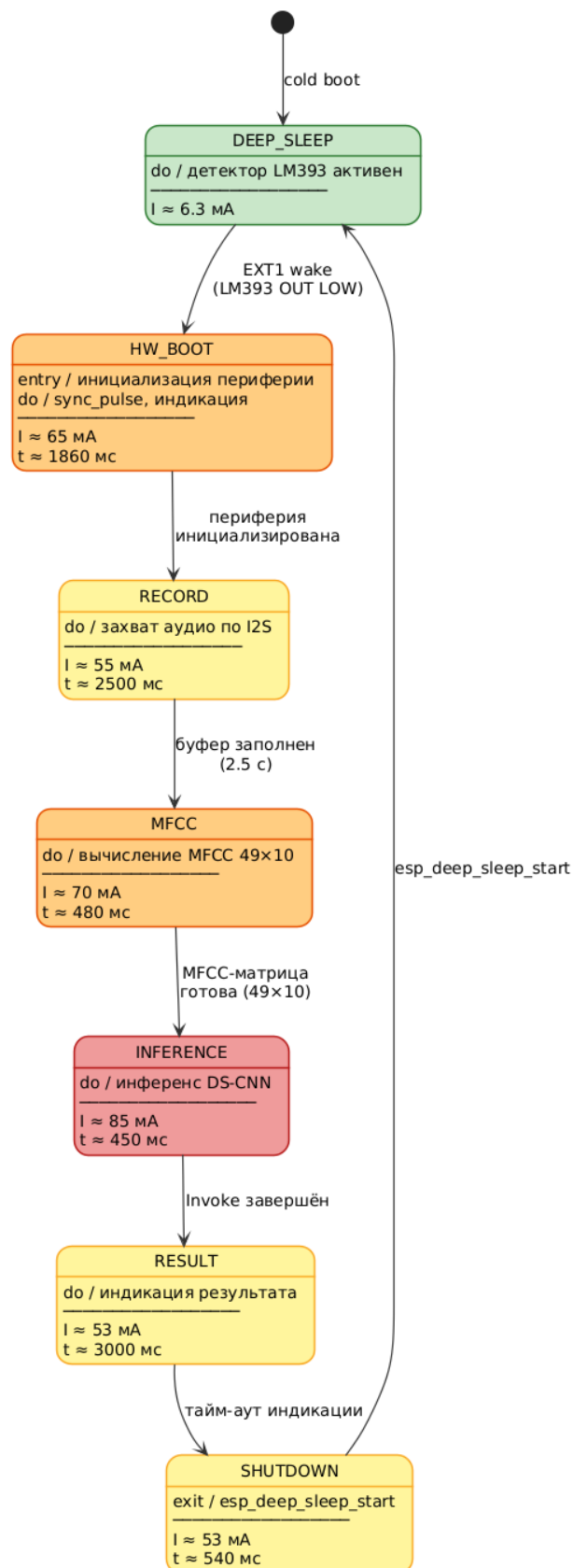


Рисунок 21 — Диаграмма состояний конечного автомата каскадной обработки

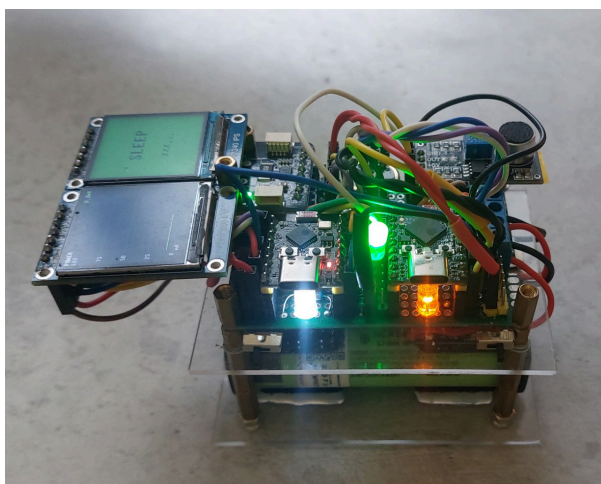
Приложение В

Фотографии лабораторного стенда в различных режимах работы

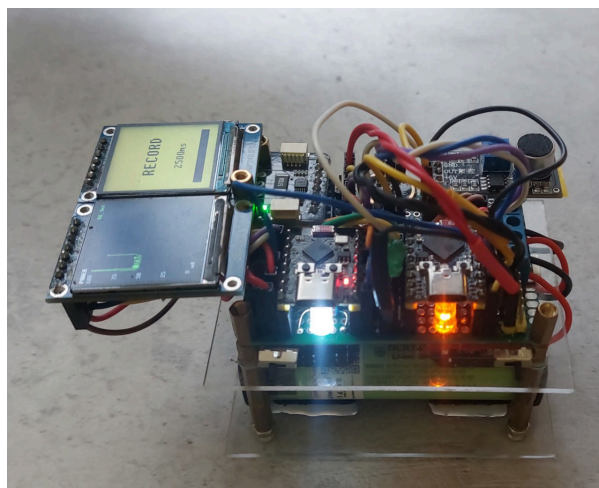
Общий вид собранного лабораторного стенда в различных фазах функционирования приведён на рис. 22. Каждое изображение соответствует определённому состоянию конечного автомата каскадной системы обработки, описанного в подразделе 3.1, и демонстрирует визуальную индикацию текущего режима на периферийных устройствах стенда.

На представленной сетке снимков отображены следующие режимы:

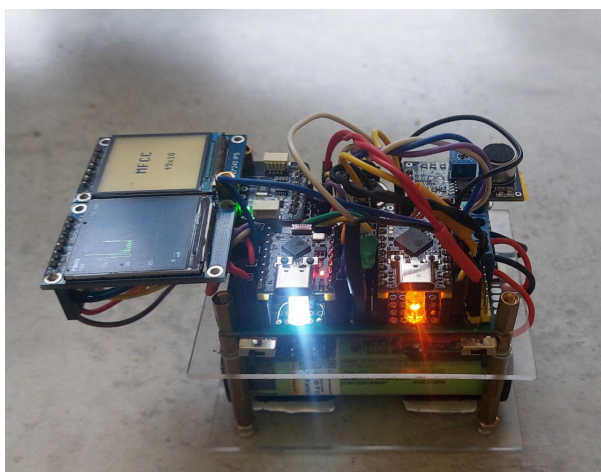
- режим ожидания (рис. 22a) — состояние DEEP_SLEEP, минимальное энергопотребление до срабатывания аппаратного компаратора;
- режим записи (рис. 22b) — состояние RECORD, процесс захвата аудиопотока с MEMS-микрофона INMP441 в буфер DMA;
- извлечение признаков (рис. 22c) — состояние MFCC, расчёт матриц спектральных коэффициентов с использованием инструкций ESP-DSP;
- нейросетевой инференс (рис. 22d) — состояние INFERENCE, выполнение квантованной модели DS-CNN интерпретатором TFLite Micro;
- индикация отсутствия полезного сигнала (рис. 22e) — состояние RESULT, алгоритм классифицировал входной фрейм как фоновый шум или тишину;
- индикация распознанной команды (рис. 22f) — состояние RESULT, фиксация положительного исхода классификации и вывод имени команды на дисплей.



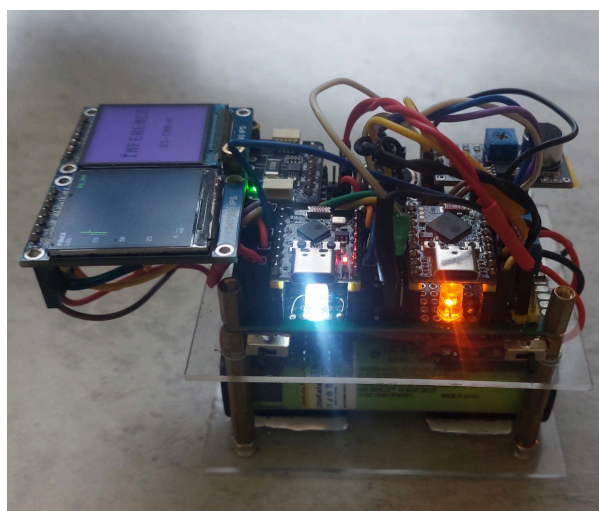
(a) Режим ожидания



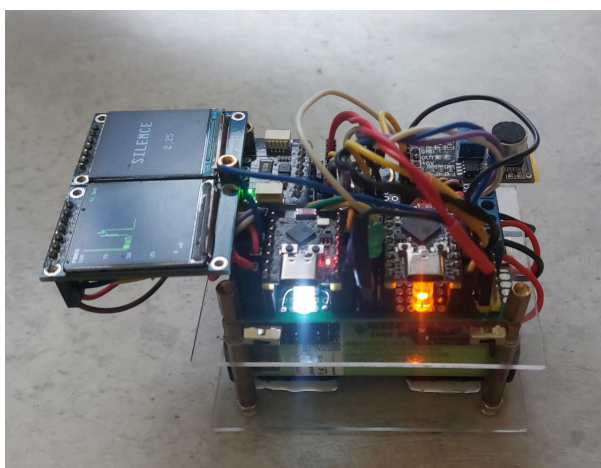
(b) Режим записи (Record)



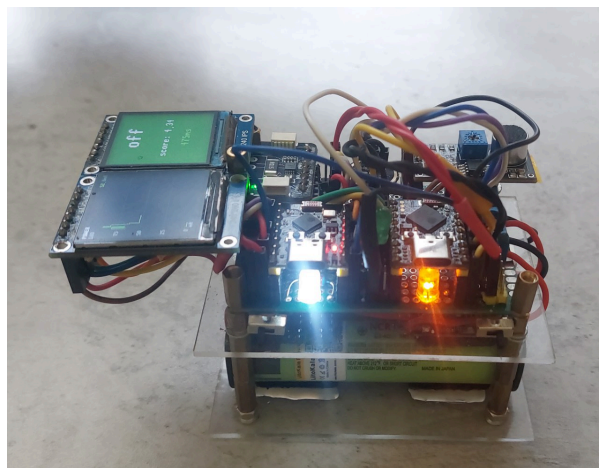
(c) Извлечение признаков



(d) Инференс модели



(e) Результат: «Тишина»



(f) Результат: команда «off»

Рисунок 22 — Лабораторный измерительный стенд в различных фазах работы конвейера обработки

Приложение Г

Исходный код и репозитории проекта

Полные исходные тексты программного обеспечения, разработанного в рамках настоящей работы, размещены в публичных репозиториях автора. Объём кода составляет несколько тысяч строк и не приводится в тексте работы полностью; ключевые фрагменты вынесены в приложения Д и Е, полная версия доступна по ссылкам ниже.

- **ПО микроконтроллера ESP32-S3** (язык C, фреймворк ESP-IDF 5.4), реализующая каскадный цикл обработки, тракт MFCC-предобработки и нейросетевой инференс через TensorFlow Lite Micro:
github.com/worhant/edge-ai-voice-recognition
- **ПО логгера измерительного стенда** (ESP32-C3, опрос INA228 по I2C, синхронизация с FSM через GPIO, трансляция CSV-потока по USB-UART):
github.com/worhant/precise-power-logger
- **Текст и исходники настоящей ВКР** (LaTeX, экспериментальные данные, скрипты построения графиков, обученные модели):
github.com/worhant/itmo-bachelor-thesis

Приложение Д

Ключевые фрагменты программного обеспечения микроконтроллера

В настоящем приложении приведены два ключевых фрагмента программного обеспечения, иллюстрирующих критические аспекты реализации: корректную последовательность операций перехода в режим глубокого сна (подраздел Д.1) и инициализацию интерпретатора TensorFlow Lite Micro с размещением арены тензоров во внутреннем SRAM (подраздел Д.2). Полный исходный код программы доступен в репозитории, указанном в приложении Г.

Д.1. Переход в режим глубокого сна (vad_sleep.c)

Порядок операций критичен: деинициализация периферии и изоляция GPIO предшествуют настройке источника пробуждения. Нарушение порядка приводит к ложному срабатыванию EXT1 из-за переходных процессов на выводах RTC GPIO в момент перехода в режим сна.

```
void vad_sleep_enter(bool enable_led) {
    if(enable_led) {
        rtc_gpio_init(SLEEP_LED_GPIO);
        rtc_gpio_set_direction(SLEEP_LED_GPIO, RTC_GPIO_MODE_OUTPUT_ONLY);
        rtc_gpio_set_level(SLEEP_LED_GPIO, 1);
        rtc_gpio_hold_en(SLEEP_LED_GPIO);
    }

    rtc_gpio_init(SOUND_GPIO);
    rtc_gpio_set_direction(SOUND_GPIO, RTC_GPIO_MODE_INPUT_ONLY);
    rtc_gpio_pullup_en(SOUND_GPIO);
    rtc_gpio_pulldown_dis(SOUND_GPIO);

    ESP_ERROR_CHECK(esp_sleep_enable_ext1_wakeup(
        1ULL << SOUND_GPIO, ESP_EXT1_WAKEUP_ANY_LOW));

    esp_deep_sleep_start();
}
```

Д.2. Инициализация инференса (kws.cpp)

Арена тензоров размером 140 КБ обязательно размещается во внутреннем SRAM микроконтроллера (флаг `MALLOC_CAP_INTERNAL`), поскольку аппаратное SIMD-ускорение библиотеки ESP-NN корректно работает только с этим типом памяти.

```
static constexpr int kArenaSize = 140 * 1024;
static uint8_t *tensor_arena = nullptr;

esp_err_t kws_init(void) {
    model = tflite::GetModel(g_model_data);
    if (model->version() != TFLITE_SCHEMA_VERSION) return ESP_FAIL;

    static tflite::MicroMutableOpResolver<8> resolver;
    resolver.AddConv2D();
    resolver.AddDepthwiseConv2D();
    resolver.AddFullyConnected();
    resolver.AddMean();
    resolver.AddReshape();
    resolver.AddQuantize();
    resolver.AddDequantize();
    resolver.AddSoftmax();

    tensor_arena = (uint8_t *)heap_caps_malloc(
        kArenaSize, MALLOC_CAP_INTERNAL | MALLOC_CAP_8BIT);
    if (!tensor_arena) return ESP_ERR_NO_MEM;

    static tflite::MicroInterpreter static_interp(
        model, resolver, tensor_arena, kArenaSize);
    interpreter = &static_interp;

    if (interpreter->AllocateTensors() != kTfLiteOk) return ESP_FAIL;

    input_tensor = interpreter->input(0);
```

```
output_tensor = interpreter->output(0);  
return ESP_OK;  
}
```

Приложение Е

Ключевые фрагменты модулей обучения

В настоящем приложении приведены два ключевых фрагмента модулей обучения и квантизации, иллюстрирующих архитектуру нейросетевой модели (подраздел Е.1) и слияние слоёв BatchNorm в предшествующие свёрточные слои перед применением QAT (подраздел Е.2). Полный исходный код доступен в репозитории, указанном в приложении Г.

Е.1. Архитектура DS-CNN (ds_cnn.py)

Параметризованная реализация архитектуры, используемая для обучения 134 различных конфигураций сетки (F, B) из подраздела 3.4.

```
def _ds_block(x, filters, kernel, name):
    """Блок разделимой свёртки: DW -> BN -> ReLU -> PW -> BN -> ReLU."""
    reg = _regularizer()
    x = layers.DepthwiseConv2D(
        kernel_size=kernel, padding="same", strides=(1, 1),
        depthwise_regularizer=reg, use_bias=False, name=f"{name}_dw"
    )(x)
    x = layers.BatchNormalization(name=f"{name}_dw_bn")(x)
    x = layers.ReLU(name=f"{name}_dw_relu")(x)
    x = layers.Conv2D(
        filters=filters, kernel_size=(1, 1), padding="same",
        strides=(1, 1), kernel_regularizer=reg, use_bias=False,
        name=f"{name}_pw"
    )(x)
    x = layers.BatchNormalization(name=f"{name}_pw_bn")(x)
    x = layers.ReLU(name=f"{name}_pw_relu")(x)
    return x

def build_ds_cnn(input_shape, num_classes, cfg):
    reg = _regularizer()
```



```

inputs = tf.keras.Input(shape=input_shape, name="mfcc_input")

x = layers.Conv2D(
    filters=cfg["first_conv_filters"],
    kernel_size=cfg["first_conv_kernel"],
    strides=cfg["first_conv_stride"],
    padding="same", kernel_regularizer=reg,
    use_bias=False, name="stem_conv"
)(inputs)
x = layers.BatchNormalization(name="stem_bn")(x)
x = layers.ReLU(name="stem_relu")(x)

for i in range(cfg["num_ds_blocks"]):
    x = _ds_block(x, cfg["ds_filters"], cfg["ds_kernel"],
                  name=f"ds_{i+1}")

x = layers.GlobalAveragePooling2D(name="gap")(x)
x = layers.Dropout(0.2, name="dropout")(x)
outputs = layers.Dense(num_classes, activation=None,
                        kernel_regularizer=reg, name="logits")(x)

return tf.keras.Model(inputs, outputs, name="ds_cnn")

```

Е.2. Слияние BatchNorm перед QAT (quantize_qat.py)

Слияние BN в предшествующий свёрточный слой выполняется перед вставкой узлов симуляции квантования и обеспечивает численную устойчивость дообучения, как описано в подразделе 2.5.

```

def fold_bn(conv_layer, bn_layer):
    """Слияние BN в предшествующий Conv/DepthwiseConv.

    new_weight = weight * gamma / sqrt(var + eps)
    new_bias   = beta - gamma * mean / sqrt(var + eps)
    """
    gamma, beta, moving_mean, moving_var = bn_layer.get_weights()

```

```

eps = bn_layer.epsilon
scale = gamma / np.sqrt(moving_var + eps)

kernel = conv_layer.get_weights()[0]
if isinstance(conv_layer, tf.keras.layers.DepthwiseConv2D):
    new_kernel = kernel * scale[np.newaxis, np.newaxis, :, np.newaxis]
else:
    new_kernel = kernel * scale[np.newaxis, np.newaxis, np.newaxis, :]

new_bias = beta - moving_mean * scale
return new_kernel.astype(np.float32), new_bias.astype(np.float32)

```

Приложение Ж

Таблица разделов флеш-памяти микроконтроллера

Конфигурационный файл `partitions.csv` задаёт разметку флеш-памяти ESP32-S3 (общий объём 8 МБ), используемую загрузчиком ESP-IDF. Раздел `nvs` (24 КБ) предназначен для хранения системных параметров; раздел `factory` (2000 КБ) — для программы приложения вместе с квантизованной моделью, встраиваемой в бинарный образ.

```
# Name,   Type, SubType, Offset,  Size
# -----
# 0x00000 - bootloader
# 0x08000 - partition table
# -----
nvs,     data, nvs,    0x9000,  0x6000
factory, app,  factory, 0x10000, 2000K
```

Приложение И

Диаграмма последовательности сбора измерительных данных

Диаграмма последовательности сбора измерительных данных, отражающая временные интервалы и порядок взаимодействия компонентов измерительного стенда, приведена на рис. 23. Схема подробно иллюстрирует совместную работу целевого микроконтроллера (ESP32-S3), контроллера логгера (ESP32-C3) и цифрового монитора питания INA228 при формировании пакетов данных для последующей передачи на ПК.

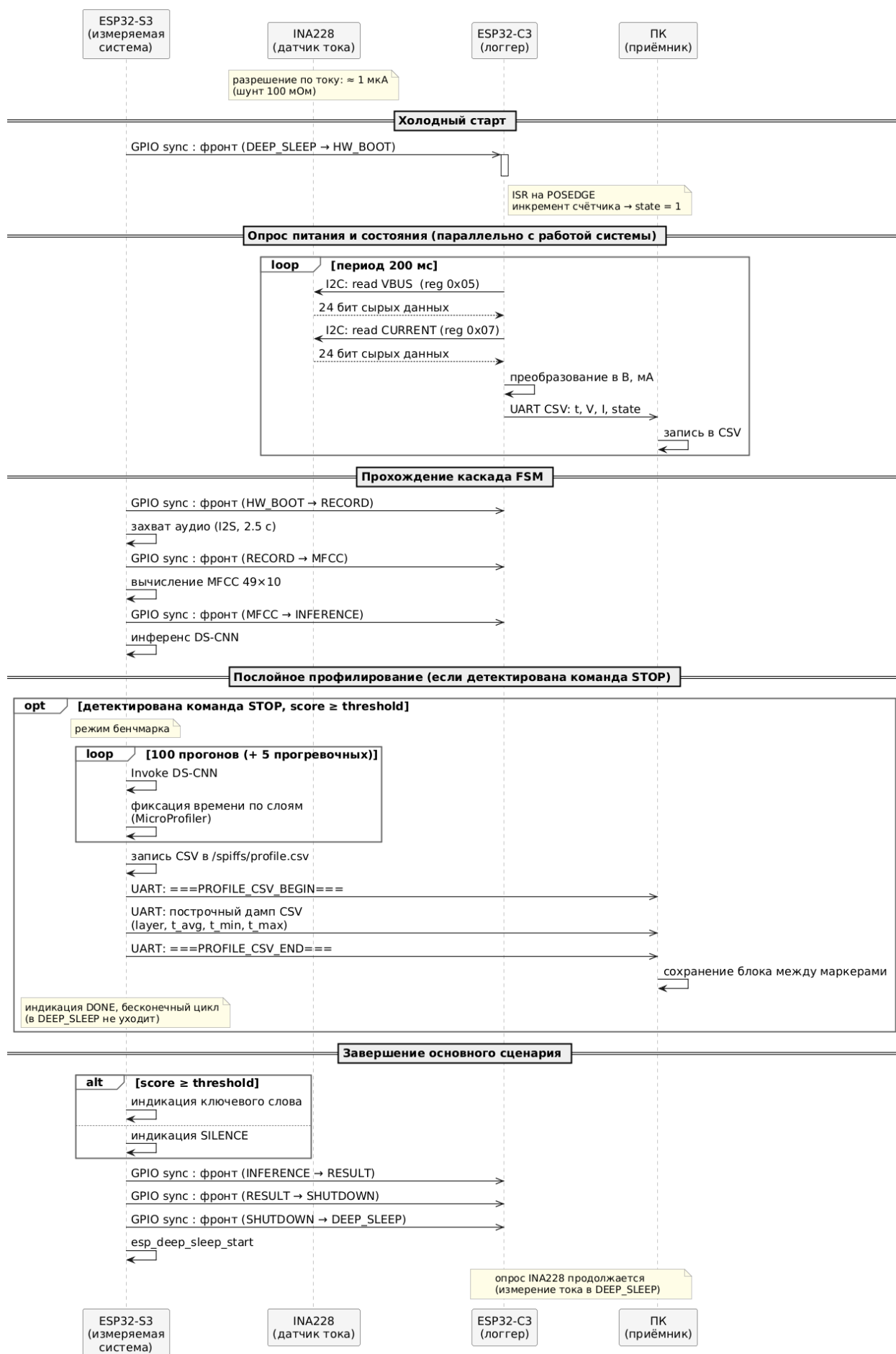


Рисунок 23 — Диаграмма последовательности сбора измерительных данных

109